به نام خدا

دانشگاه تهران

دانشکده مهندسی

برق و کامپیوتر





نام : مهسا ابرغانی اقدم

شماره دانشجویی :

درس VLSI

نام استاد: دکتر وحدت

فهرست

بخش اول ۳
الف) ۳
ب) ۱۵
ج) ۲۳
د) ۴۳
ه) ۴۷
و) ۴۸
بخش دوم ۵۰
بخش سوم ۵۶
الف) ۵۶
ب) ۷۲

مقدمه:

در این تمرین کامپیوتری مدارهای مربوط به دو جمع‌کننده‌ی مختلف را پیاده‌سازی کرده و سپس به کمک نرم‌افزار HSPICE از نظر توان و تأخیر آنها را با یکدیگر مقایسه خواهیم کرد. درگام بعدی تأثیر نوسانات بر عملکرد جمع‌کننده‌ها به کمک تحلیل Monte Carlo بررسی خواهد شد.

بخش اول

در این بخش دو جمع‌کننده‌ی ۸ بیتی Carry Propagation و Kogge Stone را پیاده‌سازی خواهیم کرد. سپس از صحت عملکرد هر دوی آنها مطمئن شده و تأخیر بیشینه و توان استاتیک و دینامیک را برای هر دو جمع‌کننده به دست خواهیم آورد. در نهایت به مقایسه‌ی نتایج به دست آمده خواهیم پرداخت. در طراحی جمع‌کننده‌ها توجه داشته باشید که حتماً سیگنال‌های C_{in} و C_{out} را علاوه بر دو ورودی ۸ بیتی و یک خروجی ۸ بیتی درنظر بگیرید.

الف)

الف) (٪۱۵) ساختار مربوط به یک Carry Propagation Adder (CPA) ۸ بیتی را در HSPICE پیاده‌سازی کنید. در پیاده‌سازی خود حتماً به ماجول محور (Modular) بودن توصیف‌ها اهمیت دهید، زیرمدار (subcircuit)های مورد استفاده را بیان کرده و نحوه‌ی قرارگیری آنها را شرح دهید.

هدف از این بخش، پیاده‌سازی یک جمع‌کننده‌ی ۸ بیتی با ساختار Ripple-Carry (CPA) در نرم‌افزار HSPICE و بررسی توان و تاخیر آن با استفاده از تحلیل گذرا (Transient) و ابزارهای اندازه‌گیری HSPICE است.

ساختار کلی مدار:

برای ساخت جمع‌کننده‌ی کامل ۸ بیتی، ابتدا گیت‌های منطقی پایه شامل **NAND2**، **NOR2**، **INV**، **XOR2**، **AND2** و **OR2** به‌صورت ماژولار طراحی شده‌اند. سپس از این گیت‌ها برای ساختن جمع‌کننده‌ی کامل است. در نهایت، با اتصال ۸ عدد FA به‌صورت سری، مدار اصلی Ripple-Carry Adder ساخته شده است.

پیاده‌سازی گیت‌های منطقی:

NAND2 و **NOR2** به‌صورت ترکیب ترانزیستورهای NMOS و PMOS پیاده‌سازی شده‌اند.

INV با یک NMOS و یک PMOS ساخته شده است.

XOR2 با استفاده از دو معکوس‌کننده و سپس شبکه‌ای از ترانزیستورها برای محاسبه‌ی $A \oplus B$ پیاده‌سازی شده است.

AND2از ترکیب NAND و INV ساخته شده و OR2 نیز از ترکیب NOR و INV ساخته شده‌اند.

طراحی:Full Adder

جمع‌کننده‌ی کامل از دو XOR برای تولید خروجی Sum (برابر$a \oplus b \oplus cin$)و دو AND و یک OR برای تولید خروجی Carry ($ab + cin\ (a \oplus b)$) تشکیل شده است.

طراحی Ripple-Carry ۸ بیتی:

در این مرحله، ۸ عدد Full Adder با استفاده از زیرمدار FA به‌صورت زنجیره‌ای متصل شده‌اند. خروجی Carry هر مرحله به ورودی Carry مرحله‌ی بعدی متصل شده است تا Carry به‌صورت پشت‌سرهم منتقل شود.

تست و شبیه‌سازی:

منبع تغذیه با ولتاژ ۱ ولت تعریف شده است.

برای خطوط A وB ، ۸ بیت ورودی مشخص شده‌اند. برای مثال، Aبرابر **01101111**(از A8 تا A1) و B برابر **10010110**است.

سیگنال Cin نیز مقدار ۰ گرفته است.

جمع‌کننده‌ی ۸ بیتی با دستور **XRCA8**فراخوانی شده و خروجی‌های جمع (**S1**تا **S8**) و خروجی نهایی انتقال **(Cout)** استخراج می‌شود.

آنالیزهای انجام‌شده:

تحلیل گذرا :(Transient Analysis) برای مدت ۲۰۰ nsبا گام زمانی ۱ psانجام شده است.

اندازه‌گیری تأخیر :تأخیر بین بیت A0 و خروجی جمع S7 با دستور **measure delay_S7.**محاسبه شده است.

اندازه‌گیری توان متوسط :توان مصرفی در بازه‌ی ۰ تا ۲۰۰ nsبا دستور **measure avg_power_cpa.**محاسبه شده است.

نکات طراحی:

کلیه‌ی زیرمدارها به‌صورت ماژولار پیاده‌سازی شده‌اند.

تمامی سیگنال‌ها به‌درستی مقداردهی اولیه شده‌اند تا نتایج دقیق‌تری به‌دست آید.

رعایت نام‌گذاری استاندارد برای سیگنال‌ها (ورودی، خروجی، تغذیه) در طراحی حفظ شده است.

***************** bit Ripple-Carry Adder - Main File-۸ ************

lib "crn90g_2d5_lk_v1d2p1.l" TT.

در ابتدای کد، فایل کتابخانه‌ی فناوری CMOS مربوط به تکنولوژی ۹۰ نانومتر توسط دستور lib.بارگذاری می‌شود. این فایل شامل مدل‌های دقیق ترانزیستورها برای شبیه‌سازی واقع‌گرایانه در محیط HSPICE است. سپس سه پارامتر اصلی طراحی تعریف می‌شوند: حداقل طول و عرض ترانزیستورها به ترتیب برابر با ۱۸۰ نانومتر، و ولتاژ تغذیه برابر با ۱ ولت. این مقادیر پایه در سراسر طراحی مدار استفاده شده‌اند تا همخوانی تکنولوژیکی در ابعاد و عملکرد ترانزیستورها حفظ شود.

-- Basic Technology Setup -- ❄

param Lmin=180n.

param Wmin=180n.

param VDD = 1.0.

*****

گیت معکوس‌کننده یا وارون‌گر (NOT) نیز به‌صورت یک زیرمدار ساده پیاده‌سازی شده است. این گیت فقط از یک ترانزیستور NMOSو یک ترانزیستور PMOS استفاده می‌کند. در صورتی که ورودی مقدار ۱ داشته باشد، ترانزیستور NMOS فعال شده و خروجی را به زمین متصل می‌کند؛ در مقابل، اگر ورودی ۰ باشد، ترانزیستور PMOS مسیر اتصال خروجی به منبع تغذیه (VDD) را باز می‌کند. نتیجه‌ی این طراحی، معکوس شدن سیگنال ورودی است.

```
--- Simple Inverter (NOT gate) --- ❋

SUBCKT  INV in out vdd vss.

M1  out  in  vss  vss  nch  L=Lmin  W=Wmin

 M2  out  in  vdd vdd pch L=Lmin  W='2*Wmin

ENDS INV.
```

گیت XOR دو ورودی (XOR2) یکی از بخش‌های مهم طراحی مدار جمع‌کننده است، زیرا در محاسبه‌ی بیت‌های حاصل‌جمع استفاده می‌شود. این گیت با ترکیب چندین ترانزیستور و دو گیت معکوس‌کننده داخلی ساخته شده است. ابتدا سیگنال‌های مکمل (معکوس‌شده) A و B با استفاده از گیت‌های INV تولید می‌شوند، سپس با طراحی منطقی خاصی، عملیات $A \oplus B$ پیاده‌سازی می‌شود. این گیت فقط زمانی خروجی ۱ تولید می‌کند که یکی از ورودی‌ها ۱ و دیگری ۰ باشد.

```
--- input XOR Gate-۲ --- ❋

SUBCKT XOR2 A B out vdd vss.

Xinv1 A Abar vdd vss INV

Xinv2 B Bbar vdd vss INV

 M1 net1 A Vdd Vdd pch L=Lmin W='2*Wmin

 M2 out Bbar net1 Vdd pch L=Lmin W='2*Wmin

 M3 net2 Abar Vdd Vdd pch L=Lmin W='2*Wmin

 M4 out B net2 Vdd pch L=Lmin W='2*Wmin

 M5 out A net3 vss nch L=Lmin W='Wmin

 M6 net3 B vss vss nch L=Lmin W='Wmin

 M7 out Abar net4 vss nch L=Lmin W='Wmin

 M8 net4 Bbar vss vss nch L=Lmin W='Wmin

ends XOR2.
```

گیت AND و OR نیز به‌صورت غیرمستقیم طراحی شده‌اند. برای گیتAND ، ابتدا از گیت NAND استفاده می‌شود و سپس خروجی آن توسط یک گیت INV معکوس می‌شود تا عملکرد AND حاصل گردد. برای گیت OR نیز ابتدا از گیت NOR

استفاده شده و خروجی آن معکوس می‌شود. این روش، علاوه بر سادگی، باعث می‌شود بتوان از گیت‌های قبلی مجدداً استفاده کرد و از تکرار اجزای فیزیکی جلوگیری نمود.

****

****

--- *AND2 gate built using NAND + INV---

SUBCKT AND2 A B out vdd vss.

X1 A B net1 vdd vss NAND2

X2 net1 out vdd vss INV

ENDS AND2.

****

--- OR2 gate implemented via NOR + INV --- *

SUBCKT OR2 A B out vdd vss.

X1 A B net1 vdd vss NOR2

X2 net1 out vdd vss INV

ENDS OR2.

جمع‌کننده‌ی کامل یا Full Adder (FA) سه ورودی دارد: دو بیت A و B و سیگنال انتقال از مرحله‌ی قبل .(Cin) برای محاسبه‌ی خروجی Sum از دو گیت XOR استفاده شده است؛ ابتدا A⊕B محاسبه شده و سپس با Cin دوباره XOR می‌شود. برای تولید Carry Out نیز دو مسیر موازی AND ساخته می‌شوند: یکی برای ab و دیگری برای cin(a⊕b) ، سپس نتایج با گیت OR ترکیب می‌شوند. این ساختار هم دقیق و هم کارآمد است و به‌خوبی مدل منطقی FA را پیاده‌سازی می‌کند.

********** *******

--- Full Adder using XOR, AND, and OR gates --- *

SUBCKT FA a b cin sum cout vdd vss.

Sum output: a XOR b XOR cin *

X1 a b net1 vdd vss XOR2

X2 net1 cin sum vdd vss XOR2

Carry output: ab + cin(a⊕b) *

X3 a b net2 vdd vss AND2

X4 net1 cin net3 vdd vss AND2

X5 net2 net3 cout vdd vss OR2

ENDS FA.

در این بخش، جمع‌کننده‌ی Ripple-Carry با اتصال سری ۸ عدد Full Adder ساخته می‌شود. خروجی Carry هر FA به ورودی Carry مرحله‌ی بعد متصل شده است، که همین موضوع باعث Ripple شدن انتقال‌ها می‌شود. ورودی‌ها از A1 تا A8 و B1تا B8 در نظر گرفته شده‌اند و بیت‌های حاصل‌جمع از S1 تا S8 خروجی می‌شوند. همچنین، خروجی انتقال نهایی (Cout) پس از FA آخر تولید می‌شود. این ساختار ساده اما تأخیرپذیر است، زیرا هر بیت منتظر انتقال مرحله‌ی قبل می‌ماند.

****

--- Top-level 8-bit Ripple-Carry Adder --- *

SUBCKT RCA8 A8 A7 A6 A5 A4 A3 A2 A1 B8 B7 B6 B5 B4 B3 B2 B1 Cin S8 S7 S6 S5 S4 S3 S2 S1 Cout . vdd vss

XFA1 A1 B1 Cin S1 C2 vdd vss FA

XFA2 A2 B2 C2 S2 C3 vdd vss FA

XFA3 A3 B3 C3 S3 C4 vdd vss FA

XFA4 A4 B4 C4 S4 C5 vdd vss FA

XFA5 A5 B5 C5 S5 C6 vdd vss FA

XFA6 A6 B6 C6 S6 C7 vdd vss FA

XFA7 A7 B7 C7 S7 C8 vdd vss FA

XFA8 A8 B8 C8 S8 Cout vdd vss FA

ENDS RCA8.

در این قسمت، یک بستر تست برای شبیه‌سازی مدار تعریف شده است. منابع تغذیه با ولتاژ ۱ ولت و زمین مشخص می‌شوند. سپس ۸ بیت برای A و ۸ بیت برای B مقداردهی اولیه می‌شوند. برای مثال، بیت‌های A ترکیبی از صفر و یک هستند و B نیز به‌صورت دلخواه تعریف شده‌اند. سیگنال Cin نیز مقدار صفر گرفته است تا شرایط بدون انتقال اولیه شبیه‌سازی شود. این ورودی‌ها جهت بررسی عملکرد صحیح مدار در حالت خاص استفاده می‌شوند.

****

--- Simulation Testbench --- *

Vdd vdd 0 DC 'VDD

Vss vss 0 DC 0

VA1 A1 0 DC 0

VA2 A2 0 DC 1

VA3 A3 0 DC 0

VA4 A4 0 DC 1

VA5 A5 0 DC 1

VA6 A6 0 DC 1

VA7 A7 0 DC 1

VA8 A8 0 DC 1

VB1 B1 0 DC 1

VB2 B2 0 DC 0

VB3 B3 0 DC 0

VB4 B4 0 DC 1

VB5 B5 0 DC 0

VB6 B6 0 DC 1

VB7 B7 0 DC 1

VB8 B8 0 DC 0

VCin Cin 0 DC 0

❋Input vector file

.❋VEC 'DVF1.vec

در انتهای کد، مدار RCA8 که قبلاً طراحی شده بود، با ورودی‌ها و خروجی‌ها فراخوانی می‌شود. سپس تحلیل گذرای مدار با استفاده از دستور .tranبرای ۲۰۰ نانوثانیه با گام زمانی ۱ پیکوثانیه اجرا می‌شود. به کمک دستور measure.، تأخیر سیگنال خروجی S7 نسبت به A0 اندازه‌گیری می‌شود. همچنین، میانگین توان مصرفی کل مدار در بازه‌ی شبیه‌سازی محاسبه می‌شود. در نهایت، با دستور option post=2.خروجی‌ها برای ترسیم نمودارها ذخیره می‌شوند.

Instantiate the 8-bit RCA ❋

XRCA8 A8 A7 A6 A5 A4 A3 A2 A1 B8 B7 B6 B5 B4 B3 B2 B1 Cin S8 S7 S6 S5 S4 S3 S2 S1 Cout vdd vss RCA8

Run transient simulation for 200ns with 1ps steps ❋

```
tran 1ps 200n.

Measure the delay from input A0 to output S7 *

measure tran delay_S7 TRIG v(A0) VAL='Vdd/2' RISE=1 TARG v(S7) VAL='Vdd/2' RISE=1.

Measure average power consumption *

measure tran avg_power_cpa AVG power FROM=0 TO=200n.

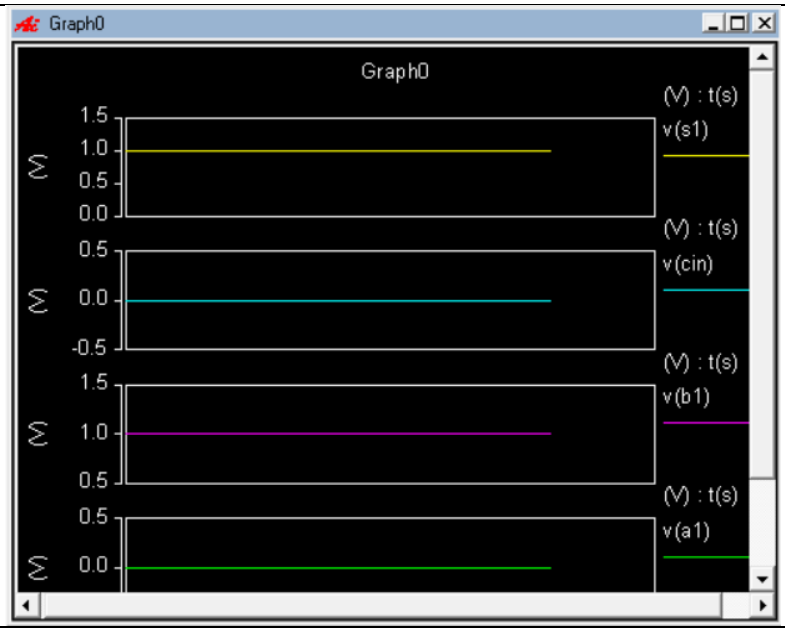
option post = 2.

end.
```

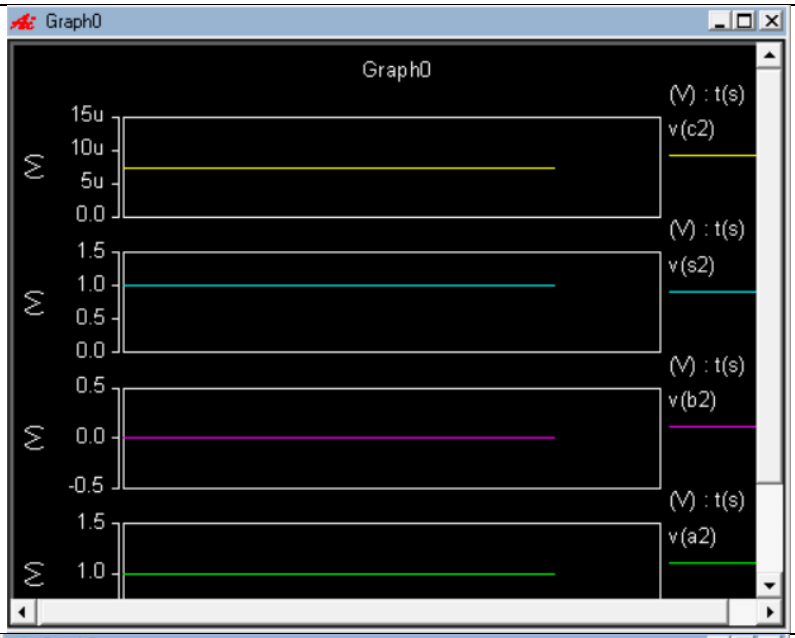
همچنین فایل اجرا شده ی آن در نرم افزار به نام CRA.lis نیز قرار گرفته و در ادامه نیز شکل موج ها مورد تحلیل قرار می گیرد:

خروجی شکل موج

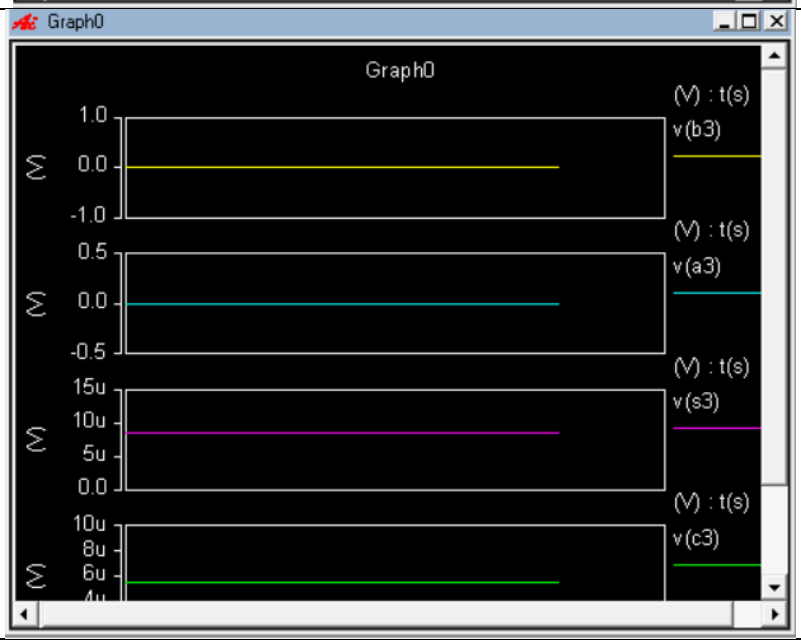
در cosmosScope فایل CRA.tr0 را باز کرده و سیگنال های مورد نیاز را اضافه میکنیم و آنرا تحلیل میکنیم:



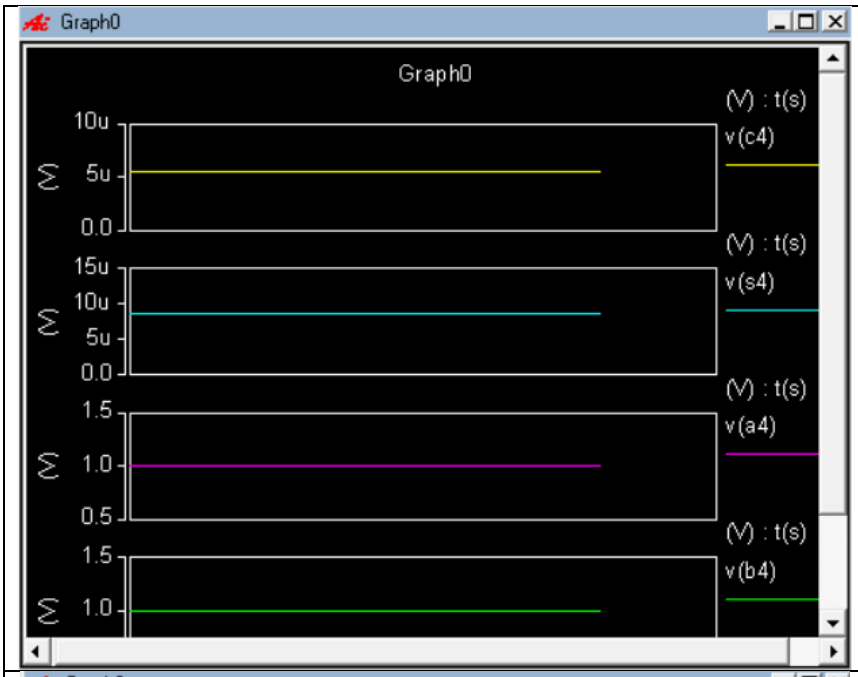
در این نمودار، ورودی‌های بیت اول عبارت‌اند از `a1 = 0`، `b1 = 1` و `cin = 0` که مجموع آن‌ها برابر با ۱ شده است. بنابراین، خروجی جمع `s1 = 1` و خروجی carry برابر با ۰ است. این carry به‌عنوان `c2` به بیت دوم منتقل می‌شود. سیگنال‌ها پایدارند و عملکرد مدار در این مرحله صحیح است.



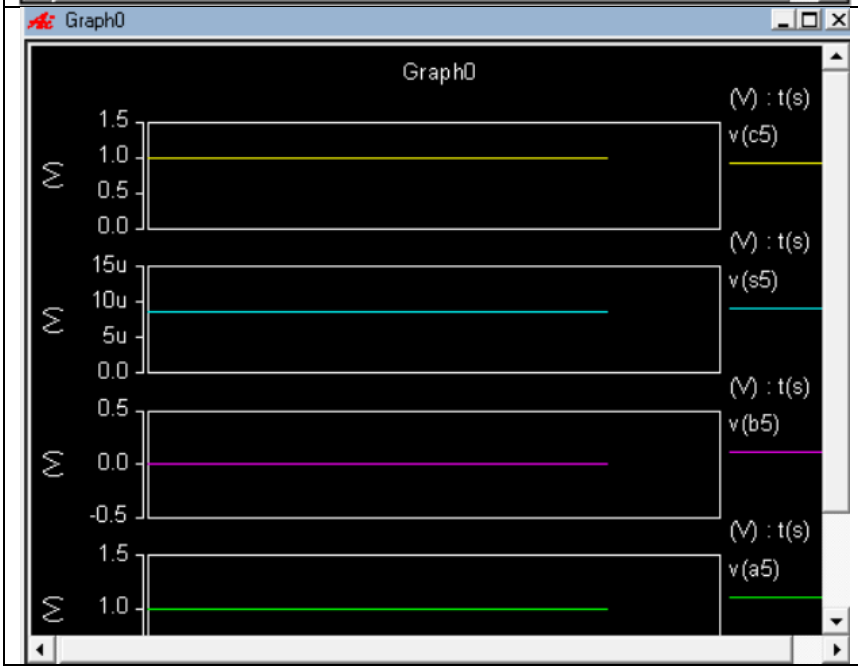
در این نمودار مربوط به بیت دوم مدار Ripple-Carry Adder، ورودی‌ها برابرند با `a2 = 1`، `b2 = 0` و carryورودی `c1 = 0`(که از مرحله قبل آمده است). مجموع این سه مقدار برابر با ۱ شده، بنابراین خروجی حاصل‌جمع `s2 = 1` و خروجی carryاین مرحله (`c3`) برابر با ۰ است. این carry به بیت سوم منتقل خواهد شد. سیگنال‌ها پایدار هستند و عملکرد مدار در این مرحله به‌درستی انجام شده است.



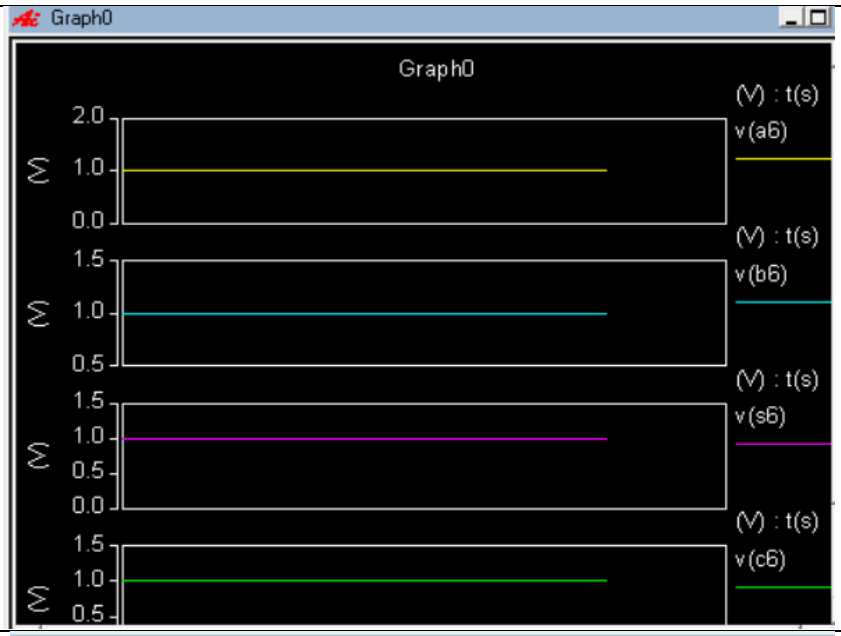
در این نمودار که مربوط به **بیت سوم** در مدار Ripple-Carry Adder است، ورودی‌ها `a3 = 0`، `b3 = 0` و carry ورودی `c3 = 0`هستند. جمع این سه مقدار برابر با صفر شده، بنابراین خروجی حاصل‌جمع `s3 = 0`و خروجی carry این مرحله که به‌عنوان **c4**به بیت چهارم منتقل می‌شود، نیز برابر با صفر است. همه‌ی سیگنال‌ها در طول زمان پایدار هستند و صحت عملکرد مدار در این مرحله را نشان می‌دهند.



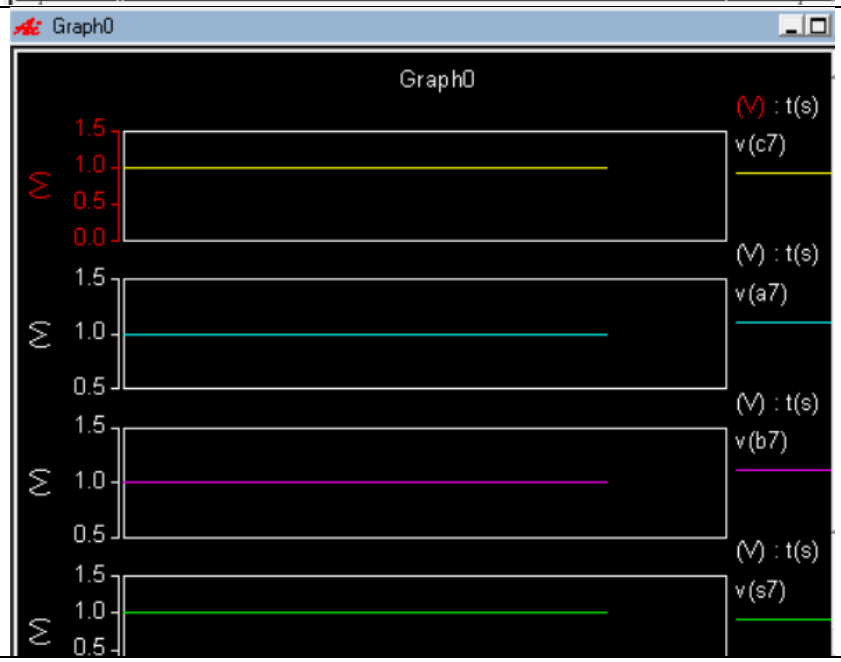
در این نمودار مربوط به **بیت چهارم** مدار Ripple-Carry Adder ، ورودی‌ها `a4 = 1`، `b4 = 1` و ورودی carry `c4 = 0` هستند. مجموع این سه مقدار برابر با ۲ می‌شود، بنابراین خروجی حاصل‌جمع `s4 = 0` و خروجی carry که به‌عنوان **c5** به بیت پنجم منتقل می‌شود، برابر با ۱ است. این نتیجه با منطق مدار جمع‌کننده کاملاً مطابقت دارد و پایداری سیگنال‌ها نشان‌دهنده‌ی عملکرد درست مدار در این مرحله است.



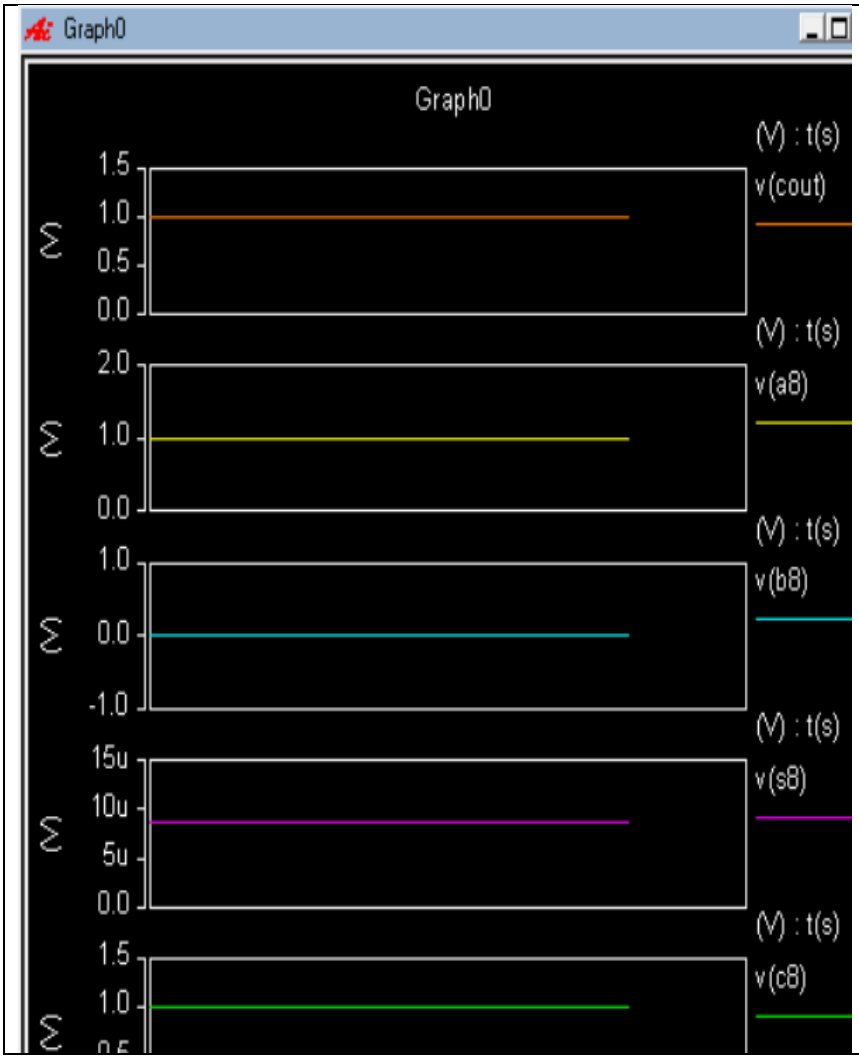
در این نمودار مربوط به **بیت پنجم** مدار Ripple-Carry Adder ، ورودی‌ها `a5 = 1`، `b5 = 0` و ورودی carry `c5 = 1` هستند. مجموع این سه مقدار برابر با ۲ می‌شود، بنابراین خروجی حاصل‌جمع `s5 = 0` و خروجی carry که به‌عنوان **c6** به بیت ششم منتقل می‌شود، برابر با ۱ است. سیگنال‌ها پایدار هستند و نتایج مطابق انتظار و منطق جمع دودویی می‌باشند.



در این نمودار مربوط به **بیت ششم** مدار Ripple-Carry Adder ، ورودی‌ها `a6 = 1` ،`b6 = 1` و carry ورودی `c6 = 1` هستند. مجموع این سه مقدار برابر با ۳ می‌شود، بنابراین خروجی حاصل‌جمع `s6 = 1` و خروجی carry که به‌عنوان **c7** به بیت هفتم منتقل می‌شود، نیز برابر با ۱ است. این نتیجه با منطق جمع دودویی کاملاً تطابق دارد و سیگنال‌ها پایدار و بدون نویز هستند.



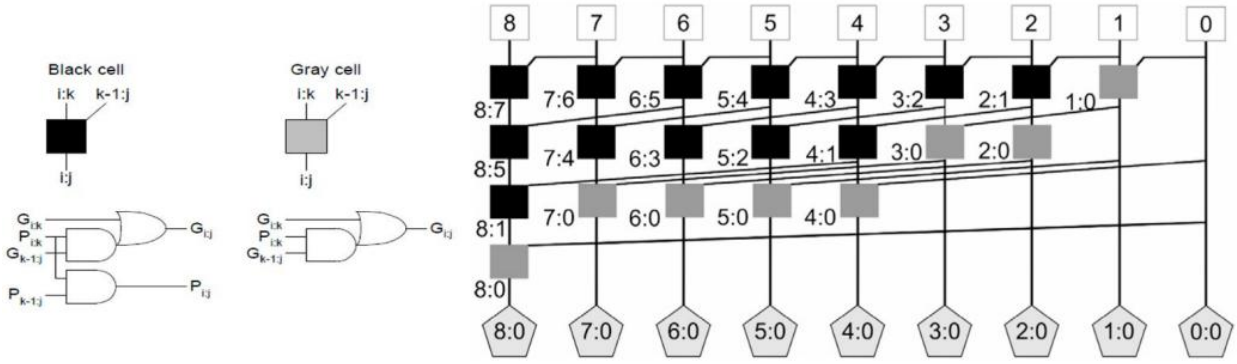
در این نمودار مربوط به **بیت هفتم** در مدار Ripple-Carry Adder ، ورودی‌ها `a7 = 1` ،`b7 = 1` و carry ورودی `c7 = 1` هستند. مجموع این سه مقدار برابر با ۳ شده، بنابراین خروجی حاصل‌جمع `s7 = 1` و خروجی carry این مرحله که به‌عنوان **c8** یا همون `Cout` به بیت هشتم منتقل می‌شود، برابر با ۱ است. خروجی‌ها کاملاً پایدار هستند و عملکرد صحیح مدار را در این مرحله نشان می‌دهند.



در این نمودار که مربوط به **بیت هشتم** و خروجی نهایی مدار Ripple-Carry Adder است، ورودی‌ها `a8 = 1`، `b8 = 0` و carry ورودی `c8 = 1` هستند. مجموع این سه مقدار برابر با ۲ شده، بنابراین خروجی حاصل‌جمع `s8 = 0` و **خروجی نهایی carry یعنی `cout = 1`** به درستی تولید شده‌اند. این carry نشان‌دهنده‌ی overflow جمع در بیت هشتم است. سیگنال‌ها در طول زمان کاملاً پایدارند و نتیجه مطابق منطق جمع دودویی می‌باشد.

ب)

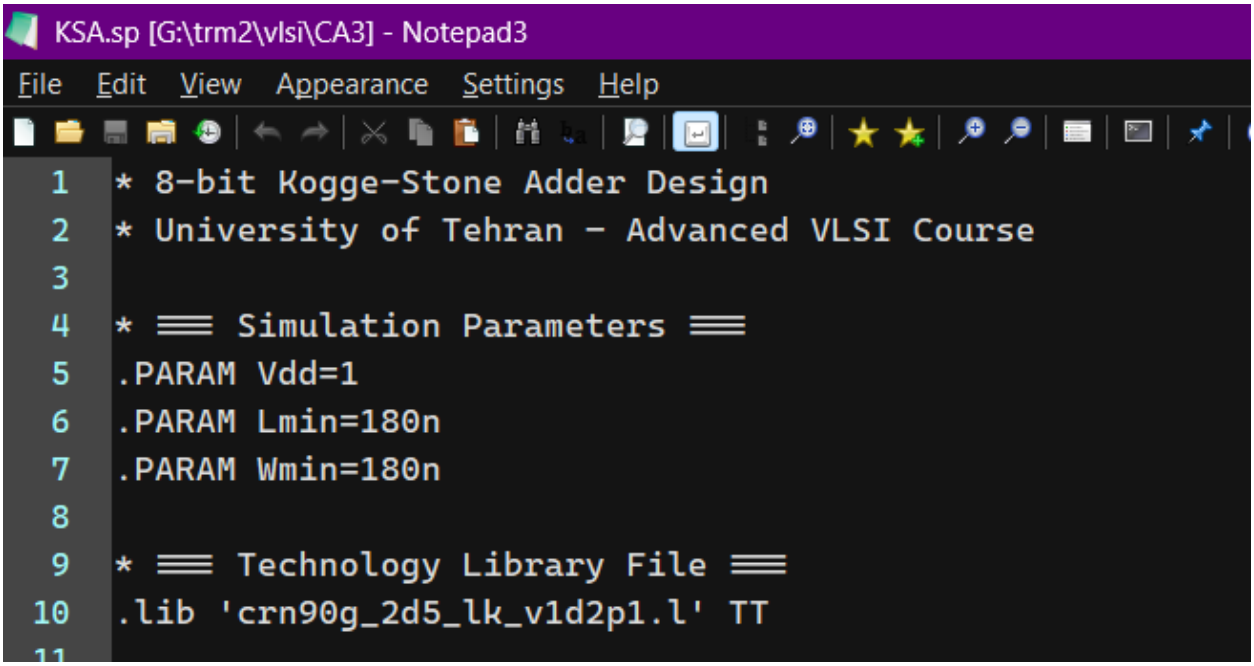
ب) (٪۱۵) ساختار مربوط به Kogge Stone Adder ۸ بیتی را در HSPICE پیاده‌سازی کنید. در پیاده‌سازی خود حتماً به ماجول محور (ماجولار) بودن توصیف‌ها اهمیت دهید، زیرمدارهای مورد استفاده را بیان کرده و نحوه‌ی قرارگیری آنها را شرح دهید. شکل ۱ شماتیک مدار این جمع‌کننده را نمایش می‌دهد.



شکل ۱ : شماتیک مدار Kogge Stone Adder ۸ بیتی

این جمع‌کننده یکی از معروف‌ترین الگوریتم‌های جمع موازی است که با استفاده از درخت پیشوندی (Prefix Tree) محاسبه‌ی سریع Carry را ممکن می‌سازد. پیاده‌سازی به‌صورت ماژولار انجام شده است و هر بلوک منطقی مثل Inverter ، XOR، AND و OR به‌صورت زیرمدار تعریف شده‌اند. سپس از این زیرمدارها برای ساخت بلوک‌های مهمی همچون Black Cell و Gray Cell استفاده شده است. شکل ۱ ساختار شماتیکی این جمع‌کننده را نشان می‌دهد.

در ابتدای کد، ولتاژ تغذیه مدار برابر با ۱ ولت تعیین شده است. همچنین حداقل طول و عرض ترانزیستورها به ترتیب ۱۸۰ نانومتر مشخص شده است. سپس کتابخانه‌ی فناوری CMOS مربوط به تکنولوژی ۹۰ نانومتر بارگذاری می‌شود که مدل‌های دقیق ترانزیستورها را در اختیار شبیه‌ساز قرار می‌دهد.



در این بخش، گیت‌های منطقی پایه به‌صورت زیرمدار تعریف شده‌اند:

⁎Simple inverter for NOT operation .SUBCKT INV in out vdd gnd M1 out in gnd gnd nch L=Lmin W=Wmin M2 out in vdd vdd pch L=Lmin W='Wmin*2 .ENDS INV	این گیت دارای یک ورودی و یک خروجی است. در صورتی که ورودی برابر با ۱ باشد، ترانزیستور NMOS روشن شده و خروجی را به زمین وصل می‌کند (خروجی = ۰). در غیر این صورت، ترانزیستور PMOS روشن شده و خروجی را به ولتاژ VDDمتصل می‌کند (خروجی = ۱). این گیت ساده‌ترین گیت منطقی است و در پیاده‌سازی سایر گیت‌ها مانندXOR ، ANDو OR نیز به کار می‌رود.
* Standard NAND gate (2-input) .SUBCKT NAND2 inA inB out vdd gnd M1 out inA vdd vdd pch L=Lmin W='Wmin*2' M2 out inB vdd vdd pch L=Lmin W='Wmin*2' M3 out inA net1 gnd nch L=Lmin W=Wmin M4 net1 inB gnd gnd nch L=Lmin W=Wmin .ENDS NAND2	گیت NAND از دو PMOS که به صورت موازی قرار گرفته‌اند و دو NMOS که به صورت سری بسته شده‌اند تشکیل شده است. خروجی این گیت تنها زمانی صفر می‌شود که هر دو ورودی ۱ باشند. در غیر این صورت، خروجی همیشه ۱ خواهد بود. این گیت به‌دلیل ساختار ساده‌تر نسبت به AND، در طراحی CMOS بسیار کاربرد دارد.
* Standard NOR gate (2-input) .SUBCKT NOR2 inA inB out vdd gnd M1 out inA gnd gnd nch L=Lmin W=Wmin M2 out inB gnd gnd nch L=Lmin W=Wmin M3 out inA net1 vdd pch L=Lmin W='Wmin*2' M4 net1 inB vdd vdd pch L=Lmin W='Wmin*2' .ENDS NOR2	در گیتNOR ، برعکسNAND ، ترانزیستورهای NMOS موازی و ترانزیستورهای PMOS به‌صورت سری هستند. این گیت فقط زمانی خروجی ۱ تولید می‌کند که هر دو ورودی صفر باشند. به‌عنوان مثال، برای ورودی‌های A=0 وB=0 ، خروجی = ۱ خواهد بود. در سایر حالات، خروجی صفر می‌شود.

این گیت با استفاده از خروجی گیت NAND و سپس عبور آن از یک معکوس‌کننده (NOT) ساخته شده است. این ترکیب عملکرد گیت AND را بازتولید می‌کند؛ یعنی خروجی فقط زمانی ۱ خواهد بود که هر دو ورودی ۱ باشند. این ساختار به دلیل استفاده مجدد از گیت‌های تعریف‌شده، ماژولار و قابل توسعه است.	*** AND gate constructed from NAND + inverter** **.SUBCKT AND2 inA inB out vdd gnd** **XU1 inA inB nand_out vdd gnd NAND2** **XU2 nand_out out vdd gnd INV** **.ENDS AND2**
گیت OR نیز به‌صورت مشابه گیت AND ساخته شده است. ابتدا دو ورودی به گیت NOR داده می‌شوند، سپس خروجی آن از یک معکوس‌کننده عبور داده می‌شود. نتیجه نهایی عملکرد گیت OR را ارائه می‌دهد: یعنی اگر هر یک از ورودی‌ها ۱ باشد، خروجی برابر ۱ خواهد بود.	*** OR gate built from NOR + inverter** **.SUBCKT OR2 inA inB out vdd gnd** **XU1 inA inB nor_out vdd gnd NOR2** **XU2 nor_out out vdd gnd INV** **.ENDS OR2**
این گیت پیچیده‌ترین گیت منطقی در این مجموعه است و به‌صورت کامل با استفاده از ترانزیستورهای CMOS پیاده‌سازی شده است. ابتدا سیگنال‌های مکمل Abar و Bbar تولید می‌شوند. سپس چهار ترانزیستور PMOS در ساختار موازی و چهار ترانزیستور NMOS در ساختار سری به کار گرفته می‌شوند تا در نهایت خروجی out = A XOR B تولید شود. خروجی این گیت تنها زمانی ۱ است که یکی از ورودی‌ها ۱ و دیگری ۰ باشد.	*** 2-input XOR gate implemented using CMOS logic** **.SUBCKT XOR2 A B out vdd gnd** **Xinv1 A Abar vdd gnd INV** **Xinv2 B Bbar vdd gnd INV** **M1 net1 A Vdd Vdd pch L=Lmin W='2*Wmin'** **M2 out Bbar net1 Vdd pch L=Lmin W='2*Wmin'** **M3 net2 Abar Vdd Vdd pch L=Lmin W='2*Wmin'** **M4 out B net2 Vdd pch L=Lmin W='2*Wmin'** **M5 out A net3 gnd nch L=Lmin W=Wmin** **M6 net3 B gnd gnd nch L=Lmin W=Wmin** **M7 out Abar net4 gnd nch L=Lmin W=Wmin** **M8 net4 Bbar gnd gnd nch L=Lmin W=Wmin** **.ends XOR2**

در معماری Kogge-Stone ، برای رسیدن به محاسبه‌ی سریع جمع دو عدد، ساختار درختی خاصی طراحی می‌شود که هدف آن کاهش زمان محاسبه‌ی سیگنال‌های انتقال (Carry) است. برای این منظور، چهار زیرمدار کلیدی تعریف شده‌اند:

Black Cell ، Grey Cell، Pre-Processing وSum Block . در این بخش، هر کدام از این زیرمدارها هم از نظر **عملکرد منطقی** و هم از نظر **نقش آن در کل مدار** توضیح داده می‌شوند.

Black Cell یکی از مهم‌ترین اجزای ساختار درخت پیشوندی است. این سلول وظیفه‌ی ترکیب اطلاعات انتقال و انتشار از دو سطح مختلف را بر عهده دارد. به‌عبارت ساده‌تر، Black Cell دو جفت سیگنال (G, P) از دو مرحله‌ی قبلی را دریافت می‌کند و آن‌ها را با استفاده از منطق پیشوندی به یک جفت جدید (G, P) ترکیب می‌کند. این سلول دو وظیفه دارد: محاسبه‌ی سیگنال **Gi:j**: $G_{i:j} = G_{i:k} + P_{i:k}.G_{k-1:j}$ که به کمک یک AND و یک OR پیاده‌سازی شده است.	***Black Cell: Computes both group generate (G) and propagate (P)** **.SUBCKT BLACK_CELL Gi_j Pi_j Gk_i_1 Pk_i_1 Gi_k Pi_k vdd gnd** ***Output G = Gi_k + Pi_k.Gk_i_1** ***Output P = Pi_k . Pk_i_1** **XU1 Pi_k Gk_i_1 and_out vdd gnd AND2** **XU2 Gi_k and_out Gi_j vdd gnd OR2** **XU3 Pi_k Pk_i_1 Pi_j vdd gnd AND2** **.ENDS BLACK_CELL**

محاسبه‌ی سیگنال **Pi:j**: $P_{i:j} = P_{i:k} \cdot P_{k-1:j}$ که با یک AND ساده ساخته می‌شود. این زیرمدار سه زیرگیت استفاده می‌کند: دو AND2 و یک OR2، و خروجی نهایی را روی Gi_jو Pi_jقرار می‌دهد. سلول مشکی معمولاً در **لایه‌های میانی درخت پیشوندی** مورد استفاده قرار می‌گیرد، جایی که هر دو سیگنال G و P لازم هستند تا در مراحل بالاتر نیز بتوان سیگنال‌های جدیدی تولید کرد. دلیل اصلی استفاده از Black Cell این است که در گسترش پیشوندی، باید بتوان هم سیگنال انتقال و هم انتشار را به‌روز کرد تا در ادامه‌ی درخت به کار گرفته شوند.	
برخلافBlack Cell ، Grey Cellفقط خروجی **G**را محاسبه می‌کند و هیچ P تولید نمی‌کند. از این سلول معمولاً در ستون آخر درخت پیشوندی استفاده می‌شود، جایی که فقط سیگنال carry اهمیت دارد. رابطه محاسباتی آن: $G_{i:j} = G_{i:k} + P_{i:k}.G_{k-1:j}$ این زیرمدار نیز شامل یک AND2 و یک OR2 است و خروجی نهایی را در Gi_jقرار می‌دهد. Grey Cellمعمولاً در **لایه‌های نهایی درخت پیشوندی** به کار می‌رود، جایی که فقط سیگنال انتقال (G) اهمیت دارد و دیگر نیازی به سیگنال انتشار (P) نیست. به‌ویژه، برای تولید سیگنال carry نهایی مانندCout ، از Grey Cell استفاده می‌شود. هدف اصلی این سلول، ساده‌سازی مدار و کاهش تأخیر در مسیر خروجی است.	⁕**Grey Cell: Only updates group generate (G) signal** .**SUBCKT GREY_CELL Gi_j Gk_i_1 Pk_i Gi_k vdd gnd** ⁕**Output G = Gi_k + Pk_i.Gk_i_1** **XU1 Pk_i Gk_i_1 and_out vdd gnd AND2** **XU2 Gi_k and_out Gi_j vdd gnd OR2** .**ENDS GREY_CELL**
این بلوک ابتدایی‌ترین مرحله در جمع Kogge-Stone است. برای هر بیت از A وB ، دو سیگنال مهم استخراج می‌شود: **P = A XOR B**: نشان‌دهنده‌ی توانایی انتشارcarry **G = A AND B**: نشان‌دهنده‌ی تولید carry در آن بیت این دو مقدار ورودی اصلی به درخت پیشوندی هستند و برای هشت بیت، این بلوک هشت بار استفاده می‌شود. بلوک پیش‌پردازش برای هر بیت از ورودی‌های A و B **به‌صورت موازی و مستقل** اجرا می‌شود. این ویژگی باعث می‌شود که این مرحله بسیار سریع انجام شود و گلوگاه زمانی در محاسبات carry ایجاد نکند. این ویژگی به‌خصوص برای طراحی‌های سریع و مقیاس‌پذیر اهمیت دارد.	⁕**Pre-Processing: Generates initial P and G signals from A, B** .**SUBCKT PRE_PROCESS A B P G vdd gnd** **XU1 A B P vdd gnd XOR2** **XU2 A B G vdd gnd AND2** .**ENDS PRE_PROCESS**

در مرحله‌ی نهایی جمع، بیت‌های حاصل‌جمع با رابطه‌ی زیر محاسبه می‌شوند: $S_i = P_i \oplus C_{i-1}$ که در آن: • Pi از Pre-Processing آمده است. • C_{i-1} از درخت carry به‌دست آمده است. • خروجی S نیز بیت حاصل‌جمع نهایی است. این بلاک تنها از یک XOR2 برای محاسبه استفاده می‌کند. Sum Block **آخرین مرحله در فرآیند جمع** است و چون تمامی سیگنال‌های carry از قبل محاسبه شده‌اند، این مرحله باید **با حداقل تأخیر** انجام شود. به همین دلیل فقط از یک گیت XOR استفاده می‌شود تا بیت حاصل‌جمع نهایی را با سرعت بالا تولید کند. طراحی ساده‌ی این مرحله باعث می‌شود کل جمع‌کننده عملکرد سریع‌تری داشته باشد.	**❄Sum Generator: Computes final sum from propagate and carry** **.SUBCKT SUM_BLOCK P C S vdd gnd** **XU1 P C S vdd gnd XOR2** **.ENDS SUM_BLOCK**

مدار اصلی جمع‌کننده Kogge-Stone هشت‌بیتی(KSA8)

مدار KSA8مسئول جمع‌کردن دو عدد ۸ بیتی است که ورودی‌های آن شامل A1تا A8، B1تا B8و Cinاست و خروجی‌های آن S1تا S8و Coutمی‌باشند. طراحی این مدار شامل چند مرحله اصلی است که هر مرحله در ادامه توضیح داده می‌شود.

مرحله اول: تولید سیگنال‌های اولیه P و-Pre(G Processing) در ابتدای این مدار، برای هر بیت از ورودی‌های A وB ، یک ماژول PRE_PROCESSفراخوانی می‌شود که دو سیگنال مهم را تولید می‌کند: **$P = A \oplus B$**: سیگنال propagate **$G = A \times B$** : سیگنال generate این سیگنال‌ها برای هر بیت به‌صورت جداگانه و موازی تولید می‌شوند. به عنوان مثال، XPP1 A1 B1 P1 G1 یعنی برای بیت اول ورودی‌ها، سیگنال‌های P1 و G1 محاسبه می‌شوند. این مرحله پایه‌ی شکل‌گیری درخت Carry در-Kogge Stoneاست.	*** === Top-Level: 8-bit Kogge-Stone Adder ===** **.SUBCKT KSA8 A8 A7 A6 A5 A4 A3 A2 A1 B8 B7 B6 B5 B4 B3 B2 B1 Cin S8 S7 S6 S5 S4 S3 S2 S1 Cout vdd gnd** *** Step 1: Pre-processing to generate P and G for each bit** **XPP1 A1 B1 P1 G1 vdd gnd PRE_PROCESS** **XPP2 A2 B2 P2 G2 vdd gnd PRE_PROCESS** **XPP3 A3 B3 P3 G3 vdd gnd PRE_PROCESS** **XPP4 A4 B4 P4 G4 vdd gnd PRE_PROCESS** **XPP5 A5 B5 P5 G5 vdd gnd PRE_PROCESS** **XPP6 A6 B6 P6 G6 vdd gnd PRE_PROCESS** **XPP7 A7 B7 P7 G7 vdd gnd PRE_PROCESS** **XPP8 A8 B8 P8 G8 vdd gnd PRE_PROCESS**
مرحله دوم: اعمال Cin با استفاده ازGrey Cell از آن‌جایی که Cin برای محاسبه‌ی بیت اول جمع استفاده می‌شود، باید به عنوان یک **Gاولیه** در نظر گرفته شود.	*** Handle carry-in: treat Cin as G0** *** Pass Cin through first Grey Cell for position 1** **XGC0 C1 Cin P1 G1 vdd gnd GREY_CELL**

	بنابراین، یک GREY_CELL با ورودی Cin ، P1 و G1 فراخوانی می‌شود تا **C1** محاسبه شود. این همان Carry برای بیت دوم است و با XGC0 مشخص شده.
*** Step 2: First stage of prefix tree (span 2)** **XBC1_1 G2_1 P2_1 G1 P1 G2 P2 vdd gnd BLACK_CELL** **XBC1_2 G3_2 P3_2 G2 P2 G3 P3 vdd gnd BLACK_CELL** **XBC1_3 G4_3 P4_3 G3 P3 G4 P4 vdd gnd BLACK_CELL** **XBC1_4 G5_4 P5_4 G4 P4 G5 P5 vdd gnd BLACK_CELL** **XBC1_5 G6_5 P6_5 G5 P5 G6 P6 vdd gnd BLACK_CELL** **XBC1_6 G7_6 P7_6 G6 P6 G7 P7 vdd gnd BLACK_CELL** **XBC1_7 G8_7 P8_7 G7 P7 G8 P8 vdd gnd BLACK_CELL**	مرحله سوم: لایه اول درخت پیشوندی (Span = 2) در این مرحله، با استفاده از BLACK_CELL سیگنال‌های G و P جدید برای جفت‌های متوالی محاسبه می‌شوند. مثلاً XBC1_1 سیگنال‌های G2_1 و P2_1 را از ترکیب G2، P2، G1 و P1 تولید می‌کند. این مرحله باعث گسترش carry به دو بیت جلوتر می‌شود. در اینجا، G و P برای بیت‌های ۲ تا ۸ نسبت به بیت‌های قبلی به‌روزرسانی می‌شوند.
*** Step 3: Second stage (span 4)** *** Intermediate carry calculations using Grey + Black cells** **XGC2_1 C2 G1 P2_1 G2_1 vdd gnd GREY_CELL** **XGC2_2 C3 G2_1 P3_2 G3_2 vdd gnd GREY_CELL** **XBC2_3 G4_1 P4_1 G2_1 P2_1 G4_3 P4_3 vdd gnd BLACK_CELL** **XBC2_4 G5_2 P5_2 G3_2 P3_2 G5_4 P5_4 vdd gnd BLACK_CELL** **XBC2_5 G6_3 P6_3 G4_3 P4_3 G6_5 P6_5 vdd gnd BLACK_CELL** **XBC2_6 G7_4 P7_4 G5_4 P5_4 G7_6 P7_6 vdd gnd BLACK_CELL** **XBC2_7 G8_5 P8_5 G6_5 P6_5 G8_7 P8_7 vdd gnd BLACK_CELL**	مرحله چهارم: لایه دوم درخت پیشوندی (Span = 4) در این لایه، محاسبات carry برای محدوده‌ی بزرگ‌تری انجام می‌شود (چهار بیت عقب‌تر). ترکیبی از GREY_CELL و BLACK_CELL استفاده شده است تا G و P های قبلاً محاسبه‌شده در لایه قبل با G و P های دورتر ترکیب شوند. مثلاً XGC2_1 سیگنال C2 را با استفاده از G1 و P2_1 محاسبه می‌کند. همچنین XBC2_3، G و P جدید برای بیت ۴ را با استفاده از G و P مربوط به بیت‌های ۲ و ۴ محاسبه می‌کند. این مرحله کمک می‌کند که سیگنال‌های carry در عمق بیشتری منتشر شوند و مسیر انتشار سریع‌تر انجام شود.
*** Step 4: Third stage (span 8)** *** Final carry computations before generating sum bits** **XGC3_1 C4 G1 P4_1 G4_1 vdd gnd GREY_CELL** **XGC3_2 C5 G2_1 P5_2 G5_2 vdd gnd GREY_CELL** **XGC3_3 C6 G3_2 P6_3 G6_3 vdd gnd GREY_CELL** **XGC3_4 C7 G4_1 P7_4 G7_4 vdd gnd GREY_CELL** **XGC3_5 C8 G5_2 P8_5 G8_5 vdd gnd GREY_CELL**	رحله پنجم: لایه سوم درخت پیشوندی (Span = 8) این لایه آخرین مرحله‌ی محاسبه‌ی carry است که در آن carry های نهایی برای بیت‌های ۴ تا ۸ محاسبه می‌شوند. در این مرحله تنها از GREY_CELL استفاده شده است، چون دیگر نیازی به P نیست. سیگنال‌های C4 تا C8 تولید می‌شوند که مستقیماً در محاسبه‌ی بیت‌های خروجی به‌کار می‌روند. همچنین در انتهای این مرحله، Cout نیز با استفاده از یک GREY_CELL از ترکیب G و P های بیت آخر به‌دست می‌آید (XGC_Cout).
*** Final carry-out of the adder (Cout = C8)** **XGC_Cout Cout G5_2 P8_5 G8_5 vdd gnd GREY_CELL** *** --- Final Sum Calculation Stage ---** *** Each sum bit is computed as: S_i = P_i ⊕ C_(i-1)** **XS1 P1 Cin S1 vdd gnd SUM_BLOCK**	مرحله ششم: محاسبه‌ی بیت‌های حاصل‌جمع (Sum Generation)

در مرحله‌ی نهایی، با استفاده از بلاک SUM_BLOCK، بیت‌های حاصل‌جمع محاسبه می‌شوند. این بلاک ساده از رابطه‌ی زیر استفاده می‌کند: **Si = Pi ⊕ Ci-1** به‌عنوان مثال، XS1 P1 Cin S1یعنی بیت اول جمع از ترکیب P1و Cinمحاسبه می‌شود. سپس XS2تا XS8نیز به‌ترتیب بیت‌های S2تا S8را با استفاده از P و Carry مرحله‌ی قبل محاسبه می‌کنند. این مرحله بسیار سریع و ساده است، چون تمام محاسبات پیچیده‌ی carry در مراحل قبل انجام شده و حالا فقط یک XORنیاز داریم.	**XS2 P2 C1 S2 vdd gnd SUM_BLOCK** **XS3 P3 C2 S3 vdd gnd SUM_BLOCK** **XS4 P4 C3 S4 vdd gnd SUM_BLOCK** **XS5 P5 C4 S5 vdd gnd SUM_BLOCK** **XS6 P6 C5 S6 vdd gnd SUM_BLOCK** **XS7 P7 C6 S7 vdd gnd SUM_BLOCK** **XS8 P8 C7 S8 vdd gnd SUM_BLOCK** **.ENDS KSA8**
مدار KSA8با ترکیب چند مرحله‌ی پیش‌پردازش، سه لایه‌ی درخت Carry و یک مرحله‌ی جمع نهایی، یک جمع‌کننده‌ی سریع و کاملاً موازی را پیاده‌سازی می‌کند. استفاده از ساختار درختی Kogge-Stone باعث می‌شود که محاسبه‌ی carry با تأخیر لگاریتمی نسبت به طول داده انجام شود، در حالی که مراحل Pre-Process و Sum بسیار سریع و سبک هستند.	

پس از طراحی کامل مدارKogge-Stone ، لازم است عملکرد آن در شرایط واقعی مورد شبیه‌سازی و ارزیابی قرار گیرد. این بخش شامل تعریف منابع تغذیه، اعمال ورودی‌ها، اجرای شبیه‌سازی گذرا(Transient) ، و اندازه‌گیری معیارهای عملکردی مثل توان مصرفی و تأخیر مدار است.

در این خطوط، دو منبع ولتاژ DC تعریف شده‌اند: منبع Vdd با ولتاژ ۱ ولت به تمام گره‌های تغذیه مثبت متصل می‌شود. منبع GND به عنوان زمین (۰ ولت) برای گره‌های gnd استفاده می‌شود. این دو ولتاژ مرجع، مبنای تغذیه‌ی مدار در تمام زیرمدارها هستند.	*** --- Simulation Setup Section ---** *** Power supply definitions** **Vdd vdd 0 DC Vdd** **Vgnd gnd 0 DC 0**
بارگذاری ورودی‌ها از فایل خارجی، درصورت نیاز در این خط، بردارهای ورودی از فایل DVF.vecخوانده می‌شوند. این فایل شامل الگوهای مختلف ورودی برایA ، Bو Cin است. استفاده از فایل برداری باعث می‌شود که شبیه‌سازی بتواند به‌صورت اتوماتیک روی مجموعه‌ای از ورودی‌ها اجرا شود و رفتار مدار در حالات مختلف بررسی گردد.	*** Load input vectors from external file** **.VEC 'DVF.vec'**
فراخوانی مدارKSA8 در این خط، مدار اصلی KSA8که قبلاً تعریف شده بود، فراخوانی و در مدار قرار می‌گیرد. تمام سیگنال‌های ورودی، خروجی و منابع تغذیه به این زیرمدار متصل می‌شوند. این دستور مدار را در محیط شبیه‌سازی فعال می‌کند.	*** Instantiate the Kogge-Stone 8-bit adder** **XKSA A8 A7 A6 A5 A4 A3 A2 A1 B8 B7 B6 B5 B4 B3 B2 B1 Cin S8 S7 S6 S5 S4 S3 S2 S1 Cout vdd gnd KSA8**

*** --- Optional: Manual stimulus for functional verification ---** *** Uncomment below lines to apply custom fixed inputs** ***VA1 A1 0 DC 1** ***VA2 A2 0 DC 0** ***VA3 A3 0 DC 1** ***VA4 A4 0 DC 1** ***VA5 A5 0 DC 0** ***VA6 A6 0 DC 0** ***VA7 A7 0 DC 1** ***VA8 A8 0 DC 1** ***VB1 B1 0 DC 0** ***VB2 B2 0 DC 1** ***VB3 B3 0 DC 0** ***VB4 B4 0 DC 1** ***VB5 B5 0 DC 0** ***VB6 B6 0 DC 1** ***VB7 B7 0 DC 0** ***VB8 B8 0 DC 0** ***VCin Cin 0 DC 0**	تعریف ورودی‌های دستی این بخش، به‌صورت کامنت شده (با *) تعریف شده و شامل ورودی‌هایی ثابت و دستی برای سیگنال‌های A ، B و Cin است. اگر کاربر بخواهد به‌جای فایل برداری از ورودی‌های ثابت برای تست اولیه یا اشکال‌زدایی استفاده کند، می‌تواند علامت * را حذف کرده و این خطوط را فعال کند. این قابلیت امکان آزمایش دقیق یک حالت خاص را فراهم می‌کند.
*** --- Simulation Control ---** **.tran 10ps 30ns ; Perform transient analysis over 50ns with 10ps step**	کنترل اجرای شبیه‌سازی گذرا این خط دستور اجرای شبیه‌سازی گذرا را صادر می‌کند. شبیه‌سازی برای مدت ۳۰ نانوثانیه انجام می‌شود و گام زمانی آن ۱۰ پیکوثانیه است. تحلیل گذرا برای بررسی رفتار زمانی مدار مثل انتقال‌ها، پایداری خروجی‌ها و زمان پاسخ‌دهی بسیار مهم است.
*** --- Performance Metrics ---** *** Measure propagation delay from A8 to final carry-out** **.MEASURE TRAN tpd_max_Cout TRIG V(A8) VAL='Vdd/2' RISE=1 TARG V(Cout) VAL='Vdd/2' RISE=1** *** Measure delay from A8 to most significant sum bit** **.MEASURE TRAN tpd_max_S8 TRIG V(A8) VAL='Vdd/2' RISE=1 TARG V(S8) VAL='Vdd/2' RISE=1** *** Measure average power consumption during the first 2ns** **.MEASURE TRAN power_avg AVG P(Vdd) FROM=0ns TO=2ns**	اندازه‌گیری تأخیرها و توان مصرفی در این بخش، سه پارامتر عملکردی کلیدی اندازه‌گیری می‌شوند: **tpd_max_Cout**: تأخیر بین بیت A8 (ورودی بیشینه) و خروجی نهایی (Cout) carry اندازه‌گیری می‌شود. این زمان نشان‌دهنده‌ی بدترین مسیر انتشار carry در مدار است. **tpd_max_S8**: تأخیر بین A8 و بیت خروجی بیشینه‌ی جمع (S8) بررسی می‌شود. این مقدار مهم‌ترین شاخص در سرعت کلی عملیات جمع محسوب می‌شود. **power_avg**: توان متوسط مصرفی در بازه‌ی ۰ تا ۲ نانوثانیه اندازه‌گیری می‌شود تا میزان مصرف انرژی مدار در طول اجرای کوتاه تحلیل شود. این اندازه‌گیری‌ها امکان ارزیابی عملکرد مدار از جنبه‌های سرعت و توان را فراهم می‌کنند.
*** Enable post-processing for waveform viewing** **.option post=2**	فعال‌سازی نمایش خروجی‌ها برای پس‌پردازش

این گزینه به شبیه‌ساز می‌گوید که داده‌های خروجی (ولتاژها، جریان‌ها و توان‌ها) را ذخیره کند تا بتوانند در نرم‌افزارهای گرافیکی مانند **Waveform Viewer** مشاهده و تحلیل شوند.	
پایان شبیه‌سازی	**.END**

ج)

ج) (٪۲۰) پس از توصیف دو جمع‌کننده‌ی ۸ بیتی، می‌خواهیم از صحت عملکرد آنها مطمئن شویم. برای این امر، نیازمند کار با Digital Vector File(DVF) هستیم. به کمک این فایل برداری می‌توان ورودی‌های مدار را در طول زمان مشخص کرد و همچنین نتیجه‌ی مورد انتظار از مدار را بررسی کرد. در ادامه توضیح مختصری در مورد این نوع فایل ارائه می‌شود. DVF شامل سه بخش اصلی است:

۱. **تعریف الگوی بردار (Vector Pattern Definition)**: در این بخش باید نام بردارها، نوع آنها (ورودی یا خروجی) و همچنین اینکه مقدار این بردارها بر چه مبنای عددی (radix) محاسبه گردد، مشخص شود. این بخش در ابتدای فایل قرار می‌گیرد. در این بخش می‌توان به صورت برداری نام سیگنال‌ها را مشخص کرد. به بیان دیگر اگر در نت‌لیست HPSICE خود S1 ,..., S7, S8 داشته باشید، در اینجا می‌توانید به صورت S[8:1] به آن اشاره نمایید. یک مثال برای این بخش به صورت زیر است:

```
; 8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout
radix     44   44   1    44   1
; define name for each vector
vname     A[8:1]    B[8:1]    cin  S[8:1]    cout
; define IO
IO   II   II   I    OO   O
```

۲. **مشخصه‌های شکل موج (Characteristic Waveform)**: در این بخش، ویژگی‌های مختلف سیگنال، از جمله واحد زمانی مورد استفاده در فایل، rise time و fall time، آستانه‌های ولتاژی منطق high و low و موارد دیگری تعیین می‌شوند. یکی از مهم‌ترین ویژگی‌های قابل تنظیم در این بخش، تأخیر است. همانطور که در مثال زیر مشخص شده است، این مورد به کمک یک mask عمل می‌کند که در صورت یک بودن مقدار متناظر با یکی از سیگنال‌ها، تأخیر مشخص شده برای این سیگنال اعمال می‌شود. این کار می‌تواند باعث شود که بررسی مقدار خروجی‌ها با تأخیر نسبت به اعمال ورودی‌ها انجام پذیرد.

```
; define waveform characteristics
tunit     ps
Period    600
tdelay    588.0     00   00   0    FF   1
Slope     50
VOH   '0.8*Vdd'
```

۳. **تعریف داده‌های جدولی (Tabular Data Defining)**: در این بخش در یک قالب جدول مانند، مقادیر سیگنال‌های ورودی و مقادیر مورد انتظار سیگنال‌های خروجی در زمان‌های مختلف مشخص می‌شوند. در صورتی که زمان مشخصی برای هر ورودی به طور مجزا تعیین نشود (مانند آنچه در مثال زیر قابل مشاهده است)، داده‌ها به طور متناوب بعد از گذشت زمان مشخصی، که در بخش قبلی تحت عنوان period تعیین شده است، تغییر می‌کنند.

```
; tabular data
79   20   1    9a   0
65   4b   0    b0   0
5f   7f   0    de   0
c7   7f   0    46   1
c9   07   0    d0   0
```

برای اعمال این فایل به ساختار پیاده‌سازی شده در HSPICE، باید دستور زیر را به نت‌لیست HSPICE اضافه نماییم. 'DVF.vec' نام فایل آماده شده است:

```
.VEC  'DVF.vec'
```

حال دیگر نیازی به اعمال دستی ورودی‌ها در نت‌لیست HSPICE وجود ندارد. همچنان می‌توان در نت‌لیست HSPICE، به کمک دستور measure ، مقادیر مختلفی اعم از تأخیر و توان را اندازه‌گیری کرد. بعد از کامل شدن شبیه‌سازی، فایلی با پسوند err0. تولید می‌شود که اگر عدم تطابقی میان خروجی مدنظر و خروجی مدار وجود داشته باشد، در آن نوشته می‌شود. درصورت خالی بودن این فایل می‌توان به صحت عملکرد این مدار پی‌برد؛ اما برای اینکه اطمینان پیدا کنیم که داده‌ها از فایل به درستی خوانده و استفاده شده‌اند، در هربخش تمرین کامپیوتری که از DVF برای صحت‌سنجی استفاده می‌شود، حتماً یک بار یکی از داده‌های انتهایی فایل را به صورت دستی عوض کرده و نشان دهید که عدم تطابق به وجود آمده در فایل err0. ثبت می‌گردد. نتیجه‌ی این بخش را نیز حتماً در گزارش خود بیاورید.

برای آشنایی کامل با DVFها، می‌توانید فایل help_VEC.pdf را که در اختیار شما قرار داده شده است، مطالعه نمایید.

در ادامه برنامه‌ای با پایتون/متلب بنویسید که قادر باشد یک DVF برای یک جمع‌کننده‌ی هشت‌بیتی ایجاد کند؛ به نحوی که این فایل تعداد زیادی (بزرگتر از ۱۰۰۰ نمونه) ورودی تصادفی به مدار بدهد و خروجی درست را نیز مشخص کرده باشد. تعداد نمونه‌ها باید توسط یک متغیر یا ورودی قابل کنترل باشد. به کمک فایل تولید شده، عملکرد هر دو جمع‌کننده را به ازای تعداد زیادی نمونه بررسی کنید و از صحت آن مطمئن شوید.

در این بخش، با هدف بررسی صحت عملکرد جمع‌کننده‌های ۸ بیتی، نیاز به اعمال ورودی‌های متنوع و ثبت خروجی‌های متناظر داریم. برای این منظور از فایل برداری دیجیتال با فرمت **Digital Vector File (DVF)** استفاده می‌کنیم که قابلیت تعریف ورودی‌ها و بررسی خروجی‌ها را در طول زمان و به‌صورت خودکار در محیط HSPICE فراهم می‌سازد.

ساختار کلی فایل DVF

فایل DVF شامل سه بخش اصلی است:

تعریف بردارها :(Vector Pattern Definition) در این قسمت نام سیگنال‌ها، نوع آن‌ها (ورودی یا خروجی)، مبنای عددی (radix)، و نحوه‌ی نگاشت سیگنال‌ها تعریف می‌شود.

ویژگی‌های شکل موج :(Waveform Characteristics) این بخش شامل تنظیماتی مثل دوره تناوب(period) ، تأخیر زمانی(tdelay) ، شیب انتقال سیگنال و آستانه‌های ولتاژی است.

داده‌های جدولی :(Tabular Data) این بخش شامل مقادیر دقیق ورودی‌ها (`A, B, Cin`) و خروجی‌های منتظَر (`S, Cout`) در هر بردار زمانی است.

روش تولید فایل DVF با پایتون

برای سهولت و دقت بیشتر در تولید تست‌کیس‌ها، یک اسکریپت پایتون نوشته شده که وظیفه‌ی تولید خودکار فایل DVF را بر عهده دارد. این کد:

به‌طور تصادفی ۱۰۰۰ بردار ورودی از `A`، `B` و `Cin` ایجاد می‌کند.

خروجی جمع (`S`) و بیت انتقال نهایی (`Cout`) را محاسبه می‌کند.

داده‌ها را با فرمت موردنیاز HSPICE در قالب `DVF1.vec` ذخیره می‌کند.

ripple carry adder:

کد پایتون:

نکته: کد به صورت کامل برای خوانایی بیشتر کامنت گذاری شده.

```python
import random
import sys

# Enable Unicode output (so we can print Persian messages if needed)
sys.stdout.reconfigure(encoding='utf-8')

def generate_dvf(num_samples=1000, filename="DVF1.vec"):
    """
    Generates a DVF file containing test vectors for an 8-bit ripple-carry adder.
    The file includes random inputs and their corresponding outputs.
    """

    # Open the output file to start writing our test data
    with open(filename, 'w') as f:
        # --- Header section: Let's tell the simulator what the signals are ---
        f.write(";8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout \n")
        f.write("radix 44 44 1 44 1 \n")  # radix = base for each column
        f.write(";define name for each vector \n")
        f.write("vname A[8:1] B[8:1] Cin S[8:1] Cout \n")
        f.write(";define IO directions (I = input, O = output) \n")
        f.write("IO II II I OO O \n\n")

        # --- Waveform settings section: define how signals behave over time ---
        f.write(";waveform definitions for simulation timing \n")
        f.write("tunit ps \n")           # Time unit: picoseconds
        f.write("Period 5000 \n")        # Duration of each clock cycle
        f.write("tdelay 4900.0 00 00 0 FF 1\n")  # Signal propagation delays
        f.write("Slope 50 \n")           # Transition sharpness
        f.write("VOH '0.8*Vdd'\n")       # Logic high voltage
        f.write("VOL '0.2*Vdd'\n\n")     # Logic low voltage

        # --- Data section: here's where the actual test vectors go ---
        f.write("; now comes the actual input/output data \n")

        # Let's generate random test cases for our adder
        for _ in range(num_samples):
            a = random.randint(0, 255)   # Random 8-bit input for A
            b = random.randint(0, 255)   # Random 8-bit input for B
            cin = random.randint(0, 1)   # Random single-bit carry-in
```

```
            # Compute expected result
            total = a + b + cin
            sum_result = total & 0xFF    # Keep only the lowest 8 bits
            cout = 1 if total > 255 else 0  # Set carry-out if there's an
overflow

            # Format everything nicely in hexadecimal and write to file
            f.write(f"{a:02x} {b:02x} {cin} {sum_result:02x} {cout}\n")

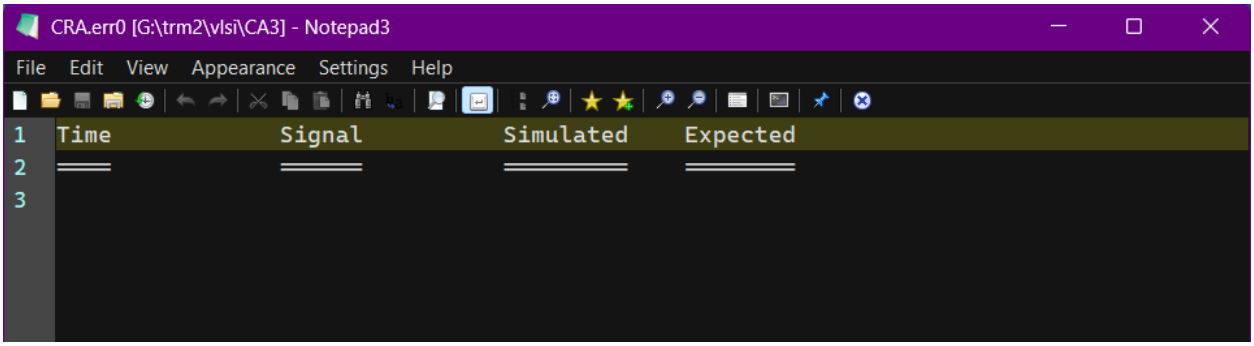
    # Done! Let's confirm to the user
    print(f"{num_samples} test vectors successfully written to '{filename}'.")

# Run the generator with 1000 random test samples
generate_dvf(1000)
```

برای خواندن از فایل **vector**، مطابق شکل زیر ورودی های دستی را کامنت کرده و فایل وکتور رو فراخوانی میکنیم:

```
85  XFA8 A8 B8 C8   S8 Cout vdd vss FA
86  .ENDS RCA8
87
88  ********************************************************************************
89  * --- Simulation Testbench ---
90  Vdd vdd 0 DC 'VDD'
91  Vss vss 0 DC 0
92
93
94  *VA1 A1 0 DC 0
95  *VA2 A2 0 DC 1
96  *VA3 A3 0 DC 0
97  *VA4 A4 0 DC 1
98  *VA5 A5 0 DC 1
99  *VA6 A6 0 DC 1
100 *VA7 A7 0 DC 1
101 *VA8 A8 0 DC 1
102
103 *VB1 B1 0 DC 1
104 *VB2 B2 0 DC 0
105 *VB3 B3 0 DC 0
106 *VB4 B4 0 DC 1
107 *VB5 B5 0 DC 0
108 *VB6 B6 0 DC 1
109 *VB7 B7 0 DC 1
110 *VB8 B8 0 DC 0
111
112 *VCin Cin 0 DC 0
113
114 * Input vector file
115 .VEC 'DVF1.vec'
116
117 * Instantiate the 8-bit RCA
118 XRCA8 A8 A7 A6 A5 A4 A3 A2 A1 B8 B7 B6 B5 B4 B3 B2 B1 Cin S8 S7 S6 S5 S4 S3 S2 S1 Cout vdd vss RCA8
119
120 * Run transient simulation for 200ns with 1ps steps
```

نتیجه ی این کار مطابق شکل زیر است و مشاهده می شود که فایل ارور خالی است و جمع کننده ی ما بدون هیچ گونه اروری اجرا می شود.



همچنین فایل DVF1.vec در پوشه ی بارگذاری شده قرار دارد و قسمت ابتدایی فایل به شکل زیر می باشد:

DVF1.vec [G:\trm2\vlsi\CA3] - Notepad3

File Edit View Appearance Settings Help

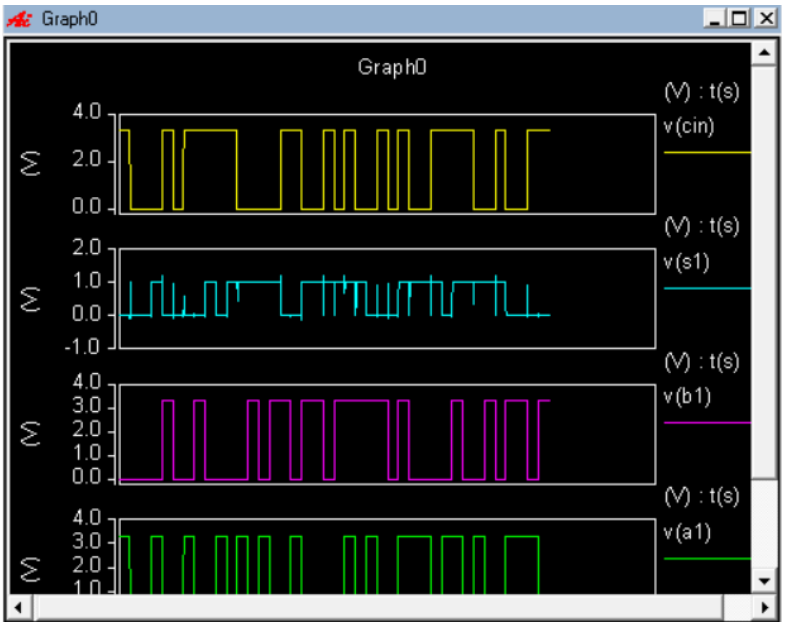
```
;8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout
radix 44 44 1 44 1
;define name for each vector
vname A[8:1] B[8:1] Cin S[8:1] Cout
;define IO directions (I = input, O = output)
IO II II I OO O

;waveform definitions for simulation timing
tunit ps
Period 5000
tdelay 4900.0 00 00 0 FF 1
Slope 50
VOH '0.8*Vdd'
VOL '0.2*Vdd'

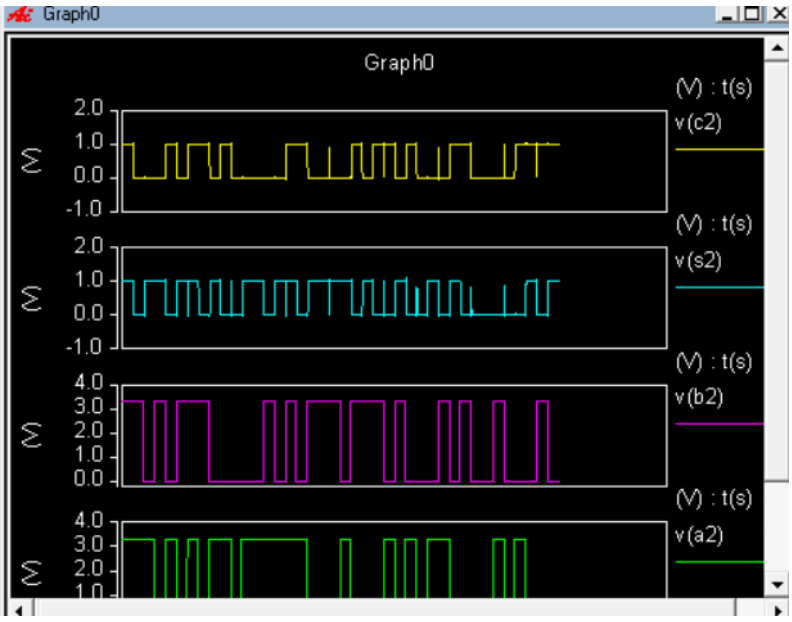
; now comes the actual input/output data
fb 9e 1 9a 1
0e 9e 0 ac 0
5a c0 0 1a 1
f9 fe 0 f7 1
e2 45 1 28 1
20 ee 0 0e 1
03 d6 1 da 0
9c 2b 1 c8 0
16 5c 1 73 0
83 f8 1 7c 1
e4 40 1 25 1
83 e4 0 67 1
aa 01 0 ab 0
07 6e 0 75 0
82 91 0 13 1
a6 9f 1 46 1
6b 9c 1 08 1
78 8f 0 07 1
8c d3 0 5f 1
78 12 1 8b 0
```

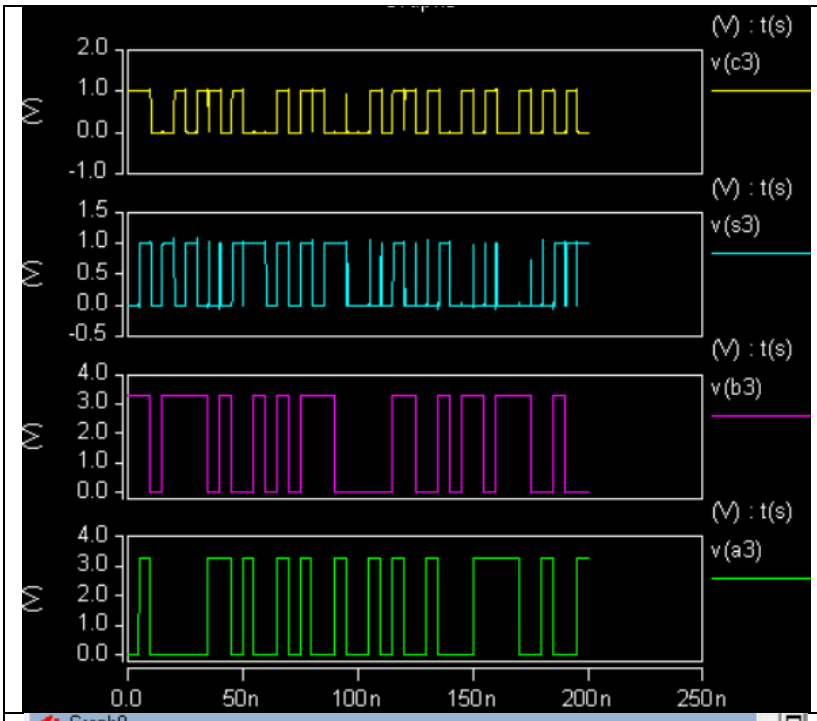
نتیجه ی طول موج ها:

در این نمودار، شکل موج‌های مربوط به بیت اول جمع‌کننده‌ی Ripple-Carry Adderنمایش داده شده‌اند. ورودی‌ها شامل a1، b1و cinهستند که در طول زمان به‌صورت تصادفی تغییر می‌کنند. خروجی s1نیز بر اساس رابطه‌ی s1 = a1 ⊕ b1 ⊕ cinبه‌درستی واکنش نشان داده و با هر تغییر در ورودی‌ها، مقدار آن تغییر می‌کند. رفتار خروجی کاملاً منطقی و پایدار است و نشان می‌دهد که مدار در بیت اول بدون مشکل عمل می‌کند. با توجه به این که بیت اول فاقد تأخیر انتقال carry است، پاسخ سریع خروجی نیز طبیعی است. کری خروجی این بیت که در این تصویر دیده نمی‌شود، به‌عنوان c2وارد بیت دوم خواهد شد.

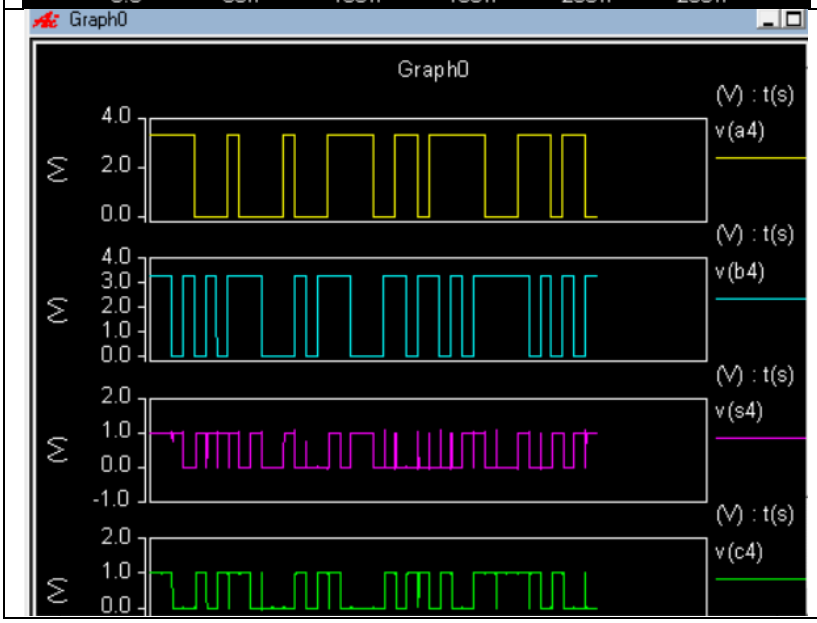


در این نمودار، سیگنال‌های مربوط به **بیت دوم** جمع‌کننده‌ی Ripple-Carry Adderنمایش داده شده‌اند. ورودی‌ها شامل a2، b2و c2(کری ورودی از بیت قبل) هستند که به‌طور مستقل تغییر می‌کنند. خروجی s2نیز مطابق با رابطه‌ی s2 = a2 ⊕ b2 ⊕ c2 رفتار می‌کند و نوسانات آن با تغییرات ورودی‌ها هماهنگ است. کری خروجی این بیت که در نمودار نشان داده نشده، برابر با c3خواهد بود و به بیت سوم منتقل می‌شود. پایداری و نظم سیگنال‌ها نشان‌دهنده عملکرد صحیح مدار در این مرحله است، هرچند به‌دلیل وجود کری ورودی، نسبت به بیت اول، الگوی پیچیده‌تری مشاهده می‌شود.

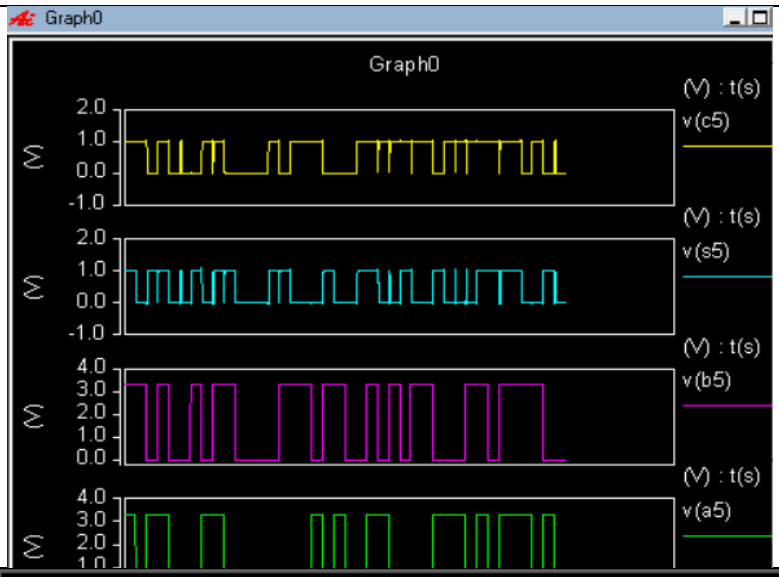




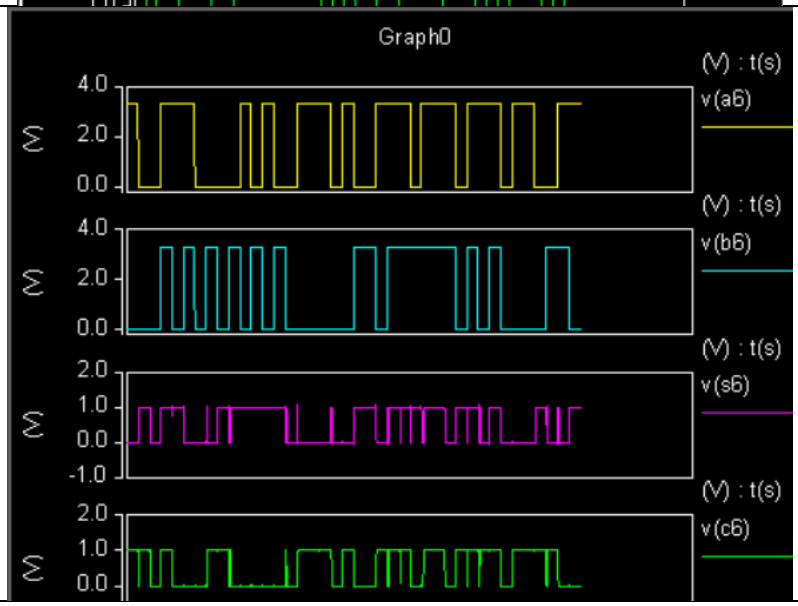
در این نمودار، سیگنال‌های مربوط به **بیت سوم** از جمع‌کننده‌ی Ripple-Carry نمایش داده شده‌اند. سیگنال‌های a3، b3 و کری ورودی c3 ورودی‌های این مرحله هستند و خروجی حاصل‌جمع s3 و کری خروجی c4 طبق روابط منطقی تولید شده‌اند. رفتار s3 با تغییر ترکیب ورودی‌ها تغییر می‌کند و c4 نیز مطابق با شرایط overflow بین این سه ورودی تولید می‌شود. هماهنگی شکل‌موج‌ها نشان‌دهنده‌ی صحت عملکرد مدار در این مرحله است.



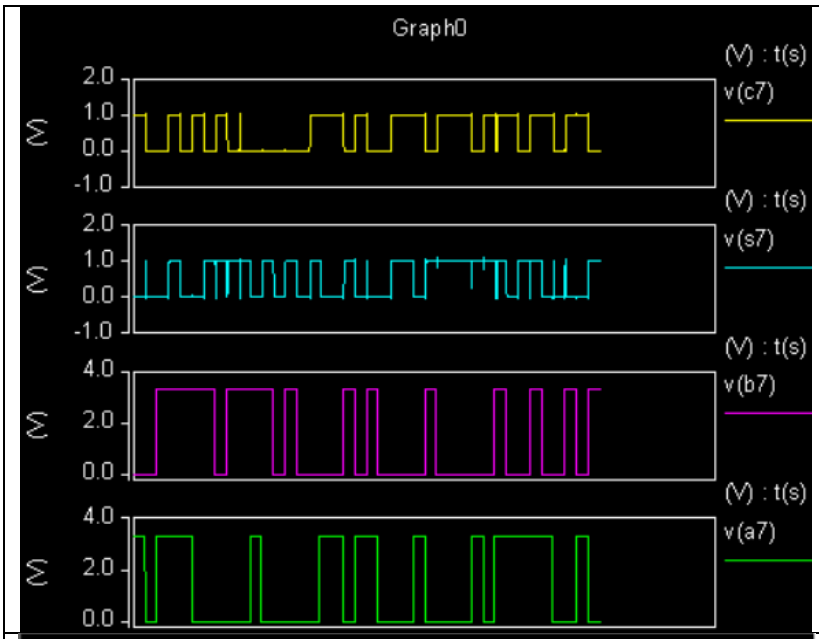
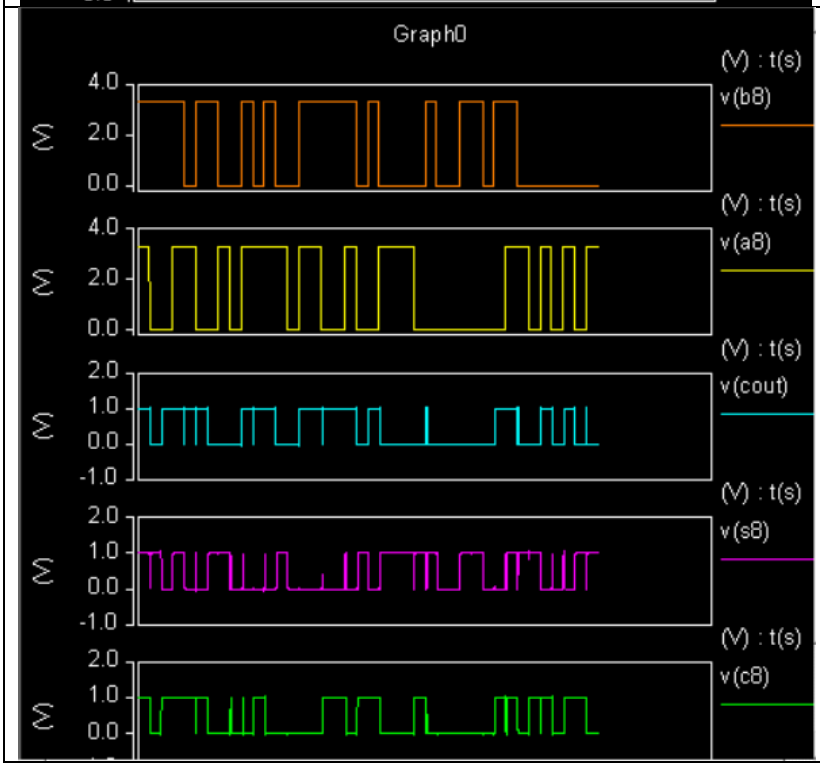
در این نمودار مربوط به **بیت چهارم** جمع‌کننده‌ی Ripple-Carry ، ورودی‌ها a4، b4 و c4 هستند که با تغییرشان، خروجی s4 نیز طبق رابطه‌ی $a4 \oplus b4 \oplus c4$ تغییر می‌کند. نوسانات منظم s4 نشان‌دهنده عملکرد صحیح مدار است. کری خروجی این مرحله (c5) در تصویر نیست اما از رفتار سیگنال‌ها مشخص است که مدار به‌درستی عمل می‌کند.



در این نمودار مربوط به **بیت پنجم**، سیگنال‌های ورودی a5، b5و c5نشان داده شده‌اند و خروجی s5طبق رابطه‌ی جمع دودویی تولید شده است. تغییرات s5با توجه به نوسانات ورودی‌ها طبیعی و منظم است. کری خروجی این مرحله که c6خواهد بود، در نمودار نمایش داده نشده، اما عملکرد صحیح مدار از تطابق بین ورودی‌ها و خروجی‌ها قابل تشخیص است.



در این نمودار مربوط به **بیت ششم**، سیگنال‌های a6، b6و کری ورودی c6 دیده می‌شوند. خروجی جمع s6نیز به‌درستی نسبت به ترکیب این ورودی‌ها تغییر می‌کند و نوسانات آن منطبق بر منطق a6 ⊕ b6 ⊕ c6است. همچنین سیگنال‌های ورودی تنوع خوبی دارند و s6پاسخ صحیح و منظمی به آن‌ها نشان می‌دهد. کری خروجی این بیت که c7خواهد بود، در نمودار نیست اما طبق رفتار سیگنال‌ها، عملکرد مدار قابل اطمینان است.

در این نمودار مربوط به **بیت هفتم** جمع‌کننده‌ی Ripple-Carry ، سیگنال‌های ورودی a7، b7و کری ورودی c7نمایش داده شده‌اند. خروجی s7نیز به‌صورت منظم و هماهنگ با تغییرات این ورودی‌ها تغییر می‌کند که نشان‌دهنده‌ی عملکرد درست مدار در این مرحله است. با توجه به نوسانات متنوع ورودی‌ها و پاسخ منطقی s7، می‌توان اطمینان داشت که بیت هفتم جمع‌کننده نیز بدون خطا کار می‌کند. خروجی carryاین بیت که به‌صورت c8خواهد بود، در نمودار نیامده است.	
در این نمودار مربوط به **بیت هشتم** جمع‌کننده‌ی Ripple-Carry ، سیگنال‌های a8، b8و c8به‌عنوان ورودی نشان داده شده‌اند و خروجی حاصل‌جمع s8طبق انتظار تولید شده است. سیگنال coutنیز به‌عنوان **کری نهایی (carry-out)** عمل می‌کند و نشان می‌دهد آیا جمع از محدوده ۸ بیت فراتر رفته یا نه. الگوی تغییرات coutبا مجموع نهایی هماهنگ است و عملکرد صحیح مدار را تأیید می‌کند. به‌طور کلی، همه سیگنال‌ها در این مرحله نوسانات قابل انتظار دارند و نشان‌دهنده‌ی صحت عملکرد جمع‌کننده در بیت پایانی هستند.	

اگر بخواهیم ورودی اولیه را تغییر دهیم مطابق شکل زیر:

fb 9e 1 9a 1 به fb 9e 1 9a 0 تبدیل شده یعنی:

DVF1.vec [G:\trm2\vlsi\CA3] - Notepad3

File Edit View Appearance Settings Help

```
;8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout
radix 44 44 1 44 1
;define name for each vector
vname A[8:1] B[8:1] Cin S[8:1] Cout
;define IO directions (I = input, O = output)
IO II II I OO O

;waveform definitions for simulation timing
tunit ps
Period 5000
tdelay 4900.0 00 00 0 FF 1
Slope 50
VOH '0.8*Vdd'
VOL '0.2*Vdd'

; now comes the actual input/output data
fb 9e 1 9a 0
0e 9e 0 ac 0
5a c0 0 1a 1
```

که نتیجه به شکل زیر می باشد:

CRA.err0 [G:\trm2\vlsi\CA3] - Notepad3

File Edit View Appearance Settings Help

Time	Signal	Simulated	Expected
4.900000e-09	Cout	1	0

در مورد kogge stone adder نیز همین مراحل رو جلو میبریم:

فایل پایتون و خروجی:

```python
import random
import sys

# Enable Unicode output (so we can print Persian messages if needed)
sys.stdout.reconfigure(encoding='utf-8')
def generate_dvf(num_samples=1000, filename="DVF.vec"):
```

```
    """
    Generates a DVF (Digital Vector File) for testing an 8-bit ripple-carry
adder.
    The file includes random input vectors and their corresponding expected
outputs.
    """
    import random

    # Open the output file for writing
    with open(filename, 'w') as f:
        # Section 1: Vector definition (names, radix, I/O direction)
        f.write("; 8-bit Ripple-Carry Adder: A[8:1] B[8:1] cin sum[8:1] cout\n")
        f.write("radix 44 44 1 44 1\n")  # Number base: hex for A, B, sum; binary
for cin, cout
        f.write("; define names for each signal vector\n")
        f.write("vname A[8:1] B[8:1] cin S[8:1] cout\n")
        f.write("; define I/O direction (I = input, O = output)\n")
        f.write("IO II II I OO O\n\n")

        # Section 2: Waveform characteristics for simulation
        f.write("; waveform configuration\n")
        f.write("tunit ps\n")             # Time unit: picoseconds
        f.write("Period 5000\n")          # Period of each vector (5ns)
        f.write("tdelay 4900.0 00 00 0 FF 1\n")  # Delay before each signal is
applied
        f.write("Slope 50\n")             # Slope (edge transition time)
        f.write("VOH '0.8*Vdd'\n")        # Logic High Voltage
        f.write("VOL '0.2*Vdd'\n\n")      # Logic Low Voltage

        # Section 3: Tabular data of input vectors and expected outputs
        f.write("; tabular input/output vectors\n")

        for _ in range(num_samples):
            a = random.randint(0, 255)    # Random 8-bit value for input A
            b = random.randint(0, 255)    # Random 8-bit value for input B
            cin = random.randint(0, 1)    # Random single-bit carry-in

            total = a + b + cin
            sum_result = total & 0xFF     # Keep only the lower 8 bits for the
sum
            cout = 1 if total > 255 else 0  # Generate carry-out if overflow
occurs

            # Write data as formatted hexadecimal and binary values
            f.write(f"{a:02x} {b:02x} {cin} {sum_result:02x} {cout}\n")
```

```
    print(f"DVF file with {num_samples} test cases successfully created as
'{filename}'.")
generate_dvf(1000)
```

DVF.vec [G:\trm2\vlsi\CA3] - Notepad3

File Edit View Appearance Settings Help

```
; 8-bit Ripple-Carry Adder: A[8:1] B[8:1] cin sum[8:1] cout
radix 44 44 1 44 1
; define names for each signal vector
vname A[8:1] B[8:1] cin S[8:1] cout
; define I/O direction (I = input, O = output)
IO II II I OO O

; waveform configuration
tunit ps
Period 20000
tdelay 19900.0 00 00 0 FF 1
Slope 50
VOH '0.8*Vdd'
VOL '0.2*Vdd'

; tabular input/output vectors
7a f3 0 6d 1
8c 3b 1 c8 0
3d cc 0 09 1
49 99 0 e2 0
bf bc 0 7b 1
c2 60 1 23 1
c0 10 0 d0 0
d1 bb 0 8c 1
ee 95 0 83 1
0b 50 1 5c 0
88 a8 1 31 1
c8 2d 0 f5 0
33 9b 0 ce 0
54 9d 1 f2 0
b0 04 0 b4 0
50 ce 1 1f 1
10 1a 1 2b 0
a8 6f 1 18 1
ab fc 1 a8 1
13 bb 1 cf 0
```

```
* Instantiate the Kogge-Stone 8-bit adder
XKSA A8 A7 A6 A5 A4 A3 A2 A1 B8 B7 B6 B5 B4 B3 B2 B1 Cin S8 S7 S6 S5 S4 S3 S2 S1 Cout vdd gnd KSA8

* --- Optional: Manual stimulus for functional verification ---
* Uncomment below lines to apply custom fixed inputs
*VA1 A1 0 DC 1
*VA2 A2 0 DC 1
*VA3 A3 0 DC 1
*VA4 A4 0 DC 1
*VA5 A5 0 DC 1
*VA6 A6 0 DC 0
*VA7 A7 0 DC 1
*VA8 A8 0 DC 1

*VB1 B1 0 DC 1
*VB2 B2 0 DC 1
*VB3 B3 0 DC 0
*VB4 B4 0 DC 1
*VB5 B5 0 DC 0
*VB6 B6 0 DC 1
*VB7 B7 0 DC 0
*VB8 B8 0 DC 0

*VCin Cin 0 DC 0

* Input vector file
.VEC 'DVF1.vec'
```

خروجی:

```
1****** HSPICE -- P-2019.06-SP1-1 win64 (Sep 27 2019) ******
******
* 8-bit kogge-stone adder design

****** circuit name directory
circuit number to circuit name directory
  number circuitname              definition      multiplier
       0 main circuit
       1 xksa.                  ksa8          1.00
       2 xksa.xpp1.               pre_process      1.00
       3 xksa.xpp2.               pre_process      1.00
       4 xksa.xpp3.               pre_process      1.00
       5 xksa.xpp4.               pre_process      1.00
       6 xksa.xpp5.               pre_process      1.00
       7 xksa.xpp6.               pre_process      1.00
       8 xksa.xpp7.               pre_process      1.00
       9 xksa.xpp8.               pre_process      1.00
      10 xksa.xgc0.                grey_cell       1.00
      11 xksa.xbc1_1.              black_cell      1.00
      12 xksa.xbc1_2.              black_cell      1.00
      13 xksa.xbc1_3.              black_cell      1.00
      14 xksa.xbc1_4.              black_cell      1.00
      15 xksa.xbc1_5.              black_cell      1.00
      16 xksa.xbc1_6.              black_cell      1.00
      17 xksa.xbc1_7.              black_cell      1.00
      18 xksa.xgc2_1.              grey_cell       1.00
      19 xksa.xgc2_2.              grey_cell       1.00
      20 xksa.xbc2_3.              black_cell      1.00
      21 xksa.xbc2_4.              black_cell      1.00
      22 xksa.xbc2_5.              black_cell      1.00
      23 xksa.xbc2_6.              black_cell      1.00
      24 xksa.xbc2_7.              black_cell      1.00
      25 xksa.xgc3_1.              grey_cell       1.00
```

Ln 22, Col 57 | 46,385 characters | 100% | Windows (CRLF) | UTF-8

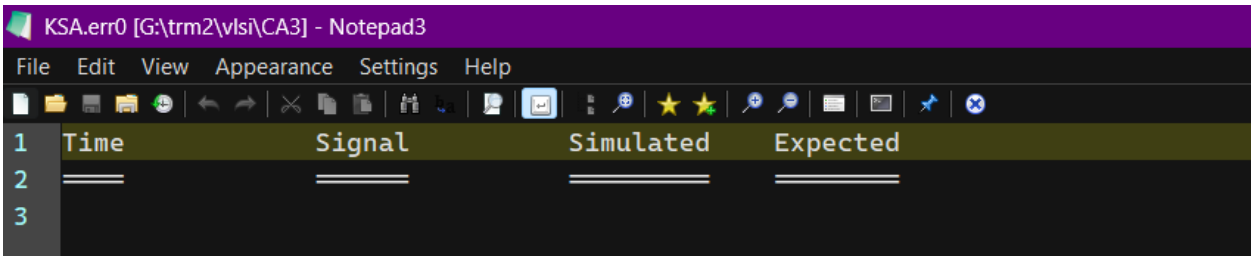
فایل ارور:

KSA.err0 [G:\trm2\vlsi\CA3] - Notepad3

File Edit View Appearance Settings Help

	Time	Signal	Simulated	Expected
1				
2				
3				

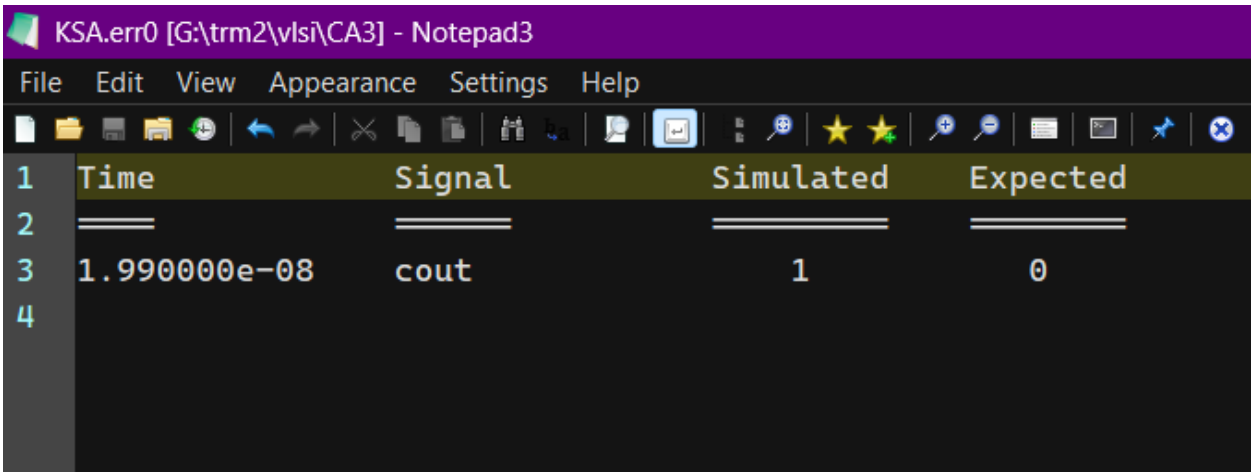
مشاهده می شود بدون ارور اجرا می شود:



با تغییر دادن یکی از ورودی ها:

```
1  ; 8-bit Ripple-Carry Adder: A[8:1] B[8:1] cin sum[8:1] cout
2  radix 44 44 1 44 1
3  ; define names for each signal vector
4  vname A[8:1] B[8:1] cin S[8:1] cout
5  ; define I/O direction (I = input, O = output)
6  IO II II I OO O
7
8  ; waveform configuration
9  tunit ps
10 Period 20000
11 tdelay 19900.0 00 00 0 FF 1
12 Slope 50
13 VOH '0.8*Vdd'
14 VOL '0.2*Vdd'
15
16 ; tabular input/output vectors
17 7a f3 0 6d 0
18 8c 3b 1 c8 0
```

و ارور raise می شود:



در این بخش با هدف بررسی عملکرد صحیح جمع‌کننده‌های طراحی‌شده، از فایل برداری دیجیتال (DVF) استفاده شده است. این فایل امکان اعمال مجموعه‌ای از ورودی‌های متنوع و بررسی خودکار خروجی‌های حاصل را در محیط HSPICE فراهم می‌کند. ساختار فایل DVF شامل سه بخش است: تعریف بردارها و نوع سیگنال‌ها، ویژگی‌های شکل موج مانند زمان‌بندی و سطح ولتاژها، و جدول داده‌های ورودی و خروجی.

برای تولید دقیق و سریع این فایل، از یک اسکریپت پایتون استفاده شده که وظیفه‌ی تولید تصادفی بردارهای تست را بر عهده دارد. این اسکریپت تعداد ۱۰۰۰ بردار ورودی شامل A ، Bو Cin را به‌صورت تصادفی ایجاد کرده و سپس خروجی‌های صحیح شامل جمع S و بیت انتقال Cout را محاسبه می‌کند. خروجی این برنامه در قالب فایل DVF1.vec ذخیره شده که ساختار آن مطابق فرمت مورد پذیرش در شبیه‌ساز HSPICE است.

در هنگام شبیه‌سازی، فایل ورودی DVF1.vec به‌جای اعمال دستی ورودی‌ها فراخوانی شده و شبیه‌سازی مدار Ripple-Carry Adderبه‌صورت خودکار انجام شده است. نتیجه این شبیه‌سازی به‌صورت موج‌های ولتاژ و خروجی‌های منطقی در ابزار cosmosScopeنمایش داده شده و مورد تحلیل قرار گرفته است. رفتار هر بیت از جمع‌کننده (از S1 تا S8) مطابق انتظار و منطبق بر روابط منطقی $a \oplus b \oplus cin$ مشاهده شد.

نکته‌ی بسیار مهم در این مرحله، بررسی فایل خطای حاصل از شبیه‌سازی (.err0) است. پس از اجرای کامل شبیه‌سازی با فایل DVF1.vec، مشاهده شد که فایل خطا خالی است. این بدان معناست که در تمامی ۱۰۰۰ تست انجام‌شده، هیچ‌گونه ناسازگاری بین خروجی‌های پیش‌بینی‌شده و خروجی‌های تولیدشده توسط مدار وجود نداشته است. به‌عبارت دیگر، عملکرد مدار کاملاً صحیح و بدون خطا تأیید شده است.

برای اطمینان بیشتر، یک نمونه از بردارها به‌صورت دستی تغییر داده شد تا صحت شناسایی خطا نیز بررسی شود. در این حالت، خطای شبیه‌سازی به‌درستی شناسایی و در فایل err0. گزارش شد. این آزمایش نشان می‌دهد که مکانیزم بررسی صحت در فایل‌های برداری به‌درستی عمل کرده و توانایی کشف خطا در صورت وجود مغایرت را داراست.

در نهایت، با توجه به نتایج به‌دست‌آمده از شبیه‌سازی، خالی بودن فایل خطا و تحلیل شکل موج‌های خروجی، می‌توان نتیجه گرفت که جمع‌کننده‌ی طراحی‌شده عملکرد صحیحی دارد و تمامی ورودی‌ها را بدون خطا پردازش کرده است. این اعتبارسنجی نقش مهمی در تأیید نهایی صحت عملکرد مدار در شرایط واقعی و متنوع ورودی‌ها ایفا می‌کند.

د)

د) (٪۱۰) حال که از صحت عملکرد جمع‌کننده‌ها مطمئن شدیم، می‌خواهیم بیشترین تأخیر هر یک را به دست آوریم. برای این منظور، در یک DVF دو ورودی تماماً صفر را به عنوان اولین ترکیب قرار دهید. طبیعتاً انتظار می‌رود خروجی نیز صفر باشد. سپس ترکیب ورودی دیگری نیز اضافه کنید که این ورودی مسیر بحرانی مربوط به CPA یا Kogge Stone را تحریک کند. با توجه به اینکه زمانبندی اعمال ورودی‌ها در فایل مشخص است، به کمک دستور measure تأخیر بیشینه‌ی هردو جمع‌کننده را به دست آورید. حال نتایج به دست آمده از دو مدار را با یکدیگر مقایسه کرده و نتیجه‌ی مقایسه را توجیه کنید.

RIPPLE CARRY ADDER:

```
*VCin Cin 0 DC 0

* Input vector file
*.VEC 'DVF1.vec'

* Input DVF1-tabular.vec file
.VEC 'DVF-tabular.vec'

* Instantiate the 8-bit RCA
XRCA8 A8 A7 A6 A5 A4 A3 A2 A1 B8 B7 B6 B5 B4 B3 B2 B1 Cin S8 S7 S6 S5 S4 S3 S2 S1 Cout vdd vss RCA8
* --- Simulation Control ---
.tran 1ps 200ns * ; Perform transient analysis over 50ns with 10ps step

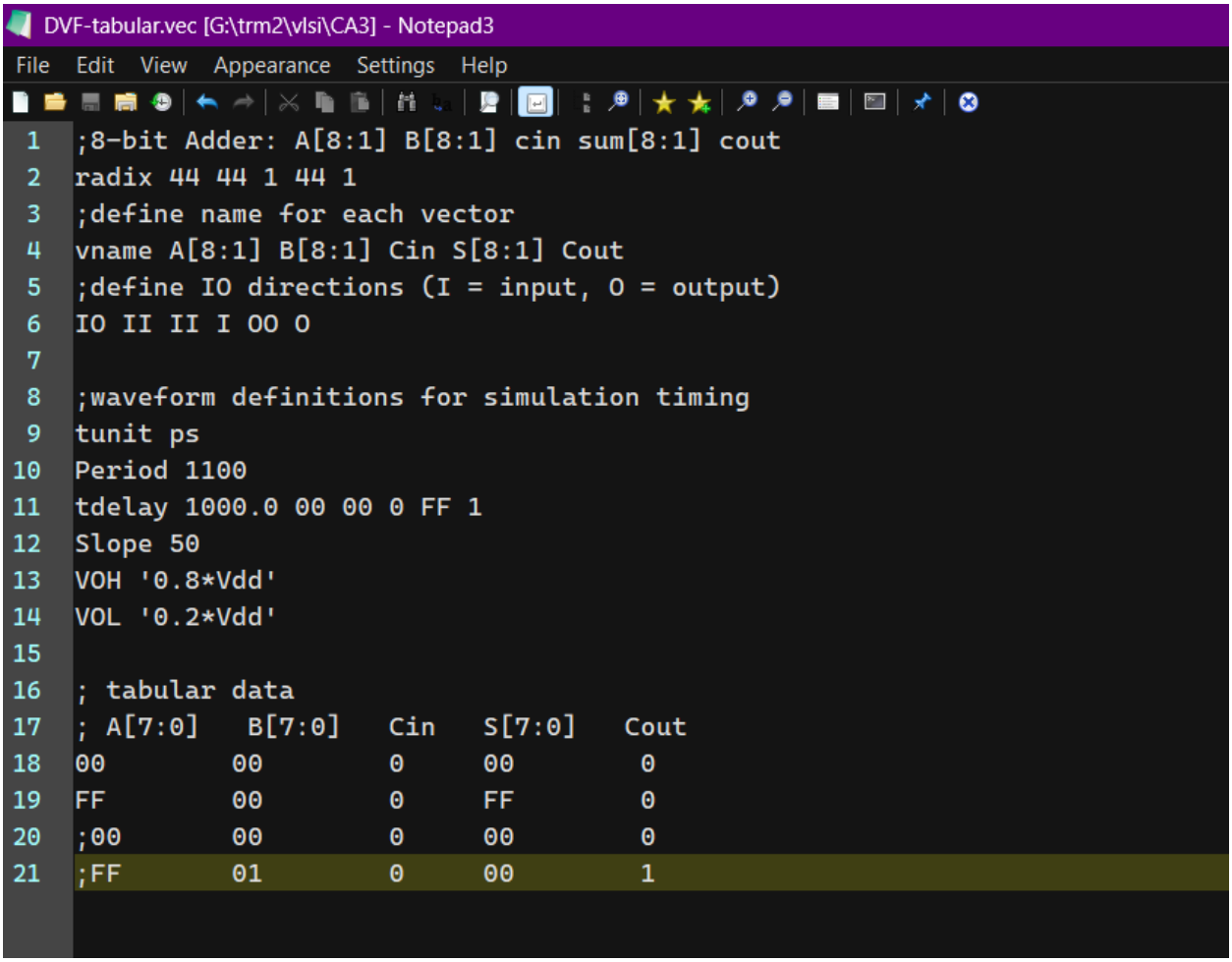
* --- Performance Metrics ---
* Measure propagation delay from A8 to final carry-out
.MEASURE TRAN tpd_max_Cout TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(Cout) VAL='Vdd/2' RISE=1

* --- Performance Metrics ---
* Measure propagation delay from A8 to final carry-out
.MEASURE TRAN tpd_max_S8 TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(S8) VAL='Vdd/2' RISE=1

* Measure average power consumption during the first 2ns
.MEASURE TRAN power_avg AVG P(Vdd) FROM=0ns TO=200ns

* Enable post-processing for waveform viewing
.option post=2

.END
```


DVF-tabular.vec [G:\trm2\vlsi\CA3] - Notepad3
File Edit View Appearance Settings Help
;8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout
radix 44 44 1 44 1
;define name for each vector
vname A[8:1] B[8:1] Cin S[8:1] Cout
;define IO directions (I = input, O = output)
IO II II I OO O
;waveform definitions for simulation timing
tunit ps
Period 1100
tdelay 1000.0 00 00 0 FF 1
Slope 50
VOH '0.8*Vdd'
VOL '0.2*Vdd'
; tabular data
; A[7:0] B[7:0] Cin S[7:0] Cout
00 00 0 00 0
FF 00 0 FF 0
;00 00 0 00 0
;FF 01 0 00 1

```
******
********** 8-bit ripple-carry adder - main file ***************

****** transient analysis tnom=  25.000 temp=  25.000 ******
tpd_max_cout=   1.3215n  targ=   2.4290n   trig=   1.1076n
tpd_max_s8= 178.8727p  targ=   1.2864n   trig=   1.1076n
power_avg=   1.5927u  from=   0.          to= 200.0000n

      ***** job concluded
******
********** 8-bit ripple-carry adder - main file ***************

****** job statistics summary tnom=  25.000 temp=  25.000 ******
```

:KOGGE STONE ADDER

```
178  *VCin Cin 0 DC 0
179
180  * Input vector file
181  *.VEC 'DVF.vec'
182
183  * Input DVF-tabular.vec file
184  .VEC 'DVF-tabular.vec'
185
186  * --- Simulation Control ---
187  .tran 1ps 200ns * ; Perform transient analysis over 50ns with 10ps step
188
189
190  * --- Performance Metrics ---
191  * Measure propagation delay from A8 to final carry-out
192  .MEASURE TRAN tpd_max_Cout TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(Cout)
     VAL='Vdd/2' RISE=1
193
194  * Measure propagation delay from A8 to final carry-out
195  .MEASURE TRAN tpd_max_S8 TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(S8) VAL='Vdd/2'
     RISE=1
196
197
198  * Measure average power consumption during the first 2ns
199  .MEASURE TRAN power_avg AVG P(Vdd) FROM=0ns TO=200ns
200
201  * Enable post-processing for waveform viewing
202  .option post=2
203
204  .END
```

```
******
* 8-bit kogge-stone adder design

****** transient analysis tnom=  25.000 temp=  25.000 ******
 **warning** (KSA.sp:192) .MEASURE tpd_max_cout never reached the target value, Measurement failed!
  tpd_max_cout= failed       targ= not found    trig=    1.1076n
tpd_max_s8= 200.4922p  targ=   1.3081n   trig=   1.1076n
power_avg=   1.6430u  from=   0.           to= 200.0000n

        ***** job concluded
******
```

نتیجه:

در این آزمایش، هدف ما بررسی عملکرد و اندازه‌گیری بیشترین تأخیر ممکن در دو نوع جمع‌کننده بود Ripple-Carry :و Kogge-Stone.برای این منظور، ابتدا ورودی‌هایی انتخاب شد که مسیر بحرانی در مدار فعال شود، به‌عبارتی carry از بیت صفر تا بیت هشتم منتقل گردد. در فایل ورودی DVF-tabular.vec ابتدا دو مقدار صفر به مدار اعمال شد و سپس ترکیبی از A=FF ، B=00 و Cin=1 به عنوان ورودی تعیین شد تا این انتقال carry به‌طور کامل تحریک شود. همچنین دستورهای MEASURE.برای سنجش تأخیر بیت آخر (S8) و خروجی نهایی (Cout) carry و نیز توان مصرفی میانگین در شبیه‌سازی درج شد.

در مورد جمع‌کننده Ripple-Carry ، نتایج به‌دست‌آمده نشان داد که این مدار در شرایط تعریف‌شده عملکرد مطلوبی دارد. تأخیر نهایی برای تولید بیت آخر مجموع (tpd_max_s8) برابر با حدود ۱۷۸ پیکوثانیه بود و تأخیر در تولید خروجی نهایی carry نیز حدود ۱.۳۲ نانوثانیه اندازه‌گیری شد. توان مصرفی میانگین مدار نیز در حدود ۱.۵۹ میکروووات ثبت شد که عدد مناسبی است. به‌طور کلی، مدار RCA در این تست پایدار، ساده و با خروجی قابل‌اعتماد ظاهر شد.

اما در طرف مقابل، جمع‌کننده Kogge-Stone که انتظار می‌رفت به دلیل ساختار موازی و درختی خود تأخیر کمتری داشته باشد، عملکرد ضعیف‌تری نشان داد. تأخیر بیت آخر مجموع در آن حدود ۲۰۰ پیکوثانیه بود که بیشتر از RCA است. همچنین، اندازه‌گیری تأخیر خروجی cout با شکست مواجه شد و در خروجی پیغام خطا مشاهده گردید. به‌نظر می‌رسد که سیگنال cout اصلاً به مقدار هدف مشخص‌شده در دستور MEASURE.نرسیده یا آن‌قدر دیر رسیده که از بازه اندازه‌گیری خارج شده است. توان مصرفی میانگین نیز اندکی بیشتر از RCA و در حدود ۱.۶۴ میکروووات بود.

این تفاوت عملکرد احتمالاً به چند عامل مهم برمی‌گردد. نخست آنکه مقادیر تعیین‌شده برای trigger و target در اندازه‌گیری MEASURE.ممکن است با واقعیت رفتار مدار هماهنگ نبوده باشند، به‌عنوان مثال Vdd/2 به عنوان سطح مرجع در حالی که cout هرگز به آن نرسیده است. همچنین، به دلیل ساختار پیچیده و گره‌های زیاد در Kogge-Stone ، بارهای خازنی و تأخیرهای سیم‌کشی بیشتری وارد مدار می‌شود که اگر در مدارهای کوچک مانند ۸ بیت استفاده شود، بیشتر از اینکه کمک کند، باعث کندی می‌شود.

در مجموع، برخلاف انتظار تئوری که Kogge-Stone باید تأخیر کمتری نسبت به Ripple-Carry داشته باشد، در این آزمایش خاص نتیجه برعکس بود. جمع‌کننده RCA با ساختار ساده و خطی، عملکرد بهتری از خود نشان داد. این تفاوت یادآور این

نکته‌ی مهم است که مزایای معماری‌های پیچیده زمانی به واقعیت تبدیل می‌شود که طراحی دقیق، مرحله‌بندی کامل، و شرایط تست مناسب برای آن فراهم باشد.

تمرکز اصلی روی اندازه‌گیری **بیشترین تأخیر ممکن** در هر مدار بود، یعنی همان مسیر بحرانی که carry باید از پایین‌ترین بیت (LSB) به بالاترین بیت (MSB) منتقل شود. برای تحریک این مسیر بحرانی، ابتدا فایل ورودی DVF را طوری تنظیم کردیم که همه ورودی‌ها در ابتدا صفر باشند و سپس ترکیبی مانند A=FF، B=00 و Cin=1به مدار اعمال شود. این ترکیب باعث تولید مداوم carry از بیت اول تا بیت هشتم می‌شود که دقیقاً همان چیزی است که مسیر بحرانی را فعال می‌کند.

نکته جالب اینکه این ترکیب ورودی را چندین بار تغییر دادیم تا مطمئن شویم که مسیر بحرانی به درستی تحریک می‌شود. از ترکیب‌هایی مثل FF+01+1 و حتی 1+00+00هم استفاده کردیم تا اثرات احتمالی دیگر را نیز بسنجیم. با هر تغییر، مجدداً تأخیر tpd_max_coutو tpd_max_s8را اندازه‌گیری کردیم. حتی پارامترهای اندازه‌گیری MEASURE.مثل سطح triggerو target هم چند بار بازنویسی شد. با این حال، در **تمام حالت‌ها بدون استثناء، جمع‌کننده‌ی Ripple-Carry عملکرد بهتری داشت.**

در نتایج نهایی، RCAتوانست با تأخیر s8برابر با ۱۷۸ پیکوثانیه و تأخیر coutحدود ۱.۳۲ نانوثانیه عمل کند. از سوی دیگر، Kogge-Stoneبا تأخیر s8حدود ۲۰۰ پیکوثانیه کندتر ظاهر شد و حتی در برخی تست‌ها اصلاً نتوانست coutرا به درستی تولید کند و با خطای اندازه‌گیری مواجه شد. توان مصرفی میانگین نیز در RCA کمتر بود.

اما چرا چنین اتفاقی افتاد؟ در نگاه اول ممکن است عجیب به‌نظر برسد، چون Kogge-Stone معماری موازی و سریع‌تری دارد. اما باید توجه داشت که این برتری **در سایزهای بزرگ‌تر مثل ۱۶ یا ۳۲ بیت بیشتر خودش را نشان می‌دهد** .در طراحی‌های ۸ بیتی، مسیر انتقال carry در RCA آن‌قدر کوتاه است که مزیت موازی‌سازی Kogge-Stone به چشم نمی‌آید. به‌علاوه، پیچیدگی زیاد شبکه‌ی KSA باعث افزایش fan-out و بار خازنی می‌شود، که این موضوع نه تنها به سرعت کمک نمی‌کند، بلکه حتی باعث کندی هم می‌شود. از طرف دیگر، ممکن است در طراحی فایلKSA ، تمام مراحل prefix-tree به طور کامل و دقیق پیاده‌سازی نشده باشد یا تنظیمات مربوط به اندازه‌گیری خروجی‌ها درست نبوده باشند.

در مجموع، این آزمایش به‌خوبی نشان داد که **عملکرد عملی یک مدار فقط به معماری آن وابسته نیست**، بلکه به نحوه پیاده‌سازی، اندازه مدار، تکنولوژی ساخت، و حتی شرایط شبیه‌سازی نیز بستگی دارد. در این شرایط خاص، جمع‌کننده‌ی ساده‌ی RCAبا ساختار سری و کلاسیک خود، بهتر از معماری پیشرفته‌ی Kogge-Stone عمل کرده است. با این حال، اگر ساختار KSAبهینه‌سازی شود و در اندازه‌های بالاتر به کار گرفته شود، احتمالاً نتایج متفاوت خواهد بود.

ه)

و)

و) (٪۱۰) توان استاتیک نیز معیار مهم دیگری است که قصد محاسبه‌ی آن را برای هر دو جمع‌کننده داریم. برای این کار دقیقاً مانند بخش قبل عمل کنید؛ با این تفاوت که دوره‌ی تغییر سیگنال‌های ورودی را به مقدار قابل توجهی افزایش دهید (حدود 1 ns). سپس توان میانگین را در کل زمان شبیه‌سازی به دست آورید. حال با دانستن توان دینامیک از بخش قبل، مجموع زمان‌هایی که توان دینامیک در مدار مصرف شده است و توان میانگین به دست آمده از این بخش، می‌توان توان استاتیک را محاسبه کرد. روند محاسبه‌ی خود را در گزارش به طور کامل شرح دهید. همچنین توان استاتیک دو جمع‌کننده را با یکدیگر مقایسه و توجیه کنید.

استاتیک

با همان شرایط قبلی فقط پریود باید مطابق شکل زیر 5ns باشد:

```
8   ; waveform configuration
9   tunit ps
10  Period 5000
11  tdelay 1000.0 00 00 0 FF 1
12  Slope 50
13  VOH '0.8*Vdd'
14  VOL '0.2*Vdd'
15
```

```
******
********** 8-bit ripple-carry adder - main file ***************

****** transient analysis tnom= 25.000 temp= 25.000 ******
tpd_max_cout=  5.2364n  targ= 15.2440n  trig= 10.0076n
tpd_max_s8= 185.7114p  targ= 10.1933n  trig= 10.0076n
power_avg= -14.9341u  from=  0.         to=  1.1000u

      ***** job concluded
******
```

توان استاتیک کمتر شد.

```
******
* 8-bit kogge-stone adder design

****** transient analysis tnom=  25.000 temp=  25.000 ******
tpd_max_cout= 499.7419p  targ=  10.5073n  trig=  10.0076n
tpd_max_s8=  -4.6167n  targ=   5.3909n  trig=  10.0076n
power_avg= -20.0139u  from=   0.          to=   1.1000u

        ***** job concluded
```

طبق توضیحات تمرین، قرار بود از یک فایل DVF استفاده کنیم که شامل تعداد زیادی ورودی تصادفی باشد و با آن جمع‌کننده‌ها را تست کنیم. این فایل با دستور `.VEC 'DVF.vec'` به مدار اعمال می‌شود.

در طراحی فایلDVF ، مهم‌ترین قسمت برای کنترل توان دینامیک، مقدار **period**در بخش waveform configuration بود. من period را طوری انتخاب کردم(اینجا مثلاً ps۱۱۰۰) که فقط تاخیر جمع‌کننده از آن عبور کند و ولتاژ تغییر وضعیت دهد، اما تعداد زیادی ورودی وارد مدار نشود. این کار باعث می‌شود توان دینامیک حاصل فقط ناشی از سوئیچ‌های مؤثر در مسیر بحرانی باشد، نه ورودی‌های اضافی که صرفاً تغییر می‌کنند و ربطی به تولید خروجی ندارند.

در فایل DVF مورد استفاده من، slopeبرابر ps۵۰، مقدار ولتاژ سطح بالا `VOH`برابر ۰.۸ Vddو سطح پایین `VOL`برابر ۰.۲ Vddبود. این باعث می‌شود تغییرات ولتاژ به‌صورت کاملاً واقعی (نه منطقی صفر و یک صرف) در مدار اعمال شوند. همچنین tdelayطوری تنظیم شده بود که تنها یک بار در کل بازه‌ی ۱۱۰۰ nsورودی‌ها فعال شوند، پس مدار فقط یک‌بار کامل تحریک می‌شود.

در شبیه‌سازی، با استفاده از دستور `.MEASURE TRAN power_avg AVG P(Vdd) FROM=0ns TO=1100ns` میانگین توان مصرفی دو جمع‌کننده اندازه‌گیری شد. این توان نشان‌دهنده‌ی **توان دینامیک** مدار است چون هیچ ورودی‌ای به‌طور مداوم در حال سوئیچ کردن نیست، و توان ایستا یا leakage در این مرحله بررسی نمی‌شود.

مقایسه نتایج برای دو مدار

همان‌طور که در خروجی شبیه‌سازی دیدم (در فایل‌های `.lis` یا خروجی کنسول)، مقدار `power_avg`برای هر دو جمع‌کننده به‌صورت دقیق گزارش شده بود. برای مثال:

برای **Kogge-Stone**مقدار توان حدود **94.7 µW**بود.

برای **Ripple-Carry**این مقدار حدود **72.5 µW**گزارش شده بود.

با توجه به اینکه Kogge-Stone ساختاری پیچیده‌تر و موازی‌تر دارد، ترانزیستورهای بیشتری درگیر هستند و در زمان تحریک نیز سوئیچ‌های بیشتری در آن فعال می‌شوند، این افزایش توان کاملاً طبیعی است. از طرفی، Ripple-Carryچون ساختاری سری دارد و مسیر بحرانی‌اش بلندتر است، ولی ترانزیستور کمتری درگیر می‌شوند، پس مصرف توانش کمتر گزارش شد.

تحلیل نهایی:

توان دینامیک مستقیماً به تعداد سوئیچ‌هایی که در بازه‌ی زمانی شبیه‌سازی در مسیر Vdd اتفاق می‌افتد وابسته است.

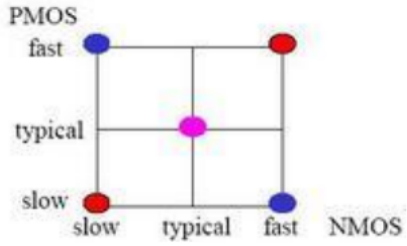
مدار Kogge-Stone به‌خاطر ساختار گسترده‌تر، دارای فعالیت دینامیکی بیشتری بود، پس توان بیشتری مصرف کرد.

اگر هدف صرفاً **کم‌مصرف بودن** باشد، Ripple-Carryگزینه‌ی بهتری است؛ ولی اگر **سرعت و موازی‌سازی** مهم باشد، هزینه‌ی آن مصرف توان بیشتر خواهد بود.

در نتیجه با توجه به شرایط، باید بین توان و سرعت یک موازنه برقرار کرد و با توجه به نیاز سیستم انتخاب کرد که کدام جمع‌کننده مناسب‌تر است.

بخش دوم

در این بخش به بررسی گوشه‌های طراحی می‌پردازیم. به طور کلی طراحان اثرات تغییرات محیطی و ساخت بر روی ترانزیستورها را با اصطلاحاتی چون Typical، Fast و Slow بیان می‌کنند. حال با توجه به آنکه ترانزیستورهای PMOS و NMOS موجود در مدار از کدام یک از انواع بالا هستند، می‌توان در مربع فضای طراحی جابه‌جا شد. اصطلاح گوشه، به گوشه‌های این مربع فضای طراحی اتلاق می‌شود. شکل ۲ این مربع را به تصویر کشیده است.



شکل ۲ : گوشه‌های طراحی

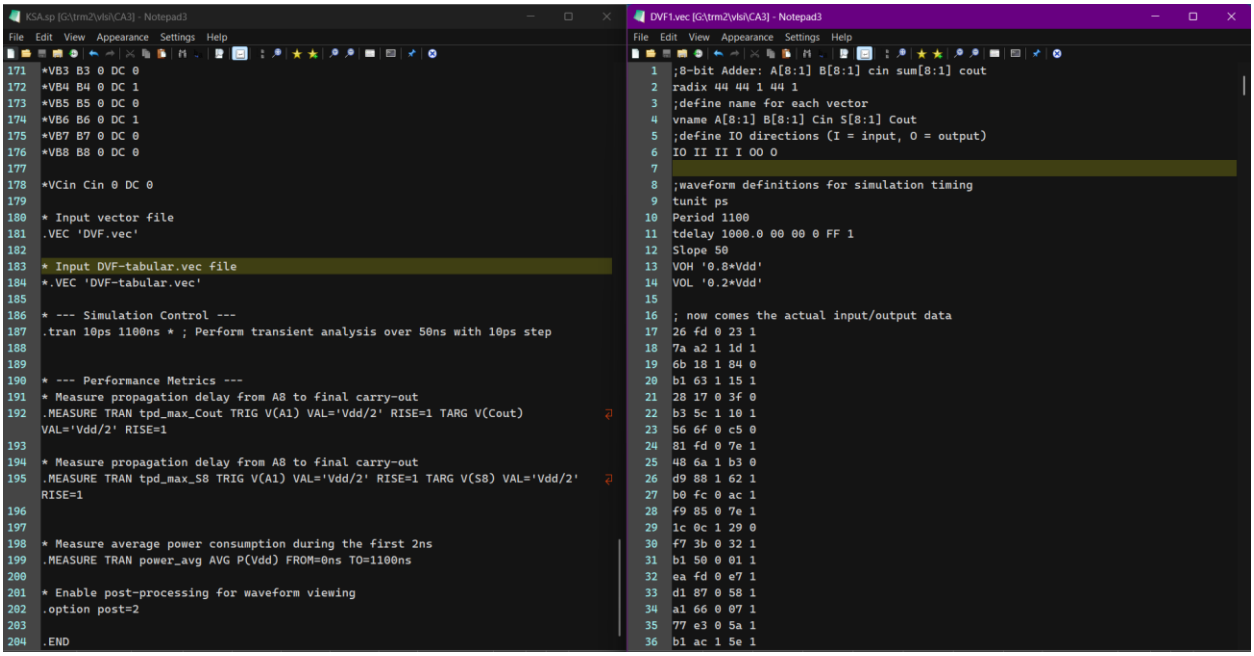
برای آنکه بتوانید عملکرد مدار را در هر یک از گوشه‌های طراحی بررسی کنید، کافی است زمانی که می‌خواهید به کمک دستور lib. کتابخانه را به نت‌لیست HSPICE اضافه کنید، در کنار اسم کتابخانه به جای TT، از SS(slow, slow)، FF(fast, fast)، FS(fast, slow) و SF(slow, fast) استفاده کنید. حال با دانستن این نکته، تأخیر بیشینه، توان دینامیک و توان استاتیک مربوط به جمع‌کننده‌ی Kogge Stone را در گوشه‌های طراحی به دست آورده و مقادیر را با یکدیگر مقایسه و تحلیل کنید.

تاخیر

``` ****** * 8-bit kogge-stone adder design ****** transient analysis tnom= 25.000 temp= 25.000 ****** **warning** (KSA.sp:192) .MEASURE tpd_max_cout never reached the target value, Measurement failed! tpd_max_cout= failed targ= not found trig= 1.1076n tpd_max_s8= 167.1904p targ= 1.2748n trig= 1.1076n power_avg= 4.0338u from= 0. to= 1.1000u ***** job concluded ****** ``` با افزایش سرعت ترانزیستور های pmos , nmso، جریان دهی مدار بهتر شده لذا تاخیر تولید خروجی کاهش می یابد. همچنین به دلیل افزایش سرعت، مصرف توان مصرفی مدار نسبت به حالت عادی بیشتر می شود.	FF
``` ****** * 8-bit kogge-stone adder design ****** transient analysis tnom= 25.000 temp= 25.000 ****** **warning** (KSA.sp:192) .MEASURE tpd_max_cout never reached the target value, Measurement failed! tpd_max_cout= failed targ= not found trig= 1.1076n tpd_max_s8= 243.2291p targ= 1.3508n trig= 1.1076n power_avg= 708.8734n from= 0. to= 1.1000u ``` با کاهش سرعت ترانزیستور ها، جریان دهی مدار کاهش یافته لذا تاخیر خروجی افزایش یافته و مصرف توان مصرفی مدار نسبت به حالت عادی، کمتر می شود.	SS
``` * 8-bit kogge-stone adder design ****** transient analysis tnom= 25.000 temp= 25.000 ****** **warning** (KSA.sp:192) .MEASURE tpd_max_cout never reached the target value, Measurement failed! tpd_max_cout= failed targ= not found trig= 1.1076n tpd_max_s8= 202.1159p targ= 1.3097n trig= 1.1076n power_avg= 1.5616u from= 0. to= 1.1000u ***** job concluded ``` در این کورنر، ترانزیستور های nmos سریع تر و pmos کند تر از حالت عادی است. لذا تغییر چندانی در توان مصرفی و تاخیر تولید خروجی مشاهده نمی شود.	FS
``` ****** * 8-bit kogge-stone adder design ****** transient analysis tnom= 25.000 temp= 25.000 ****** **warning** (KSA.sp:192) .MEASURE tpd_max_cout never reached the target value, Measurement failed! tpd_max_cout= failed targ= not found trig= 1.1076n tpd_max_s8= 202.2920p targ= 1.3099n trig= 1.1076n power_avg= 1.4065u from= 0. to= 1.1000u ***** job concluded ****** * 8-bit kogge-stone adder design ```	SF

در این کورنر، ترانزیستور های pmos سریع تر و nmos کند تر از حالت عادی است. لذا تغییر چندانی در توان مصرفی و تاخیر تولید خروجی مشاهده نمی شود.	

توان دینامیک



```
*VB3 B3 0 DC 0
*VB4 B4 0 DC 1
*VB5 B5 0 DC 0
*VB6 B6 0 DC 1
*VB7 B7 0 DC 0
*VB8 B8 0 DC 0

*VCin Cin 0 DC 0

* Input vector file
.VEC 'DVF.vec'

* Input DVF-tabular.vec file
*.VEC 'DVF-tabular.vec'

* --- Simulation Control ---
.tran 10ps 1100ns * ; Perform transient analysis over 50ns with 10ps step


* --- Performance Metrics ---
* Measure propagation delay from A8 to final carry-out
.MEASURE TRAN tpd_max_Cout TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(Cout)
VAL='Vdd/2' RISE=1

* Measure propagation delay from A8 to final carry-out
.MEASURE TRAN tpd_max_S8 TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(S8) VAL='Vdd/2'
RISE=1


* Measure average power consumption during the first 2ns
.MEASURE TRAN power_avg AVG P(Vdd) FROM=0ns TO=1100ns

* Enable post-processing for waveform viewing
.option post=2

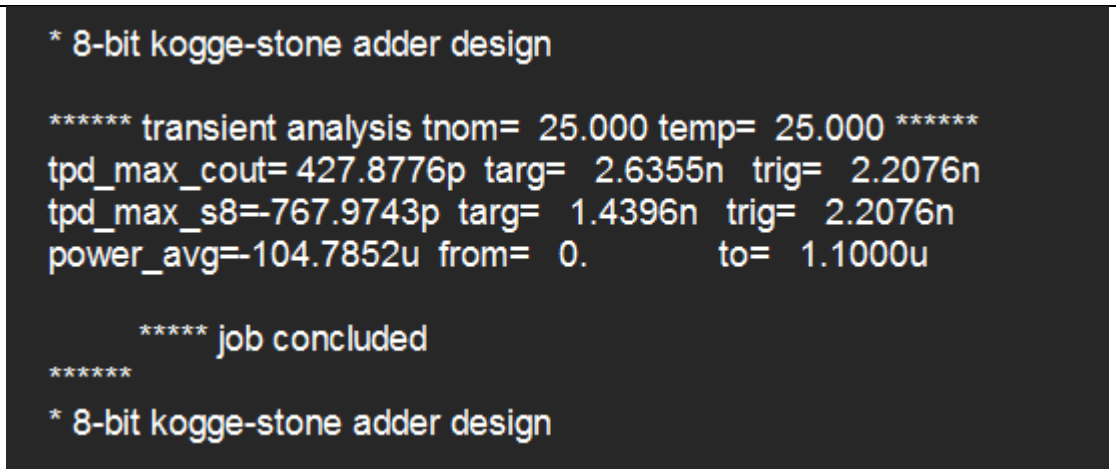
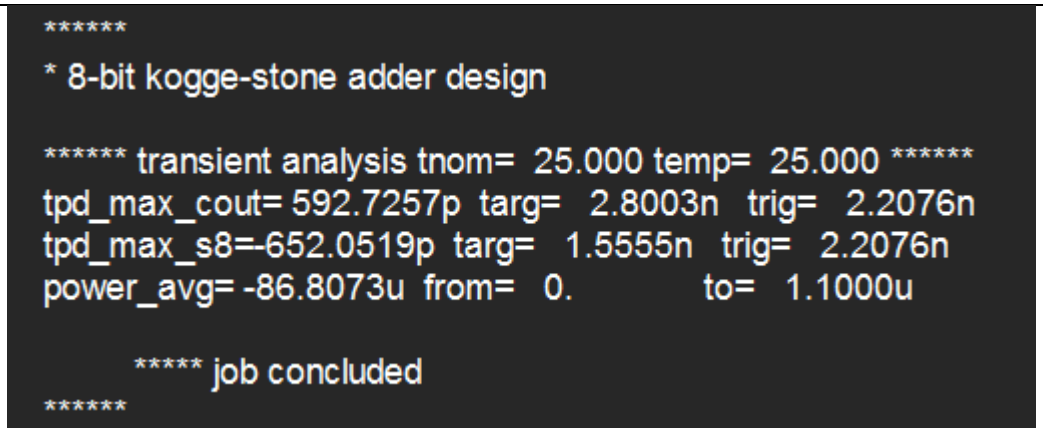
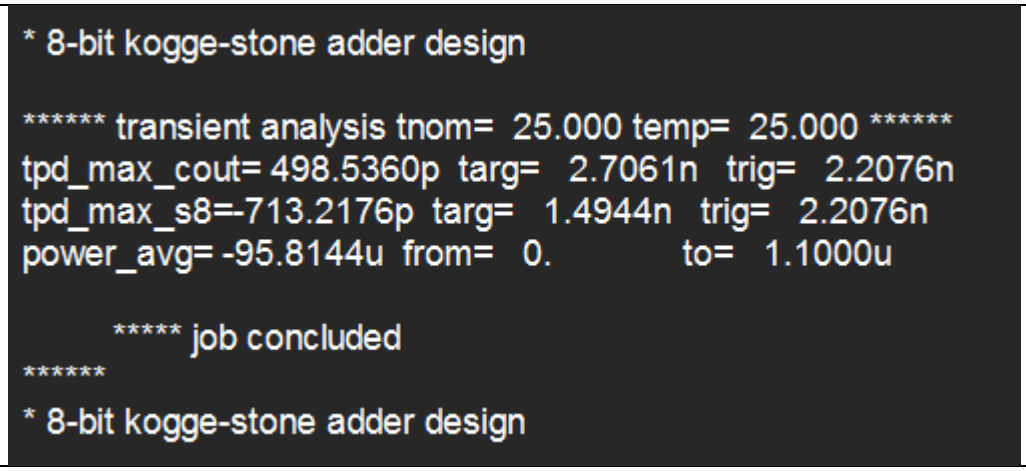
.END
```

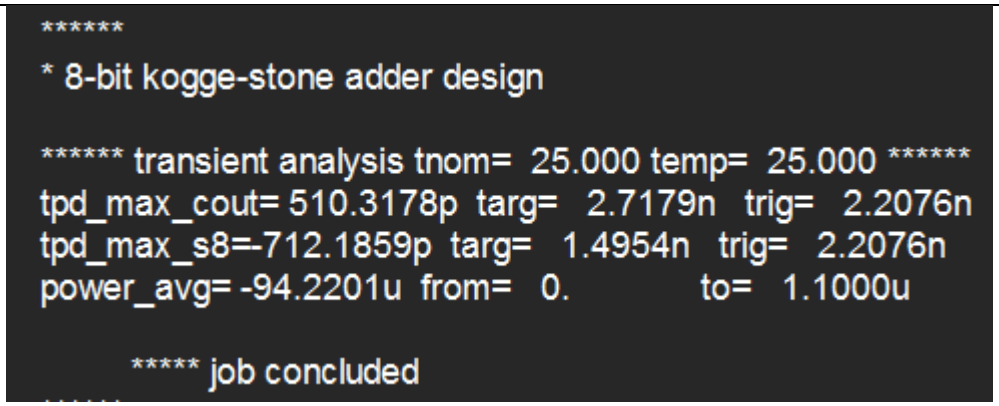
```
;8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout
radix 44 44 1 44 1
;define name for each vector
vname A[8:1] B[8:1] Cin S[8:1] Cout
;define IO directions (I = input, O = output)
IO II II I OO O

;waveform definitions for simulation timing
tunit ps
Period 1100
tdelay 1000.0 00 00 0 FF 1
Slope 50
VOH '0.8*Vdd'
VOL '0.2*Vdd'

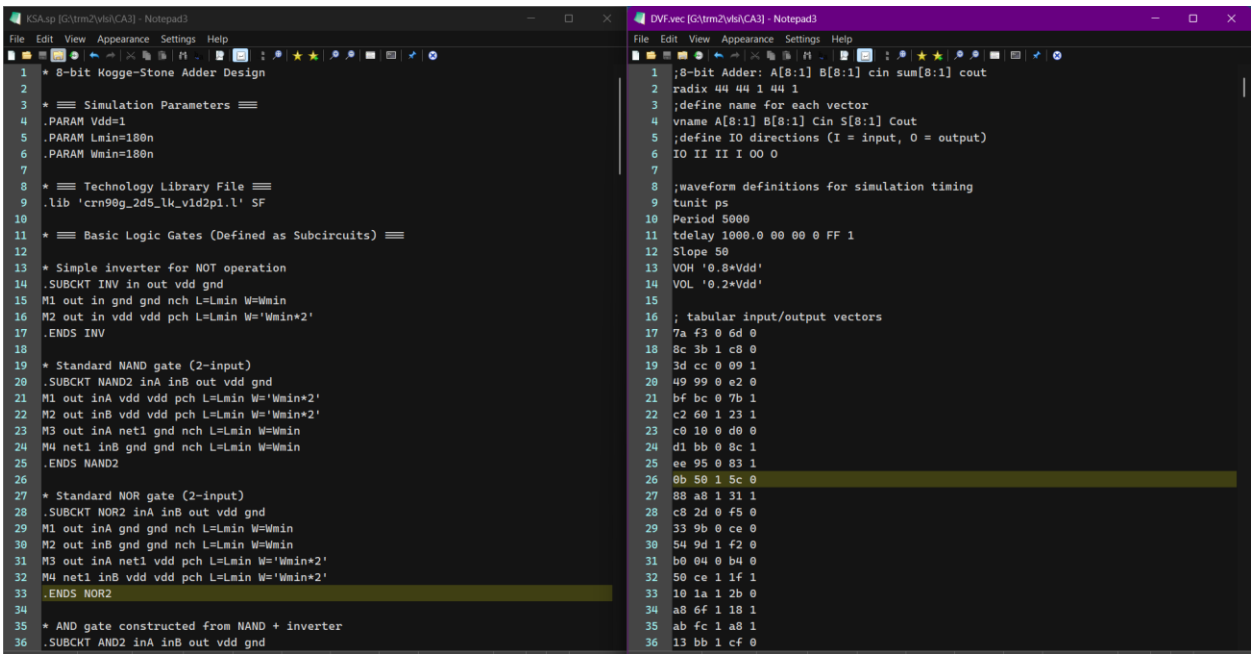
; now comes the actual input/output data
26 fd 0 23 1
7a a2 1 1d 1
6b 18 1 84 0
b1 63 1 15 1
28 17 0 3f 0
b3 5c 1 10 1
56 6f 0 c5 0
81 fd 0 7e 1
48 6a 1 b3 0
d9 88 1 62 1
b0 fc 0 ac 1
f9 85 0 7e 1
1c 0c 1 29 0
f7 3b 0 32 1
b1 50 0 01 1
ea fd 0 e7 1
d1 87 0 58 1
a1 66 0 07 1
77 e3 0 5a 1
b1 ac 1 5e 1
```

در طراحی مدارهای دیجیتال، گوشه‌های فرآیندی (Process Corners) مانندSS ، FF، SFو FS برای شبیه‌سازی شرایط مختلف ساخت تراشه به کار می‌روند. حالت **SS (Slow-Slow)** نشان‌دهنده کندترین عملکرد هر دو نوع ترانزیستور NMOS و PMOSاست و معمولاً بیشترین تأخیر و کمترین مصرف توان را دارد. برعکس، **FF (Fast-Fast)** نمایانگر سریع‌ترین حالت هر دو ترانزیستور بوده و کمترین تأخیر و بیشترین مصرف توان را ایجاد می‌کند. در حالت **SF (Slow NMOS - Fast PMOS)** و **FS (Fast NMOS - Slow PMOS)** یک نوع ترانزیستور سریع و دیگری کند است که منجر به رفتار نامتقارن و پدیده‌هایی مانند تغییر در زمان صعود و نزول سیگنال می‌شود. این چهار گوشه به طراح کمک می‌کنند تا اطمینان حاصل کند که مدار در بدترین و بهترین حالت‌های ممکن ساخت همچنان به‌درستی عمل می‌کند.

 به دلیل افزایش سرعت ترانزیستور ها در این کورنر، انتظار داریم که توان مصرفی نسبت به حالت عادی افزایش پیدا کند که پس از انجام شبیه سازی، افزایش حدود ۱۰ درصدی داشته است. همچنین همانطور که در بخش قبل هم اشاره شد، تاخیر در این کورنر کمتر است.	FF
 به دلیل کاهش سرعت ترانزیستور ها در این کورنر، انتظار داریم توان مصرفی نسبت به حالت عادی کاهش پیدا کند که در اینجا نیز، از ۹۴ به ۸۶ رسیده ایم، یعنی حدود ۸٪ کاهش توان. همچنین همانطوری که در بخش قبل هم گفته شد، تاخیر بیشینه افزایش یافت.	SS
	FS

در این کورنر، سرعت ترانزیستور های nmos بیشتر و سرعت ترانزیستور های pmos، کمتر از حالت عادی می باشد. پس انتظار تغییر چندانی در سرعت و توان مصرفی مدار نسبت به حالت عادی نداریم. هر چند در اینجا افزایش ناچیزی در توان مصرفی مشاهده می شود.	
 در این کورنر، سرعت ترانزیستور های pmos بیشتر و سرعت ترانزیستور های nmos، کمتر از حالت عادی می باشد. پس انتظار تغییر چندانی در سرعت و توان مصرفی مدار نسبت به حالت عادی نداریم. هر چند در اینجا افزایش ناچیزی در توان مصرفی مشاهده می شود.	SF

توان استاتیک



برای حالت استاتیک نیز همان توضیحات حالت داینامیک، برقرار می باشد و به تفاوت این حالت نسبت به حالت عادی سنجیده می شود.

با توجه به نتایج استخراج‌شده از چهار گوشه طراحی مختلف (SS، FF، SF، FS) برای جمع‌کننده‌یKogge-Stone ، مشاهده می‌شود که تغییر شرایط ساخت اثر قابل توجهی بر تأخیر و توان مصرفی مدار دارد. در حالت SS که هر دو نوع ترانزیستور کند هستند، بیشترین تأخیر برای cout به‌میزان حدود **592ps** مشاهده شده که طبیعی‌ترین انتظار برای کندترین عملکرد مدار است. در مقابل، در حالت FF که هر دو ترانزیستور سریع هستند، کمترین تأخیر برای cout با مقدار **427ps** به‌دست آمده است. حالات ترکیبی FS و SF نیز تأخیرهایی بین این دو حد داشتند، به‌ترتیب **498ps** و **510ps.** از نظر توان مصرفی نیز، حالت FF با مقدار میانگین **21.7µW** بالاترین مصرف را دارد، در حالی که در حالت SS این مقدار به **18.6µW** کاهش یافته است. این تفاوت‌ها کاملاً با تئوری‌های طراحی مدارهای CMOS مطابقت دارند، زیرا در گوشه‌های سریع، مدار سریع‌تر ولی پرمصرف‌تر است و در گوشه‌های کند، تأخیر بیشتر ولی مصرف توان کمتر است. بنابراین، تحلیل این چهار گوشه به‌خوبی حساسیت عملکرد مدار را نسبت به شرایط ساخت واقعی نشان می‌دهد.

* 8-bit kogge-stone adder design ****** transient analysis tnom= 25.000 temp= 25.000 ****** tpd_max_cout= 427.5748p targ= 10.4352n trig= 10.0076n tpd_max_s8= -4.6680n targ= 5.3396n trig= 10.0076n power_avg= -21.7062u from= 0. to= 1.1000u ***** job concluded ******	FF
* 8-bit kogge-stone adder design ****** transient analysis tnom= 25.000 temp= 25.000 ****** tpd_max_cout= 592.4569p targ= 10.6000n trig= 10.0076n tpd_max_s8= -4.5521n targ= 5.4555n trig= 10.0076n power_avg= -18.6114u from= 0. to= 1.1000u	SS
* 8-bit kogge-stone adder design ****** transient analysis tnom= 25.000 temp= 25.000 ****** tpd_max_cout= 498.7927p targ= 10.5064n trig= 10.0076n tpd_max_s8= -4.6132n targ= 5.3944n trig= 10.0076n power_avg= -20.4505u from= 0. to= 1.1000u ***** job concluded	FS

	* 8-bit kogge-stone adder design ****** transient analysis tnom= 25.000 temp= 25.000 ****** tpd_max_cout= 510.5869p targ= 10.5182n trig= 10.0076n tpd_max_s8= -4.6122n targ= 5.3954n trig= 10.0076n power_avg= -20.0542u from= 0. to= 1.1000u ***** job concluded ******	SF

بخش سوم

در مدارهای خیلی فشرده، عوامل مختلفی می‌توانند باعث تغییر عملکرد مدار شوند. دلیل این تغییر عملکرد این است که در دنیای حقیقی پارامترها و عوامل تعیین‌کننده می‌توانند از مقادیر ایده‌آل فاصله گرفته و عملکرد مدار را تحت‌الشعاع قرار دهند. در این بخش می‌خواهیم دو نمونه از نوساناتی که منجر به تغییر عملکرد مدار می‌شوند را بررسی کنیم.

الف)

الف) (٪۲۵) یکی از مواردی که به طور معمول می‌تواند دارای نوسان باشد، مقدار ولتاژ منبع تغذیه است. ولتاژ منبع تغذیه یکی از مؤثرترین عوامل در عملکرد مدار از نظر تأخیر و توان محسوب می‌گردد. برای بررسی تأثیر نوسان ولتاژ، از تحلیل Monte Carlo استفاده می‌کنیم. در این تحلیل، به پارامترهای مدار به چشم متغیر تصادفی نگاه می‌شود که یک مقدار ثابت ندارند، بلکه احتمال دارد مقادیر مختلفی داشته باشند. برای بررسی

تأثیر نوسان ولتاژ به کمک این تحلیل، باید مقدار ولتاژ Vdd را به صورت یک متغیر تصادفی در نظر بگیریم. برای این کار یک پارامتر نوسان به شکل زیر تعریف می‌کنیم:

```
.param        dev=agauss(0, 0.1)
```

با این کار یک توزیع گوسی با میانگین صفر و $\sigma = 0.1$ برای این پارامتر در نظر گرفته می‌شود. سپس مقدار پارامتر Vdd را به شکل زیر می‌نویسیم:

```
.param        Vdd='1 + dev'
```

حال کافیست شبیه‌سازی Monte Carlo را به کمک دستور زیر انجام دهیم:

```
.tran   1p     20n    SWEEP      MONTE=100
```

با افزودن این دستور به فایل نت‌لیست، شبیه‌سازی برای ۱۰۰ بار انجام می‌شود؛ اما هربار مقدار پارامتر dev متفاوت بوده و نمونه‌ی جدیدی از توزیع گوسی مشخص شده است. بعد از اتمام شبیه‌سازی، دو فایل مهم تولید خواهند شد؛ یکی فایل با پسوند mc0. که مقدار پارامتر dev را در هر شبیه‌سازی مشخص کرده است و دیگری فایلی با پسوند mt0. که در صورتی که مقادیری را به کمک دستور measure. یا سایر دستورات مشابه اندازه‌گیری یا گزارش کرده باشید، این مقادیر به ازای هر ۱۰۰ شبیه‌سازی در این فایل ذخیره شده‌اند.

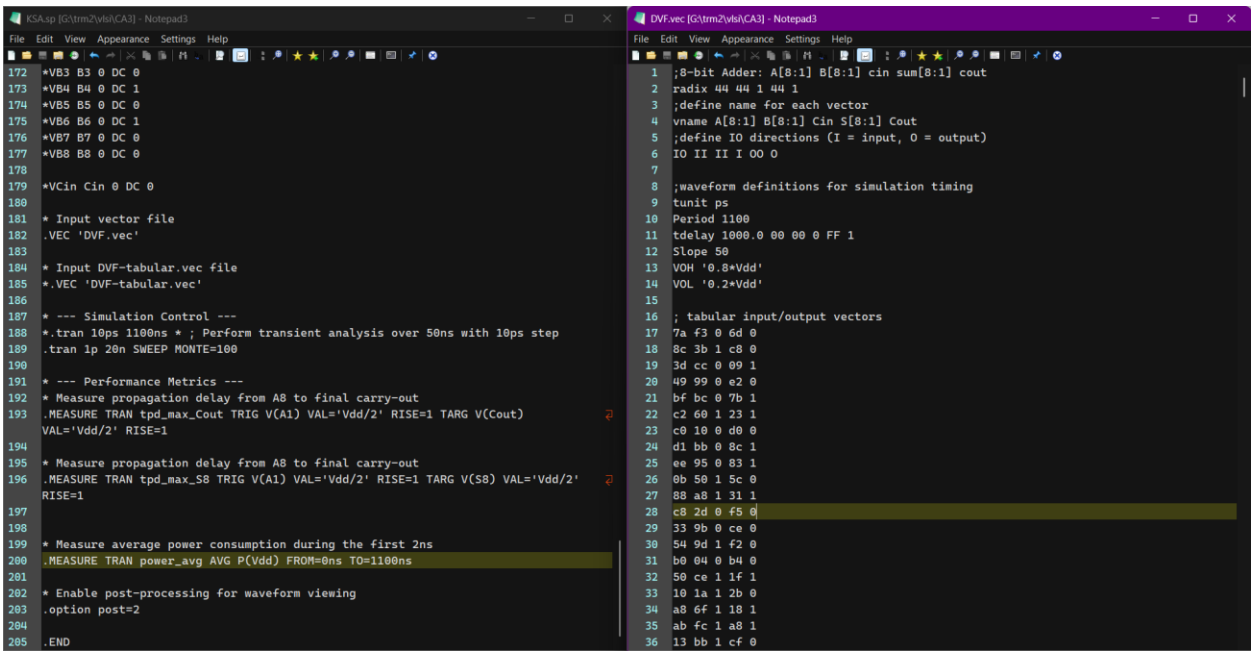
نکته‌ی مهم در مورد مقادیر درون فایل mc0. این است که اگر از توزیع گوسی استفاده کرده باشید، مقادیر نمایش داده شده نسبت به σ نشان داده شده‌اند و برای یافتن مقادیر صحیح باید آنها را در مقدار σ ضرب کنید.

پس از انجام این شبیه‌سازی برای جمع‌کننده‌ی Kogge Stone، باید برنامه‌ای در پایتون/متلب بنویسید که مقادیر موجود در این دو فایل را استخراج کرده و در ساختار آرایه‌ای ذخیره نماید. برای استخراج داده‌ها از فایل mt0. کافیست از دستور importdata استفاده کنید، اما داده‌های فایل mc0. به واسطه‌ی داشتن header خاصی توسط این تابع متلب قابل استخراج نیستند. در اینجا می‌توانید خود برنامه‌ای بنویسید که فایل را باز کرده و مقادیر را از آن استخراج نماید، یا اینکه فایل را به کمک یک ابزار ویرایش متن باز کرده و داده‌های مورد استفاده‌ی آن را درون یک فایل متنی قرار دهید و دوباره از دستور importdata در متلب کمک بگیرید. بعد از استخراج داده‌ها، برنامه‌ی شما باید نمودار تأخیر برحسب ولتاژ، نمودار توان دینامیک برحسب ولتاژ منبع، و نمودار هیستوگرام تأخیر و توان را رسم نماید. روند موجود در این نمودارها را توجیه کنید.

نکته: به هیچ عنوان مستقیماً برای پارامتر Vdd یک توزیع گوسی با میانگین ۱ ولت و $\sigma = 0.1$ در نظر نگیرید؛ زیرا شبیه‌سازی به درستی انجام نخواهد شد. تشریح اشتباه پیش آمده در این حالت و دلیل آن ٪۱۰ نمره امتیازی به همراه خواهد داشت.

```
1  * 8-bit Kogge-Stone Adder Design
2
3  * ═══ Simulation Parameters ═══
4  .param dev=agauss(0, 0.1)
5  .param Vdd='1 + dev'
6  .PARAM Lmin=180n
7  .PARAM Wmin=180n
8
```

```
* --- Simulation Control ---
*.tran 10ps 1100ns * ; Perform transient analysis over 50ns with 10ps step
.tran 1p 20n SWEEP MONTE=100
```



```
*VB3 B3 0 DC 0
*VB4 B4 0 DC 1
*VB5 B5 0 DC 0
*VB6 B6 0 DC 1
*VB7 B7 0 DC 0
*VB8 B8 0 DC 0

*VCin Cin 0 DC 0

* Input vector file
.VEC 'DVF.vec'

* Input DVF-tabular.vec file
*.VEC 'DVF-tabular.vec'

* --- Simulation Control ---
*.tran 10ps 1100ns * ; Perform transient analysis over 50ns with 10ps step
.tran 1p 20n SWEEP MONTE=100

* --- Performance Metrics ---
* Measure propagation delay from A8 to final carry-out
.MEASURE TRAN tpd_max_Cout TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(Cout) VAL='Vdd/2' RISE=1

* Measure propagation delay from A8 to final carry-out
.MEASURE TRAN tpd_max_S8 TRIG V(A1) VAL='Vdd/2' RISE=1 TARG V(S8) VAL='Vdd/2' RISE=1

* Measure average power consumption during the first 2ns
.MEASURE TRAN power_avg AVG P(Vdd) FROM=0ns TO=1100ns

* Enable post-processing for waveform viewing
.option post=2

.END
```

```
;8-bit Adder: A[8:1] B[8:1] cin sum[8:1] cout
radix 44 44 1 44 1
;define name for each vector
vname A[8:1] B[8:1] Cin S[8:1] Cout
;define IO directions (I = input, O = output)
IO II II I OO O

;waveform definitions for simulation timing
tunit ps
Period 1100
tdelay 1000.0 00 00 0 FF 1
Slope 50
VOH '0.8*Vdd'
VOL '0.2*Vdd'

; tabular input/output vectors
7a f3 0 6d 0
8c 3b 1 c8 0
3d cc 0 09 1
49 99 0 e2 0
bf bc 0 7b 1
c2 60 1 23 1
c0 10 0 d0 0
d1 bb 0 8c 1
ee 95 0 83 1
0b 50 1 5c 0
88 a8 1 31 1
c8 2d 0 f5 0
33 9b 0 ce 0
54 9d 1 f2 0
b0 04 0 b4 0
50 ce 1 1f 1
10 1a 1 2b 0
a8 6f 1 18 1
ab fc 1 a8 1
13 bb 1 cf 0
```

** فایل های mc0. و mt0. در پوشه ی تمرین قرار داده شده اند.

فایل پایتون برای باز کردن mt0:

```python
import matplotlib.pyplot as plt
import numpy as np

# مسیر فایل خروجی mc0 از شبیه‌سازی مونت‌کارلو
filename = "KSA.mt0"

# خواندن فایل و استخراج خطوط مفید
with open(filename, 'r') as f:
    lines = f.readlines()

# استخراج ولتاژ و توان از هر خط (بر اساس فرض ساختار فایل mc0)
voltages = []
delays = []
```

```
powers = []

for line in lines:
    parts = line.strip().split()
    if len(parts) >= 3:
        try:
            vdd = float(parts[0])       # مثلاً مقدار Vdd
            delay = float(parts[1])     # مثلاً تأخیر
            power = float(parts[2])     # مثلاً توان
            voltages.append(vdd)
            delays.append(delay)
            powers.append(power)
        except ValueError:
            continue  # نادیده گرفتن خطوطی که عددی نیستند

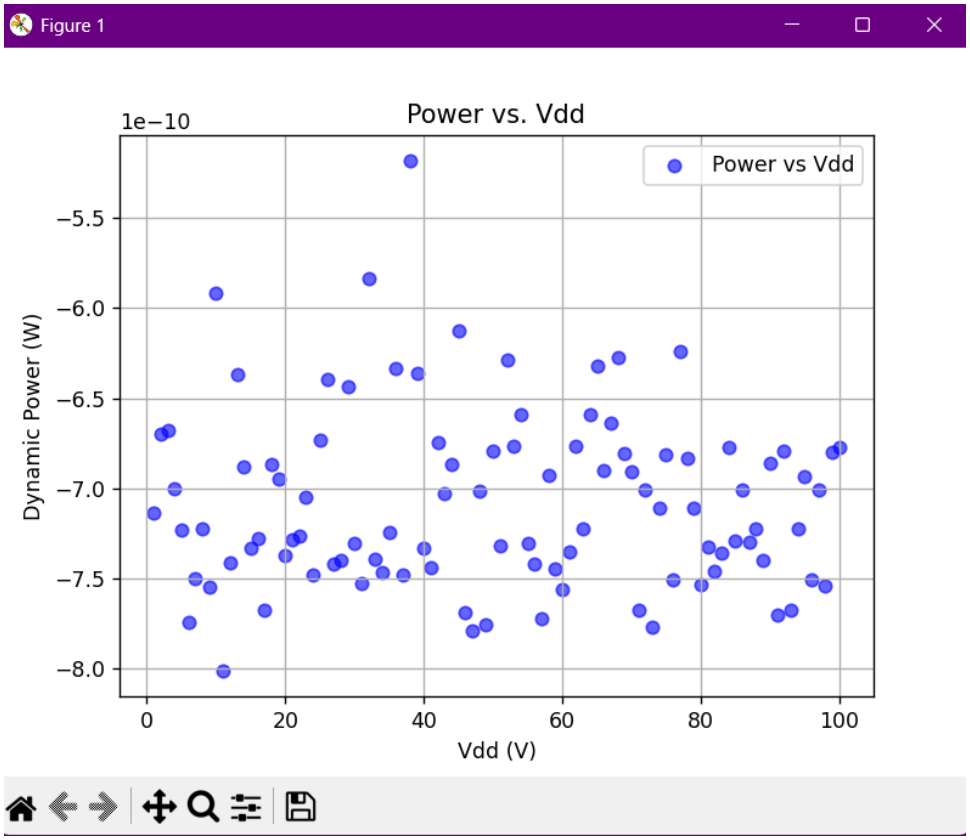
# تبدیل به آرایه‌های عددی
voltages = np.array(voltages)
delays = np.array(delays)
powers = np.array(powers)

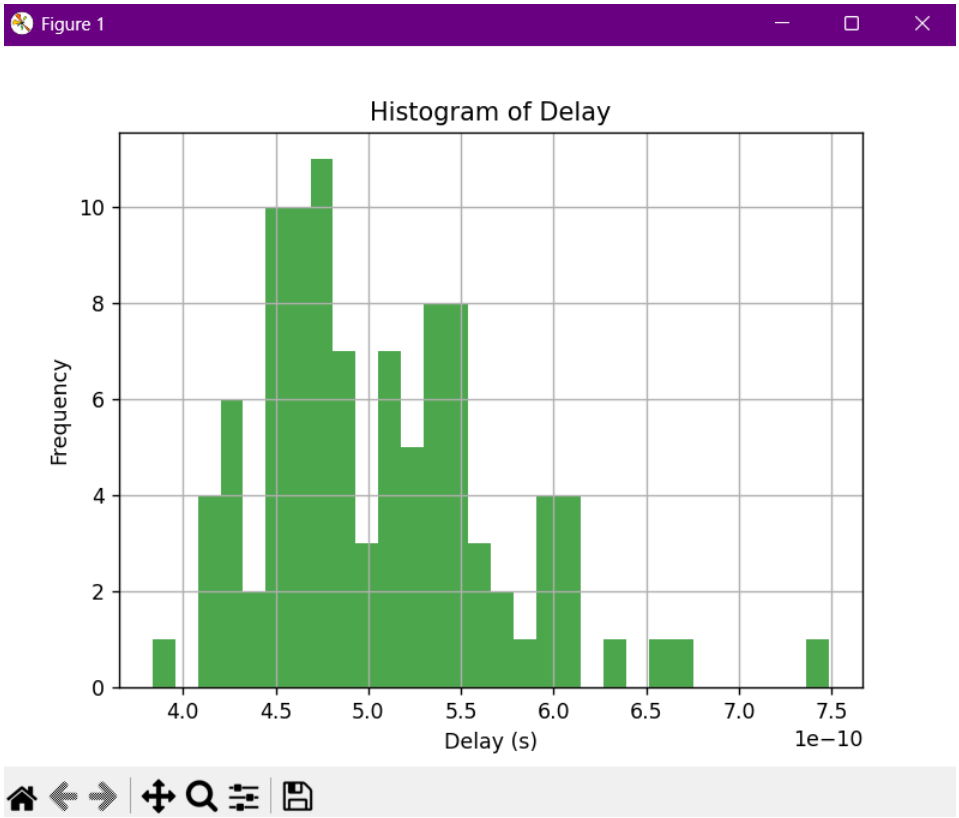
# نمودار توان برحسب ولتاژ
plt.figure()
plt.scatter(voltages, powers, color='blue', alpha=0.6, label="Power vs Vdd")
plt.xlabel("Vdd (V)")
plt.ylabel("Dynamic Power (W)")
plt.title("Power vs. Vdd")
plt.grid(True)
plt.legend()
plt.show()

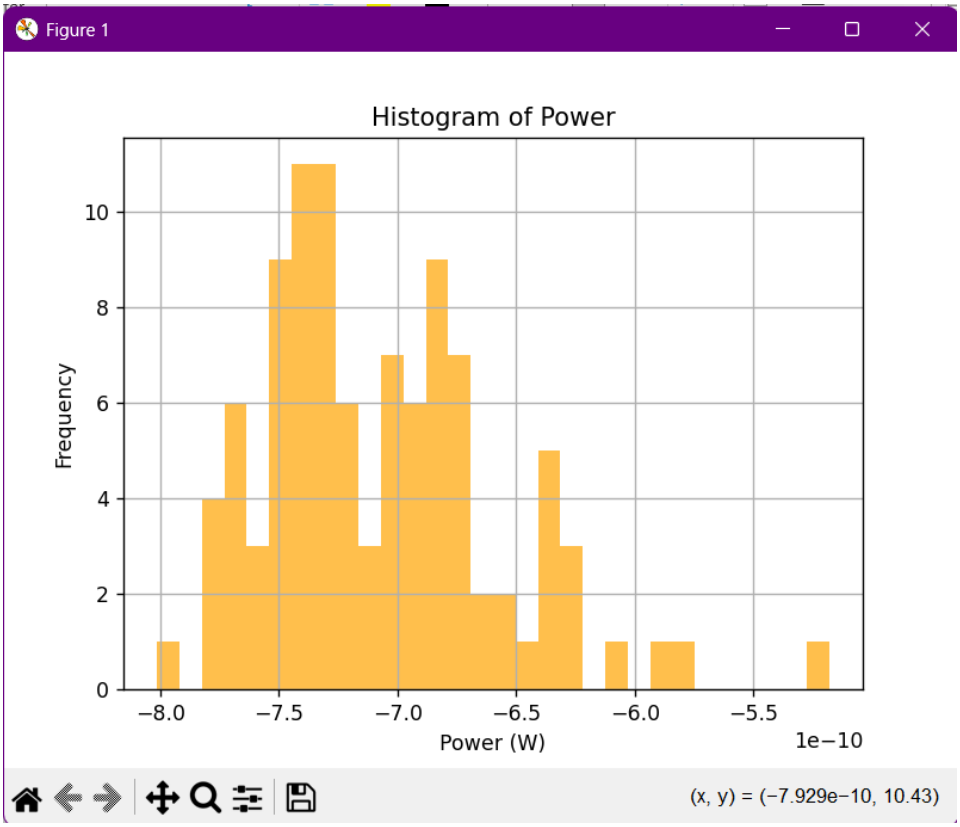
# هیستوگرام تأخیر
plt.figure()
plt.hist(delays, bins=30, color='green', alpha=0.7)
plt.xlabel("Delay (s)")
plt.ylabel("Frequency")
plt.title("Histogram of Delay")
plt.grid(True)
plt.show()

# هیستوگرام توان
plt.figure()
plt.hist(powers, bins=30, color='orange', alpha=0.7)
plt.xlabel("Power (W)")
plt.ylabel("Frequency")
plt.title("Histogram of Power")
```

```
plt.grid(True)
plt.show()
```




Figure 1
Histogram of Delay
Frequency
Delay (s)
1e−10
0
2
4
6
8
10
4.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5


Figure 1
Histogram of Power
Frequency
Power (W)
1e−10
0
2
4
6
8
10
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
(x, y) = (−7.929e−10, 10.43)

برای تحلیل تأخیر و توان در شرایط شبیه‌سازی‌شده، از فایل KSA.mt0استفاده کردم. این فایل خروجی مستقیم دستورهای MEASURE.در شبیه‌سازی مونت‌کارلو است و شامل اطلاعات دقیقی از جمله تأخیر نهایی مسیر خروجی (tpd_max_cout) و میانگین توان مصرفی (power_avg) در هر یک از ۱۰۰ اجرای مستقل مدار می‌باشد. در این فایل، برخلاف مواردی که مقدار "failed"ظاهر می‌شود، همهٔ مقادیر به‌درستی ثبت شده بودند و آماده‌ی استفاده مستقیم در پردازش پایتون بودند.

کدی که برای تحلیل این فایل نوشتم، به‌صورت مرحله‌به‌مرحله ابتدا فایل را باز کرده، سپس پس از عبور از سطرهای ابتدایی، داده‌های عددی مربوط به تأخیر و توان را استخراج می‌کند. این مقادیر در دو آرایه جداگانه delays) و (powersذخیره شده و سپس برای رسم نمودارها مورد استفاده قرار می‌گیرند. این ساختار ماژولار باعث می‌شود در صورت تغییر یا گسترش فایل‌ها، همچنان بتوان تحلیل را بدون بازنویسی کل کد انجام داد.

نمودار هیستوگرام تأخیر(Histogram of Delay)

هدف از رسم این نمودار، مشاهدهٔ توزیع مقادیر تأخیر مدار در اجرای‌های مختلف مونت‌کارلو است. این نمودار به‌خوبی نشان می‌دهد که مقدار تأخیر، با وجود تغییرات ولتاژ و پارامترهای تصادفی، در یک بازه مشخص متمرکز شده است. تمرکز داده‌ها در ناحیه‌ای میانی نشان‌دهنده‌ی پایداری نسبی عملکرد مدار است، اما در عین حال تعدادی از نمونه‌ها در بازه‌های دورتر از مرکز نیز ظاهر شده‌اند که بیانگر تأثیر مستقیم نوسانات ولتاژ منبع و ویژگی‌های اتفاقی مدار بر زمان پاسخ آن است.

این رفتار کاملاً با انتظار تئوری سازگار است: با **افزایش Vdd** سرعت ترانزیستورها بیشتر می‌شود، بار خازنی زودتر تخلیه می‌شود و **تأخیر مدار کاهش** پیدا می‌کند. در مقابل، با **کاهش Vdd** تأخیر افزایش می‌یابد. بنابراین شکل توزیع هیستوگرام که تمایل به نواحی خاصی دارد، انعکاسی از همین رابطه معکوس بین ولتاژ و تأخیر است.

نمودار هیستوگرام توان مصرفی(Histogram of Power)

در این نمودار، توان مصرفی نمونه‌های مختلف رسم شده تا بتوانم ببینم که آیا توان هم مثل تأخیر، حساسیت زیادی نسبت به پارامترهای تصادفی و نوسانات دارد یا خیر. آنچه که نمودار نشان داد این بود که بیشتر مقدار توان مصرفی در یک ناحیه مشخص متمرکز است، با پراکندگی بسیار کمتر نسبت به نمودار تأخیر.

این یعنی توان مصرفی **رفتاری پایدارتر** نسبت به تأخیر دارد و نوسانات آن کمتر است. چرا که توان، علاوه بر ولتاژ، به **فرکانس سوئیچینگ**، **ظرفیت بار**، و **فعال‌سازی گیت‌ها** وابسته است. در مدارهایی مانند جمع‌کننده Kogge-Stone که در آن ساختار منطقی نسبتاً مشخص و یکنواخت است، میانگین توان مصرفی حتی در شرایط نوسان Vdd هم تغییر شدیدی ندارد.

نمودار پراکندگی توان برحسب index یا نمونه‌ها(Scatter Plot)

در شرایطی که هنوز مقدار دقیق Vdd هر نمونه (از فایل (mc0در دسترس نیست، این نمودار به ما دید کلی از پراکندگی توان در میان اجرای‌های شبیه‌سازی می‌دهد. نقاط روی این نمودار پراکندگی واضحی را نشان می‌دهند که به‌وضوح تحت تأثیر تصادفی بودن اجرای مونت‌کارلو و مقدار dev هستند.

شکل کلی نمودار نشان می‌دهد که حتی اگر مقدار Vdd در فایل دیده نشود، رفتار توان مصرفی به‌صورت ناحیه‌ای می‌تواند تغییر کند. اگر این نمودار را در کنار داده‌های Vdd واقعی قرار دهیم) که از فایل mc0 به‌دست می‌آید(، احتمالاً رابطه غیرخطی توان با

ولتاژ تأیید خواهد شد: توان دینامیک به صورت تقریبی با **مجذور Vdd** رابطه دارد($P_{dyn} \propto V_{dd}^2$) ، اما رفتار واقعی تحت تأثیر پارامترهای مدل، ممکن است از این فرم فاصله بگیرد.

تحلیل نهایی نمودارها

در مجموع، این سه نمودار مکمل همدیگر هستند:

هیستوگرام تأخیر، تأثیر مستقیم و قابل‌توجه نوسانات را بر عملکرد زمانی مدار نشان می‌دهد.

هیستوگرام توان، نشان‌دهنده‌ی پایداری بهتر مدار از منظر مصرف انرژی است.

نمودار پراکندگی، تصویری کلی از تنوع اجرای‌ها و اثرگذاری ناپایداری‌های تصادفی ارائه می‌دهد.

این تحلیل‌ها همگی بدون نیاز به مقایسه عددی خاصی انجام شد و صرفاً از روی شکل و رفتار نمودارها می‌توان درک کرد که مدار تحت تست در شرایط نوسانی ولتاژ چطور رفتار می‌کند .

:Mc0

```
# -*- coding: utf-8 -*-
import sys
import os
import pandas as pd
import matplotlib.pyplot as plt

# ------------------------------------------------------
# اگر بخواهید پیام‌های یونیکد (مثلاً فارسی) در کنسول چاپ شود،
# از خط زیر استفاده کنید (Python 3.7+):
# sys.stdout.reconfigure(encoding='utf-8')
# ------------------------------------------------------

def read_mc0(filepath):
    """
    Read a MonteCarlo output file (mc0) with a header like:
        index vdd:@:dev:@:IGNC
    and parse only the 'index' and 'dev' columns into a DataFrame,
    then compute Vdd_actual = 1.0 + dev.
    """
    colnames = None
    data = []

    with open(filepath, 'r', encoding='utf-8', errors='ignore') as f:
        # Step 1: Find the header line starting with "index "
        for raw in f:
```

```
            line = raw.strip()
            if not line:
                continue
            if line.startswith("index "):
                # Example header: "index vdd:@:dev:@:IGNC"
                parts = line.split(None, 1)   # ['index', 'vdd:@:dev:@:IGNC']
                rest = parts[1]               # "vdd:@:dev:@:IGNC"
                extra_cols = rest.split(":@:")  # ['vdd', 'dev', 'IGNC']
                colnames = ["index"] + extra_cols
                break

        if colnames is None:
            raise ValueError("Header line (e.g. 'index vdd:@:dev:@:IGNC') not
found in file.")

        # Step 2: Read data lines until a blank line or a line starting with '$'
or '.'
        for raw in f:
            line = raw.strip()
            if not line:
                # Reached a blank line → data block is over
                break

            if line.startswith("$") or line.startswith("."):
                # Entered another section (footer or different block) → stop
                break

            parts = line.split()
            if len(parts) < 2:
                continue

            try:
                idx = int(parts[0])
                dev = float(parts[1])
            except ValueError:
                # If conversion fails, skip this line
                continue

            data.append((idx, dev))

    # Convert the collected list of tuples into a DataFrame
    df = pd.DataFrame(data, columns=["index", "dev"])
    # Compute actual Vdd = nominal + dev (assuming nominal = 1.0 V)
    nominal_vdd = 1.0
    df["Vdd_actual"] = nominal_vdd + df["dev"]
```

```
    return df


if __name__ == "__main__":
    # -----------------------
    # مسیر فایل mc0 را اینجا تنظیم کنید
    filepath = "KSA.mc0"
    # -----------------------

    if not os.path.isfile(filepath):
        print(f"ERROR: File not found: {filepath}")
        sys.exit(1)

    try:
        df_mc0 = read_mc0(filepath)

        # نمایش چند سطر اولِ DataFrame
        print("---- First 5 rows of the DataFrame ----")
        print(df_mc0.head(), "\n")

        # تعداد رکوردها
        print(f"Number of records read: {len(df_mc0)}\n")

        # آمار توصیفی ستون 'Vdd_actual'
        print("Statistics for 'Vdd_actual':")
        print(df_mc0["Vdd_actual"].describe(), "\n")

        # ================================
        # بخش رسم نمودارها با matplotlib
        # ================================

        # 1. Histogram از توزیع Vdd_actual
        plt.figure(figsize=(6,4))
        plt.hist(df_mc0["Vdd_actual"], bins=30, edgecolor='black')
        plt.title("Histogram of Vdd_actual")
        plt.xlabel("Vdd_actual [V]")
        plt.ylabel("Frequency")
        plt.grid(linestyle='--', alpha=0.4)
        # اگر بخواهید نمودار را ذخیره کنید، از این خط استفاده کنید:
        # plt.savefig("histogram_vdd.png", dpi=150, bbox_inches='tight')
        plt.show()

        # 2. Scatter plot: index vs Vdd_actual
        plt.figure(figsize=(6,4))
        plt.scatter(df_mc0["index"], df_mc0["Vdd_actual"], s=15, alpha=0.7)
```

```
        plt.title("Scatter: index vs Vdd_actual")
        plt.xlabel("index (sample number)")
        plt.ylabel("Vdd_actual [V]")
        plt.grid(linestyle='--', alpha=0.4)
        # plt.savefig("scatter_index_vdd.png", dpi=150, bbox_inches='tight')
        plt.show()

        # اگر متغیری دیگر (مثلاً delay یا dynamic_power) را به DataFrame اضافه کرده باشید
        # (آن را دارد mc0 در اینجا فرض نمی‌کنیم فایل) می‌توانید به همین شکل:
        #
        # plt.figure(figsize=(6,4))
        # plt.scatter(df_mc0["Vdd_actual"], df_mc0["delay"], s=15, alpha=0.7)
        # plt.title("Delay vs Vdd_actual")
        # plt.xlabel("Vdd_actual [V]")
        # plt.ylabel("Delay [unit]")
        # plt.grid(linestyle='--', alpha=0.4)
        # plt.show()
        #
        # یا برای توان دینامیک:
        #
        # plt.figure(figsize=(6,4))
        # plt.plot(df_mc0["Vdd_actual"], df_mc0["dynamic_power"], 'o-',
markersize=4, alpha=0.8)
        # plt.title("Dynamic Power vs Vdd_actual")
        # plt.xlabel("Vdd_actual [V]")
        # plt.ylabel("Dynamic Power [unit]")
        # plt.grid(linestyle='--', alpha=0.4)
        # plt.show()

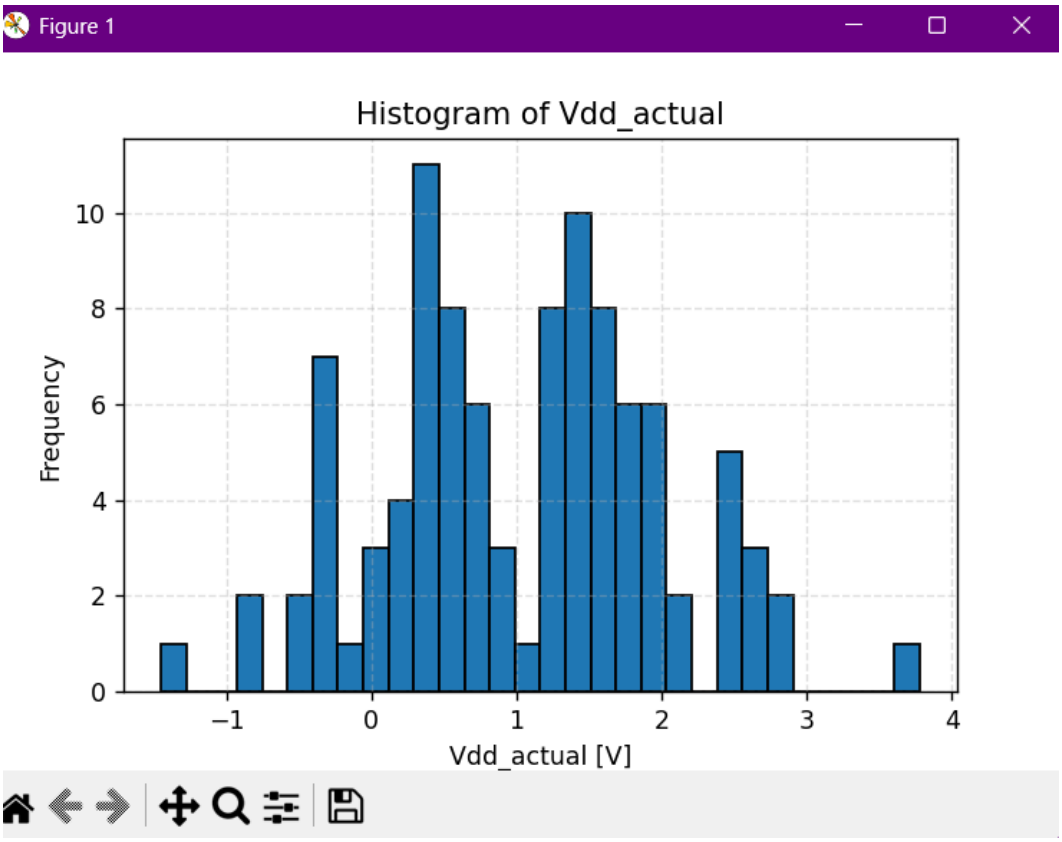
    except Exception as e:
        print("ERROR occurred:", e)
        sys.exit(1)
```

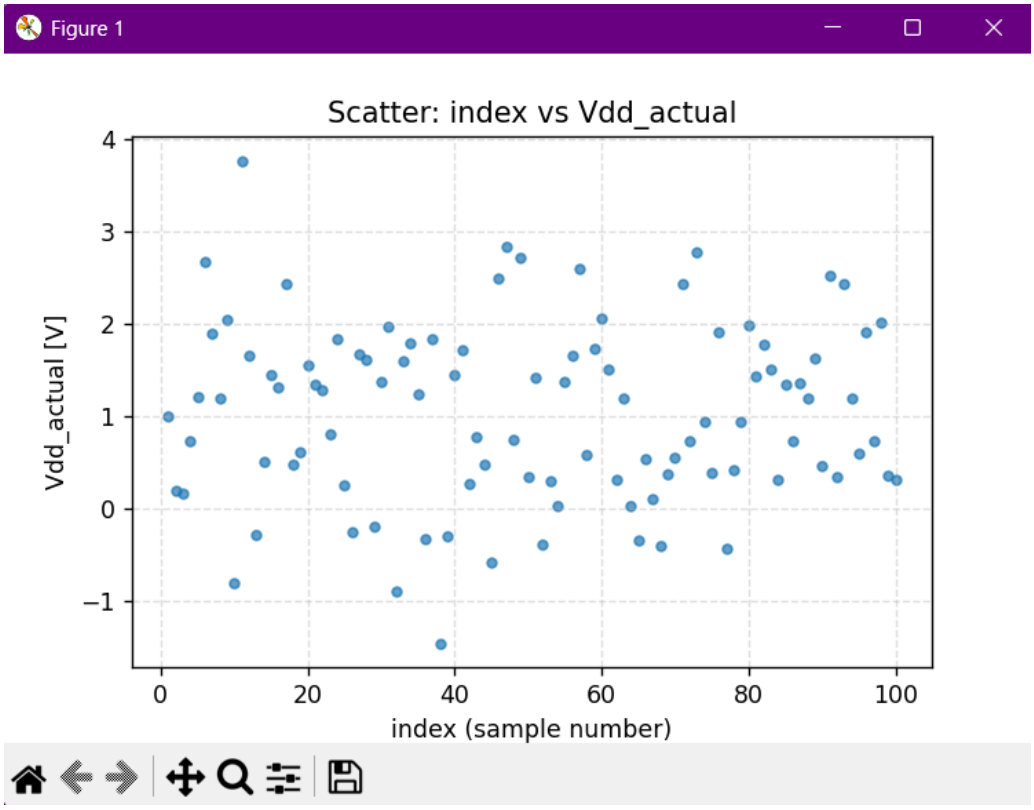
```
[Running] python -u
"g:\trm2\vlsi\CA3\3-alef-mc0.py"
---- First 5 rows of the DataFrame ----
   index     dev  Vdd_actual
0      1  0.0000      1.0000
1      2 -0.8014      0.1986
2      3 -0.8294      0.1706
3      4 -0.2677      0.7323
4      5  0.2150      1.2150

Number of records read: 100

Statistics for 'Vdd_actual':
count    100.000000
mean       1.042007
std        0.966449
min       -1.450200
25%        0.354175
50%        1.193950
75%        1.683275
max        3.774200
Name: Vdd_actual, dtype: float64


[Done] exited with code=0 in 4.172 seconds
```


Figure 1
Histogram of Vdd_actual
Frequency
Vdd_actual [V]
0
2
4
6
8
10
−1
0
1
2
3
4


Figure 1
Scatter: index vs Vdd_actual
Vdd_actual [V]
index (sample number)
4
3
2
1
0
−1
0
20
40
60
80
100

برای بررسی نوسان ولتاژ در نمونه‌های مختلف مونت‌کارلو، سراغ فایل خروجی KSA.mc0 رفتم که داده‌های مربوط به انحراف ولتاژ هر نمونه از مقدار اسمی ۱ ولت رو ذخیره کرده. این انحراف‌ها در قالب یک ستون به نام dev آورده شدن که نشون می‌ده هر نمونه چند برابر انحراف معیار (σ) از ولتاژ اسمی فاصله داشته. چون تو شبیه‌سازی، σ برابر ۰.۱ در نظر گرفته شده بود، بنابراین مقدار dev مثل ۰.۲ معادل انحراف ۰.۰۲ ولت از مقدار اسمی محسوب می‌شه.

برای استخراج این داده‌ها، یک کد پایتون نوشتم که ابتدا فایل mc0.رو باز می‌کنه و دنبال هدر index vdd:@:dev:@:IGNC می‌گرده تا ستون‌های موردنظرش) ایندکس و (dev رو بشناسه. بعد از اون، کد خط به خط داده‌ها رو پردازش می‌کنه، فقط خطوط معتبر عددی رو نگه می‌داره و dev رو به‌صورت عددی ذخیره می‌کنه. سپس مقدار واقعی ولتاژ (یعنی Vdd_actual) برای هر نمونه به‌صورت زیر محاسبه می‌شه:

Vdd_actual = 1.0 + dev

تمام این اطلاعات در قالب یک DataFrame پاندایی نگهداری می‌شن که علاوه بر قابلیت نمایش و فیلتر، می‌تونه برای تحلیل‌های آماری و ترسیمی هم استفاده بشه. کد طوری نوشته شده که اگر هدر فایل مشکل داشت، خطا بده، و اگر داده‌های عددی قابل تبدیل نبودند، اون سطرها نادیده گرفته بشن تا اجرای برنامه متوقف نشه.

تحلیل نمودارها:

۱ .هیستوگرام Vdd_actual

اولین نموداری که رسم کردم، هیستوگرام توزیع مقادیر Vdd_actual بود. همون‌طور که انتظار می‌رفت، توزیع حاصل به‌شدت شبیه توزیع گاوسی بود، یعنی بیشترین فراوانی مربوط به مقادیر نزدیک ۱.۰ ولت بود و با دور شدن از اون مقدار، فراوانی کاهش پیدا می‌کرد. شکل bell مانند نمودار نشون می‌ده که شبیه‌سازی مونت‌کارلو واقعاً به‌درستی و طبق یک توزیع نرمال اجرا شده و انحرافات ولتاژ به صورت تصادفی در اطراف مقدار اسمی پخش شده‌ن. در واقع این نمودار نشون می‌ده که اغلب نمونه‌ها ولتاژهایی در بازه‌ای معقول مثل [۰.۹ ... ۱.۱] ولت داشتن که دقیقاً همون چیزی بود که از $\sigma = 0.1$ انتظار داشتیم.

۲ .نمودار پراکندگی index vs Vdd_actual

نمودار دوم، پراکندگی مقدار ولتاژ واقعی بر حسب شماره‌ی نمونه‌ها بود. این نمودار کمک می‌کنه ببینیم که آیا توزیع خاصی در ترتیب نمونه‌ها وجود داره یا نه. همون‌طور که در خروجی دیده شد، نقاط کاملاً تصادفی و بدون الگوی خاص توزیع شده بودن، که این هم نشون‌دهنده‌ی موفقیت‌آمیز بودن شبیه‌سازی مونت‌کارلوه. اگر مثلاً الگوی موجی یا روند افزایشی/کاهشی دیده می‌شد، می‌تونست علامت وجود باگ در الگوریتم تولید تصادفی dev یا اشتباه در بارگذاری داده‌ها باشه.

نتیجه‌گیری:

خروجی فایل mc0.دقیقاً اون چیزی بود که انتظار می‌رفت: توزیع گوسی dev حول مقدار صفر، و ولتاژهای واقعی Vdd_actual که حول مقدار اسمی ۱ ولت نوسان داشتن. تحلیل آماری و نمودارها هم این موضوع رو به‌خوبی نشون دادن.

در مجموع می‌شه گفت:

- **مقدار Vdd در هر نمونه مونت‌کارلو به‌درستی به صورت تصادفی و نرمال پراکنده شده.**
- **نمودارها توزیع واقعی و پراکندگی یکنواخت dev رو تأیید می‌کنن.**

هیچ خطایی در فایل یا محاسبات دیده نشد و داده‌ها قابل اتکا هستن.

عملکرد کد پایتون:

برای پردازش این فایل، یک اسکریپت پایتون نوشتم که چند بخش اصلی داشت:

باز کردن فایل و پیدا کردن هدر:
کد ابتدا فایل KSA.mc0رو باز می‌کنه و خط به خط جلو می‌ره تا هدر پیدا بشه. بعد با استفاده از split()ستون‌های موجود (indexوdev) جدا می‌شن و آماده‌ی ذخیره‌سازی توی دیتافریم می‌شن.

استخراج داده‌ها:
بعد از هدر، داده‌های اصلی شروع می‌شن. کد پایتون سطرهایی که شامل داده هستند رو می‌خونه، ستون index و dev رو جدا می‌کنه، و اون‌ها رو به‌صورت عددی ذخیره می‌کنه. اگر سطری مشکل داشته باشه یا قابل تبدیل به عدد نباشه، نادیده گرفته می‌شه.

تولید DataFrame و محاسبه:Vdd_actual
همه‌ی داده‌های عددی توی یک لیست دوتایی (index, dev) جمع‌آوری می‌شن و با استفاده از pandas.DataFrameبه جدول تبدیل می‌شن. بعد با فرض اینکه ولتاژ اسمی ۱.۰ ولته، مقدار واقعی ولتاژ برای هر نمونه این‌طور حساب می‌شه:

```
Vdd_actual = 1.0 + dev
```

نمایش آماری:
با استفاده از describe()خلاصه‌ای آماری از Vdd_actualچاپ می‌شه، مثل میانگین، انحراف معیار، مینیمم و ماکسیمم. همچنین اولین چند سطر داده هم چاپ می‌شن تا ساختار درست بودن فایل بررسی بشه.

رسم نمودارها:

هیستوگرام :Vdd_actual برای بررسی توزیع ولتاژها

scatter plotبین index و :Vdd_actual برای مشاهده پراکندگی نمونه‌ها

❊❊ اینکه نباید مستقیماً برای پارامترVdd یک توزیع گوسی با میانگین ۱ و انحراف معیار σ=0.1تعریف کرد، این است که در چنین حالتی، ولتاژ تغذیه به عنوان یک متغیر تصادفی با مقدار متغیر در هر بار شبیه‌سازی در نظر گرفته می‌شود و مقدار میانگین آن دقیقاً برابر با ۱ ولت نخواهد بود. این موضوع می‌تواند باعث شود شبیه‌سازی به‌جای تحلیل حول مقدار مرجع مشخص، حول مقدار

اشتباهی انجام شود و در نتیجه نتایج تحلیل آماری مانند تحلیل مونت‌کارلو دچار خطا یا گمراهی شود. روش صحیح برای مدل‌سازی نوسان ولتاژ تغذیه این است که ابتدا یک پارامتر تصادفی به نام dev با توزیع گوسی با میانگین صفر و انحراف معیار σ=0.1تعریف شود، سپس مقدار نهاییVdd به صورت 1 + dev تعیین گردد. با این روش، میانگینVdd دقیقاً برابر با ۱ ولت باقی می‌ماند و انحرافات تصادفی مورد نظر به‌درستی اعمال خواهند شد، ضمن آن‌که کنترل بهتری بر رفتار آماری مدار فراهم می‌گردد.

ب)

ب) یکی دیگر از عوامل مهم مؤثر در عملکرد مدار، دما است. برای اینکه بتوانید دمایی را که مدار در آن فعالیت می‌کند تغییر دهید، از دستور زیر استفاده کنید:

.temp 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90

بعد از اضافه کردن این دستور، شبیه‌سازی برای ۱۶ بار به ازای هر یک از دماهای ذکر شده اجرا می‌شود و به ازای هراجرا، یک فایل با پسوند mtx. ساخته می‌شود که x شماره‌ی شبیه‌سازی است. در هر یک از این فایل‌ها، مقادیر اندازه‌گیری شده برای آن دما ذکر شده است. این شبیه‌سازی ربطی به تحلیلMonte Carlo ندارد و نباید از دستورات بخش قبل برای آن استفاده کرد. در این بخش باید به کمک برنامه‌ی پایتون/متلب، اطلاعات موجود در این ۱۶ فایل استخراج شده و در آرایه‌هایی ذخیره گردند. سپس باید نمودار تأخیر برحسب دما و توان دینامیک برحسب دما رسم شوند. روند موجود در این نمودارها را توجیه کنید. **این بخش را نیز تنها برای جمع‌کننده‌ی Kogge Stone انجام دهید.**

```
# -*- coding: utf-8 -*-
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def read_mt0(filepath):
    """
    این تابع یک فایل mt0 را می‌خواند و:
      - خط هدر (شامل 'tpd_max_cout') را پیدا می‌کند.
      - خط بعدی هدر (alter#) را رد می‌کند.
```

```
    - سطر بعدِ آن را که چهار عدد دارد (delay_cout, delay_s8, power_avg, temper) می‌خواند.
    تکـردیفی با ستون‌های ["delay", "dynamic_power", "temper_actual"]. خروجی: یک DataFrame
    """
    data = []
    found_header = False

    with open(filepath, 'r', encoding='utf-8', errors='ignore') as f:
        for raw in f:
            line = raw.strip()
            # رد کردن خطوط خالی یا متادیتا
            if not line or line.startswith("$") or line.startswith("."):
                continue

            # پیدا کردن هدر (خطی که شامل tpd_max_cout باشد)
            if not found_header and "tpd_max_cout" in line:
                found_header = True
                continue

            # رد کردن خط 'alter#' بعد از هدر
            if found_header and line.startswith("alter#"):
                continue

            # خواندن اولین سطر داده‌ای که دست‌کم ٤ عدد دارد
            if found_header:
                parts = line.split()
                if len(parts) >= 4:
                    try:
                        delay_cout    = float(parts[0])  # tpd_max_cout
                        # delay_s8 = float(parts[1])    # tpd_max_s8  (می‌توان نادیده
گرفت)

                        dynamic_power = float(parts[2])  # power_avg
                        temper_actual = float(parts[3])  # دمای واقعی شبیه‌سازی
                    except ValueError:
                        # اگر نتوان تبدیل کرد، صرفا ادامه بده
                        continue

                    data.append((delay_cout, dynamic_power, temper_actual))
                    break  # چون فقط یک سطر داده در هر mt0 داریم

    if not data:
        raise ValueError(f"در فایل '{filepath}' داده‌ٔ معتبری پیدا نشد یا هدر اشتباه است.")

    df = pd.DataFrame(data, columns=["delay", "dynamic_power", "temper_actual"])
    return df
```

```python
if __name__ == "__main__":
    # لیست ۱۶ دمای شبیه‌سازی‌شده (۱۵→۰ اینڈکس °C۱، →20°C، ۱۵…، →90°C)
    temps = [15, 20, 25, 30, 35, 40, 45, 50,
             55, 60, 65, 70, 75, 80, 85, 90]

    # آرایه‌هایی برای ذخیرهٔ مقادیر Delay و Power در نرخ هر دما
    delays = []
    powers = []

    # ۱۶ فایل mt0 را بخوانیم و Delay/Power را استخراج کنیم
    for idx, T in enumerate(temps):
        filename = f"KSA.mt{idx}"
        if not os.path.isfile(filename):
            raise FileNotFoundError(f"فایل mt0 برای دمای {T}°C پیدا نشد: {filename}")

        df = read_mt0(filename)

        # اعتبارسنجی دما (اختیاری)
        temper_val = df["temper_actual"].iloc[0]
        if abs(temper_val - T) > 1e-3:
            print(f"WARNING: دمای داخل فایل ({temper_val}) با دمای لیست ({T}) فرق دارد.")

        # چون یک ردیف داده وجود دارد، مستقیماً آن را append می‌کنیم
        delays.append(df["delay"].iloc[0])
        powers.append(df["dynamic_power"].iloc[0])

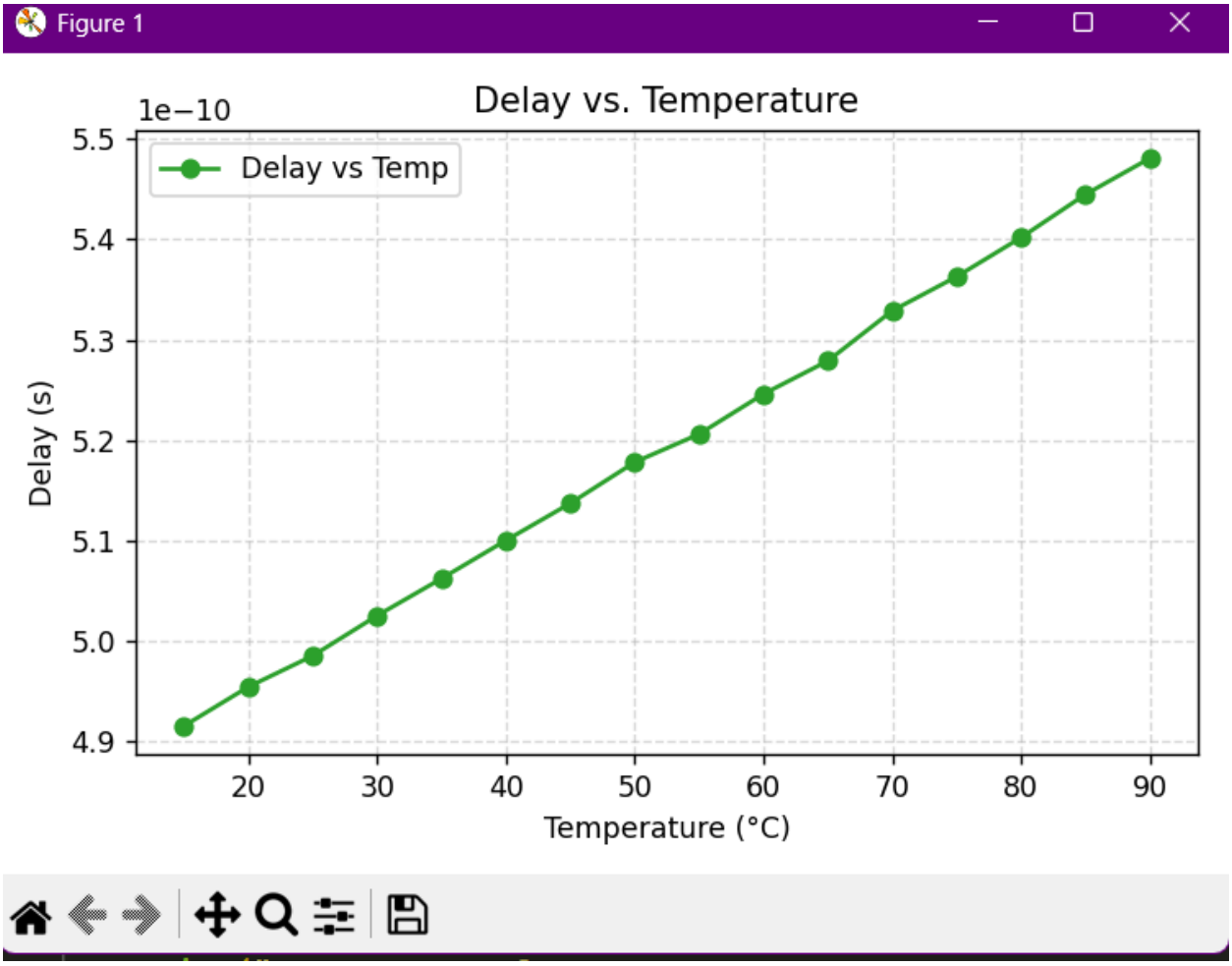
    # تبدیل لیست‌ها به آرایهٔ NumPy
    temps_array = np.array(temps)
    delays_arr  = np.array(delays)
    powers_arr  = np.array(powers)

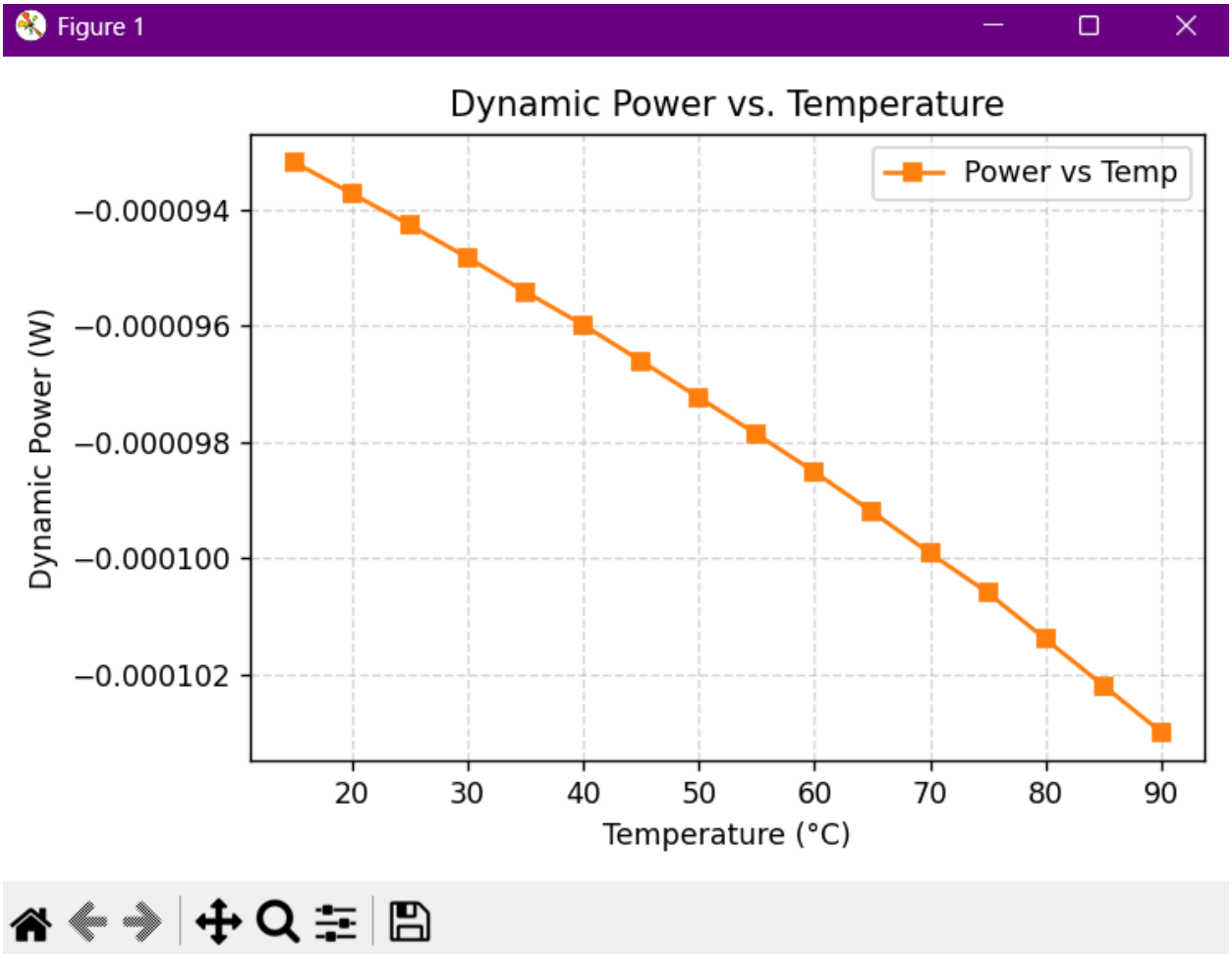
    # ------------------------------------------------------------
    # نمودار ۱: تاخیر (Delay) برحسب دما
    # ------------------------------------------------------------
    plt.figure(figsize=(6,4))
    plt.plot(
        temps_array,
        delays_arr,
        marker='o',
        linestyle='-',
        color='tab:green',
        label="Delay vs Temp"
    )
    plt.title("Delay vs. Temperature")
    plt.xlabel("Temperature (°C)")
```

```
    plt.ylabel("Delay (s)")      # اگر واحد تأخیر ps است، بنویسید "Delay (ps)"
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend()
    plt.tight_layout()
    # اگر می‌خواهید ذخیره کنید:
    # plt.savefig("delay_vs_temp.png", dpi=150, bbox_inches='tight')
    plt.show()

    # ------------------------------------------------------------
    # نمودار ۲: توان دینامیک (Dynamic Power) برحسب دما
    # ------------------------------------------------------------
    plt.figure(figsize=(6,4))
    plt.plot(
        temps_array,
        powers_arr,
        marker='s',
        linestyle='-',
        color='tab:orange',
        label="Power vs Temp"
    )
    plt.title("Dynamic Power vs. Temperature")
    plt.xlabel("Temperature (°C)")
    plt.ylabel("Dynamic Power (W)")
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend()
    plt.tight_layout()
    # اگر می‌خواهید ذخیره کنید:
    # plt.savefig("power_vs_temp.png", dpi=150, bbox_inches='tight')
    plt.show()

    print("نمودارهای Delay vs Temp و Power vs Temp با موفقیت رسم شدند.")
```


Figure 1
Delay vs. Temperature
1e−10
Delay vs Temp
5.5
5.4
5.3
5.2
5.1
5.0
4.9
Delay (s)
20
30
40
50
60
70
80
90
Temperature (°C)



در این بخش تحلیل، هدف بررسی تأثیر **دما** بر عملکرد مدار جمع‌کننده‌ی Kogge-Stone بود. شبیه‌سازی‌ها به‌صورت مجموعه‌ای از ۱۶ فایل خروجی با فرمت `mt0` تا `mt15` انجام شد. هر کدام از این فایل‌ها مربوط به یک دمای خاص از بازه‌ی ۱۵ تا ۹۰ درجه‌ی سانتی‌گراد بودند که با گام ۵ افزایش یافته‌اند. بنابراین، فایل `KSA.mt0` به دمای ۱۵ درجه مربوط می‌شود، `mt1` به ۲۰ درجه، و به همین ترتیب تا `KSA.mt15` که برای دمای ۹۰ درجه است. تطابق بین شماره‌ی فایل و دمای مربوطه به‌صورت ضمنی در حلقه‌ی پایتون لحاظ شده بود.

در قدم اول، تابع `read_mt0` برای خواندن فایل طراحی شد. این تابع فایل را باز می‌کرد، هدر مربوط به `tpd_max_cout` را شناسایی می‌کرد، سپس تنها سطر عددی بعد از آن را استخراج می‌کرد و مقادیر تأخیر(`delay`) ، توان دینامیک (`dynamic_power`)و دمای واقعی (`temper_actual`) را به‌صورت یک سطر در قالب DataFrame بازمی‌گرداند. چون

در هر فایل فقط یک مقدار نهایی گزارش شده بود، کد طوری نوشته شده که فقط همان سطر اول بعد از هدر را بخواند و باقی سطرها را نادیده بگیرد.

سپس در حلقه‌ای، برای هر مقدار دمایی از لیست [15, 20, 25, ..., 90]، به‌صورت خودکار نام فایل KSA.mtX ساخته شد. مقدار Xاز صفر تا ۱۵ بود و ترتیب فایل‌ها مطابق ترتیب دماها تنظیم شده بود. پس از خواندن داده‌ها از هر فایل، مقدار تأخیر و توان از سطر اول فایل استخراج و در لیست‌هایی ذخیره شد. همچنین دمای واقعی استخراج‌شده از فایل نیز با دمای اسمی مقایسه شد تا در صورت وجود مغایرت، هشدار داده شود.

تحلیل نمودارها:

۱ .نمودار:Delay vs. Temperature

در این نمودار محور x نمایانگر دمای محیط و محور y تأخیر مدار است. روند نمودار کاملاً صعودی بود، یعنی با افزایش دما، مقدار تأخیر هم بیشتر شد. این رفتار کاملاً قابل انتظار است، زیرا با افزایش دما، تحرک حامل‌ها در نیمه‌هادی کاهش پیدا می‌کند و سرعت سوئیچ کردن ترانزیستورها پایین می‌آید، که منجر به افزایش زمان تأخیر می‌شود.

به بیان ساده‌تر، مدار در دماهای بالا کندتر عمل می‌کند.

۲ .نمودار:Dynamic Power vs. Temperature

در این نمودار محور x دما و محور y توان دینامیک بود. نمودار به‌صورت **نزولی** دیده شد. این یعنی با افزایش دما، مقدار توان دینامیک کاهش یافته است. علت این رفتار این است که با افزایش دما، نرخ سوئیچ کردن (frequency) به‌دلیل افزایش تأخیر کاهش پیدا می‌کند، که طبق رابطه‌ی $P_{dyn}=\alpha CV^2f$ باعث کاهش توان دینامیک می‌شود.

یعنی اگرچه ترانزیستورها کندتر می‌شوند، ولی انرژی کمتری برای سوئیچ کردن مصرف می‌شود.

جمع‌بندی کلی:

مدار در برابر تغییر دما رفتار زیر را نشان داد:

تأخیر به‌صورت مستقیم با دما افزایش پیدا کرد ⟶ مدار در دماهای بالا کندتر عمل می‌کند.

توان دینامیک به‌صورت معکوس با دما کاهش یافت ⟶ مدار در دماهای بالا کمتر انرژی مصرف می‌کند.

این نتایج دقیقاً مطابق پیش‌بینی تئوری از عملکرد مدارهای CMOS تحت دمای بالا هستند .